

OpenText™ AppWorks Platform

Cloud Deployment Guide

This guide gives an overview of how to deploy images and applications using AppWorks Platform.

BPAAWP230400-ICD-EN-1

OpenText™ AppWorks Platform
Cloud Deployment Guide
BPAAWP230400-ICD-EN-1
Rev.: 2023-Sept-21

This documentation has been created for OpenText™ AppWorks Platform CE 23.4.

It is also valid for subsequent software releases unless OpenText has made newer documentation available with the product, on an OpenText website, or by any other means.

Open Text Corporation

275 Frank Tompa Drive, Waterloo, Ontario, Canada, N2L 0A1

Tel: +1-519-888-7111

Toll Free Canada/USA: 1-800-499-6544 International: +800-4996-5440

Fax: +1-519-888-0677

Support: <https://support.opentext.com>

For more information, visit <https://www.opentext.com>

Copyright © 2023 Open Text. All Rights Reserved.

Trademarks owned by Open Text.

One or more patents may cover this product. For more information, please visit <https://www.opentext.com/patents>.

Disclaimer

No Warranties and Limitation of Liability

Every effort has been made to ensure the accuracy of the features and techniques presented in this publication. However, Open Text Corporation and its affiliates accept no responsibility and offer no warranty whether expressed or implied, for the accuracy of this publication.

Table of Contents

1	Containerization	9
1.1	Benefits of containerization	10
1.2	Different artifacts in the containerization domain	10
2	AppWorks Platform containerization overview	13
2.1	Application image	13
2.2	AppWorks Platform containerization benefits	14
3	Container archive contents	15
3.1	Application-specific artifacts	17
4	Deployment requirements	19
4.1	Software requirements	19
4.2	Hardware requirements	19
4.3	Docker	20
4.4	Kubernetes	20
5	Understanding the configuration files	23
5.1	Configuration files	23
5.2	Externalized configuration	24
5.3	Configuration files for Docker	31
5.3.1	.env	32
5.3.2	docker-compose.yml	32
5.4	Configuration files for Helm charts	32
5.4.1	azure.yaml	41
5.4.2	gcp.yaml	43
5.4.3	aws.yaml	46
5.5	Application-specific configuration to the image	48
5.5.1	Direct in-container editing of solution.yml	49
5.5.2	Copying solution.yml into a container	49
5.5.3	Copying the solution.yml file in a container through the copy-to-image folder	50
5.6	Licensing	51
5.6.1	Obtaining a license	51
5.6.2	Applying a license when using Docker	52
5.6.3	Applying a license when using Kubernetes Helm chart	53
5.6.4	Renewing the AppWorks Platform license	53
6	Building the Application image	57
6.1	Extend the existing AppWorks Platform or application image to build a new application image	58
6.2	Build a separate image for the application using an init container.	59

7	Deploying Application images on Docker	61
7.1	Prerequisites	61
7.2	Deployment of default containers in Docker	61
7.3	Advanced deployment of Applications in Docker	62
7.4	Using a specific database	63
7.5	Accessing AppWorks Platform	67
7.6	Stopping containers	67
7.7	Removing containers	67
7.8	Accessing log files	67
8	Deploying Application images on Kubernetes	69
8.1	Prerequisites	69
8.2	Deploying Application images in Kubernetes	69
9	Deploying Application images on Microsoft Azure	73
9.1	Prerequisites	73
9.2	Deploying application images on Azure	74
9.2.1	Pushing the Application and OpenLDAP images to the Azure container registry	74
9.2.2	Deploying the Application image on Azure	75
10	Deploying Application images on Google Cloud Platform ..	79
10.1	Prerequisites	79
10.2	Pushing the Application and OpenLDAP images to Google Container Registry	80
10.3	Deploying the Application image on Google Cloud Platform	80
11	Deploying Application images on Amazon Web Services ..	83
11.1	Prerequisites	83
11.2	Pushing the Application and OpenLDAP images to ECR	84
11.3	Deploying the Application and OpenLDAP images on Amazon Web Services	84
12	Deploying Application images on Red Hat OpenShift	89
12.1	Prerequisites	89
12.2	Deploying application images on Red Hat OpenShift	90
12.2.1	Pushing the application and OpenLDAP images to Image Stream	90
12.2.2	Deploying the Application image on OpenShift	91
13	Deploying applications to an organization	93
14	Configuring OTDS with AppWorks Platform	95
14.1	Configuring AppWorks with OTDS	95
14.1.1	Configuring OTDS resource in AppWorks Platform	95
14.1.2	Consolidating OTDS users with AppWorks Platform	96

14.2	Installing and configuring OTDS	96
14.2.1	Installing OTDS	96
14.2.1.1	Installing OTDS using Docker	96
14.2.1.2	Installing OTDS in Kubernetes through Helm charts	99
14.2.2	Configuring OTDS	99
14.2.2.1	Configuring partitions and users	99
14.2.2.2	Configuring access roles for users	100
14.2.2.3	Configuring resources	100
15	MicroProfile Health	105
15.1	Health check probes	105
15.1.1	Readiness probe	105
15.1.2	Liveness probe	113
16	Administration tasks	121
16.1	Displaying hidden tasks in the containerized mode	121
16.2	Signing application packages	121
16.3	Running command line tools in the containerized mode	122
16.4	Changing AppWorks Platform log levels and writing logs to file for backup	123
16.5	Changing AppWorks Platform log levels	124
16.6	Configuring HTTPS in Kubernetes	124
16.7	Configuring TomEE	124
16.7.1	Securing a TomEE deployed application	124
16.8	Enabling CORS	124
16.9	Enabling Proxy SSL offloading	124
16.10	Enabling debug tools in a container	125
16.11	Enabling the PSL upgrade feature	125
16.12	Disabling TLS for AppWorks Platform and OpenLDAP communication	126
16.12.1	Enabling OpenLDAP to expose TLS disabled connection	126
16.12.2	Switch Appworks to connect to OpenLDAP on TLS disabled port	127
16.12.3	Enable TLS disabled connection between multiple OpenLDAP pods or containers	128
16.13	Enabling detailed error messages	128
17	Migrating from classic to containerized deployment	129
17.1	Migration overview	129
17.2	Prerequisites to migrate to the containerized deployment	129
17.3	Migration tasks	131
17.3.1	Preparing the Docker image	131
17.3.2	Deploying AppWorks Platform with applications	132
17.3.2.1	Deploying AppWorks Platform with applications on Docker	132
17.3.2.2	Deploying AppWorks Platform with applications on Kubernetes	132

17.4	Post migration tasks	132
18	Updating the Application container images	135
18.1	Updating an application or its configuration to a new version	135
18.2	Updating AppWorks Platform to a new version	135
18.2.1	Building a new image with application packages	139
18.3	Updating AppWorks Platform on Docker	141
18.4	Updating the Application image on Kubernetes using Helm charts	142
18.4.1	Updating AppWorks Platform 20.2, 20.3, and 21.3 and later versions to 23.4 using Helm charts	142
18.4.2	Updating AppWorks Platform 20.4, 21.1, and 21.2 to AppWorks Platform 23.4 using Helm-charts	144
19	Appendix A – MicroProfile Configuration	147
19.1	MicroProfile properties configuration	147
19.1.1	YAML files	147
19.1.2	wcp.properties	148
19.1.3	Environment variables	148
19.1.4	Secrets in a vault	148
19.2	MicroProfile properties	150
19.2.1	Database connection	150
19.2.2	Datasource properties	152
19.2.3	Business Process Management	155
19.2.4	Notification	161
19.2.5	Scheduling	168
19.2.6	RPA	169
19.2.7	Rule Management	170
19.2.8	PIMQuery Connector	171
19.2.9	ArchiveApplicationConnector	171
19.2.10	Lifecycle building block	172
19.2.11	EIS Connector	172
19.2.12	Email configuration	174
19.2.13	CWS settings	177
19.2.14	FTP Connector	181
19.2.15	Email connector	189
19.2.15.1	Classic	190
19.2.15.2	Classic (OAuth)	197
19.2.15.3	MS Graph with Application type Authentication	202
19.2.15.4	MS Graph with Delegated type Authentication	205
19.2.16	UDDI connector	209
19.2.17	HTTP connector	212
19.2.18	Magellan/iHub	217
19.2.19	Audit	218

19.2.20	Security Administration	219
19.2.21	XForms Connector	221
19.2.22	Google Maps	222
19.2.23	AppWorks Gateway(Mobile App)	222
19.2.24	XDS	223
19.2.25	Application package	224
19.2.26	XML Store	225
19.2.27	Tag	225
19.2.28	Task	226
19.2.29	Preference	226
19.2.30	WS AppServer	227
19.2.31	Email Building Block	228
19.2.32	File Building Block	228
19.2.33	Content Building Block	230
19.2.34	Business Workspace	231
19.2.35	instanceRoleToTeamRoleMap	233
19.2.36	Authenticator	233
19.2.37	Gateway	235
19.2.38	Licensing	236
19.2.39	OTDS	237
19.2.40	Security	238
19.2.41	Application package deployment	241
19.2.42	Auto logout	242
19.2.43	Configuring error behavior in item layouts	243
19.2.44	Auditing	243
19.2.45	LDAP	245
19.2.46	Generic service deployment	246
19.2.47	MicroProfile Configuration metadata	248
19.2.48	Document viewer	249
19.2.49	Platform Repository	251
19.2.50	Content Server	252
19.2.51	Content Server with Extended ECM	255
19.2.52	Documentum	257
19.2.53	Core Share	260
19.2.54	Archive Center	262
19.2.55	CMIS	263
19.2.56	AppEnhancer	264
19.2.57	Others	265
19.2.58	CORS	266
19.2.59	Content Security Policy (CSP)	267
19.2.60	Organization Creation	268
19.2.61	Core Signature	268

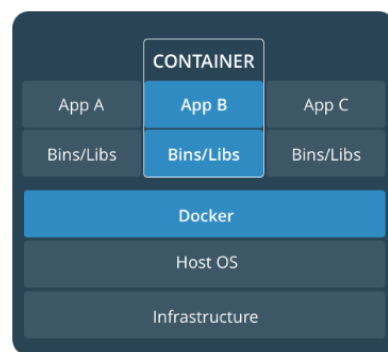
19.2.62	Core Content	270
20	Appendix B – Disaster recovery	275
20.1	Prerequisites	275
20.2	Recovering the production site to the disaster recovery site	275
21	Appendix C – Firewall issues when using Docker on Linux	277
21.1	Executing firewall commands	277
21.2	Verifying that masquerading is successful	277
22	Appendix D – Deploying applications with External Secrets Operator	279
22.1	Configuring AppWorks Platform to use Vault	279
22.2	Configuring AppWorks Platform to use AWS Secrets Manager	280
22.3	Configuring AppWorks Platform to use Azure Key Vault	281
22.3.1	Configuring AppWorks Platform to use GCP Secret Manager	283
23	Appendix E – Setting up High Availability with LDAP pods	285
23.1	Fresh installation with LDAP in High Availability mode	285
23.2	Setting LDAP High Availability on an existing AppWorks Platform Instance	286
23.2.1	Setting up HA after upscaling the pods	286
23.2.2	Upscale AppWorks to required number of pods	287
24	Appendix F— Regenerating the expired LDAP certificates in the container	289

Chapter 1

Containerization

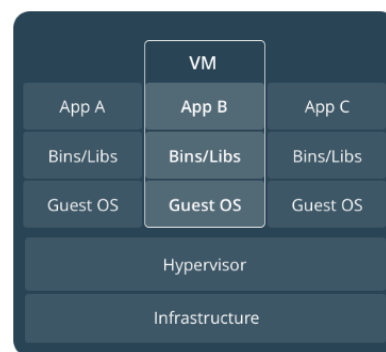
Containerization is a process to deploy applications. On a high level, a container resembles a virtual machine because you can run multiple containers on a single server. However, unlike a virtual machine, a container does not have a complete operating system (OS) in it but uses the operating system of the host server. A container image is a package of the operating system, any application server and libraries that may be needed, and the application, bundled together. It contains all the dependent components required for the application to run. Docker is a containerization product, which allows you to run containers. See also <https://www.docker.com/>.

Containerization allows OpenText and its customers to run applications developed in AppWorks Platform in a containerized environment, such as your own Kubernetes environment or the OpenText cloud.



CONTAINERS

Containers are an abstraction at the app layer that packages code and dependencies together. Multiple containers can run on the same machine and share the OS kernel with other containers, each running as isolated processes in user space. Containers take up less space than VMs (container images are typically tens of MBs in size), and start almost instantly.



VIRTUAL MACHINES

Virtual machines (VMs) are an abstraction of physical hardware turning one server into many servers. The hypervisor allows multiple VMs to run on a single machine. Each VM includes a full copy of an operating system, one or more apps, necessary binaries and libraries - taking up tens of GBs. VMs can also be slow to boot.

The above image has been sourced from the Docker website. For more information, see www.docker.com (<https://www.docker.com/>).

1.1 Benefits of containerization

Containerization is about cost reduction, automation, and predictability. The following are other advantages of using containers:

- Simplified deployment, due to faster installations and simplified configurations. Deployment time reduces significantly. The AppWorks Platform container preview can be built and run in 15 minutes or less.
- Consistent deployment results because the same image is reused multiple times. This also means that the same container image is used in test and production.
- Better use of available server resources due to additional shared usage of resources with containers and by running multiple different containers on a single server.
- Standardization by running in CF Container Runtime managed by Kubernetes to enable capabilities, such as automatic deployment, scaling, and self-healing.

1.2 Different artifacts in the containerization domain

Container

A Docker container is a running instance of a Docker image. The Docker run command is used to create a Docker container and run it.

Dockerfile

A Dockerfile includes the definition or the commands required to create a Docker image, for example, a command that enables copying files into the images. The Docker build command builds an image from a Dockerfile.

Helm-charts

Kubernetes can be managed through the tool `kubectl` and through Helm. Helm uses helm-charts. These helm-charts can contain Kubernetes objects. A helm-chart usually contains a deployment and a service, but often also contains an ingress, persistent volume claims, and secrets. Helm-charts describe the configuration and relationship between the different Kubernetes objects.

Image

A Docker image is a package that contains all components needed to run. It can be considered as a template for the actual containers that are created from an image.

Kubernetes

It is common to run a set of multiple containers together and scale up and down the number of containers depending on the load of your application. Kubernetes is an open-source platform originating from Google that helps with the management of containers and related services.

To run an AppWorks Platform multi-node cluster of containers, Kubernetes can be used to set up and manage this cluster. See <https://kubernetes.io/>. Kubernetes must be configured such that the AppWorks Platform container can run with StatefulSets. See <https://kubernetes.io/docs/concepts/workloads/controllers/>

[statefulset/](#). This results in the content of the containers to remain the same after a restart and also, the hostname does not change in this case.

Registry

A Docker registry stores Docker images. There are public registries, such as Docker Hub and Docker Cloud, and private registries, which you can host yourself. The Docker pull or Docker run command can be used on Docker registries. The Docker push command pushes a Docker image to a registry.

Volume in Docker

A Docker volume contains files that can be shared between multiple containers. Volumes can maintain data after a container is deleted and recreated.

Volume in Kubernetes

The files created in a container become ephemeral or volatile after it is started. Therefore, when a container becomes unresponsive or restarts, these files are cleaned up and lost. In some scenarios, these files must persist. Often, it might be necessary to share files between the containers in a pod. Volumes in Kubernetes enable you to achieve both these objectives. At its core, a volume is just a directory, possibly with some data in it, which is accessible to the containers in a pod. Kubernetes supports different types of volumes, depending on the storage type such as local, nfs, azuredisk, and persistentVolumeClaim.

The AppWorks container uses three volumes:

- node specific volume - stores files related to a node, for example, the log files related to a node
- cluster specific volume - stores files of a cluster that are shared across containers, for example, the application packages (CAPs)
- organization specific volume - stores files that are specific to an organization, for example, web content of an application package deployed to an organization

Each of these volumes have a default size, which can be modified by editing the persistence group in the `values.yml` (for Helm-based deployment) file, if required. Administrators must provide storageclass based on the storage type used. However, they are not allowed to change any other properties of the volumes, such as name or accessmode.

Chapter 2

AppWorks Platform containerization overview

A set of build files are for AppWorks Platform that can be used to create your own application-specific container image. This image can then be used to run the application on Docker or Kubernetes. The build files set comprises a Dockerfile, docker-compose files, build and initialization scripts, and Kubernetes and helm-chart examples. By default, a build produces the following containers, which can then be run together:

LDAP

A classic AppWorks Platform deployment uses CARS (OpenLDAP) as the LDAP server. For containers, OpenText also supports OpenLDAP as the LDAP server. The OpenLDAP image is now available in the OpenText registry.

Database

The container build files by default use PostgreSQL as the database server. PostgreSQL is an open-source database supported by AppWorks Platform. MS SQL Server and Oracle can also be run in a container. The Azure SQL database is available as a service in the Microsoft Azure environment.

AppWorks Platform

The default AppWorks Platform container that has the standard service containers. The AppWorks Platform image is now available in the OpenText registry.

2.1 Application image

Business solutions are developed using AppWorks Platform containers and are re-imaged. To create an application image, administrators apply the custom application packages (business solutions) on the AppWorks Platform image and build it again. These images are called Application images. When these Application images are deployed, the AppWorks Platform along with the business solutions are deployed at the location. See [“Building the Application image” on page 57](#) for more details.

2.2 AppWorks Platform containerization benefits

Running AppWorks Platform in a containerized environment provides several advantages. The deployment and configuration of AppWorks Platform are simplified, which results in no more time-consuming installations. The configuration is externalized and can be specified in YAML files. This configuration contains the service containers to be created. With smart-defaults approach, AppWorks Platform automatically attaches web services to the applicable service containers, thereby reducing additional administration steps.

Deployments are consistent when using a container image with the complete application and configuration in it. Use the same container image in test, acceptance, and production, and have different endpoints in the configuration per environment. From AppWorks Platform 23.2, all service containers run in TomEE, which results in better use of available server resources. By using a standard container orchestrator tool, such as Kubernetes, there are capabilities, such as simple up and down scaling, self-healing by using health-check probes, and simplifying the setup and maintenance of a multi-node cluster deployment because all nodes use the same configuration.

AppWorks Platform containerization allows you to build and run application-specific containers by adding your own application packages and configuration to the AppWorks Platform container image and running them.

Chapter 3

Container archive contents

The AppWorks Platform containerization tarball, `appworks-platform-23.4-containerization.tar.gz`, can be downloaded from OpenText My Support.

You can extract the contents and configure them to suit your requirements.

The following table lists the artifacts in the containerization archive, the type of the artifact, and their descriptions.



Note: The contents in these artifacts can be modified.

Artifact	Type	Description
<code>.env</code>	Environment file. See https://docs.docker.com/compose/compose-file/#env_file for more information.	Contains the environment settings to run the containers.
<code>env/secrets/DATABASE_ADMIN_PASSWORD</code>	Secrets file - See https://docs.docker.com/compose/compose-file/#secrets for more information.	Specifies the password of the database admin user. This is used when creating databases.
<code>env/secrets/DATABASE_PASSWORD</code>	Secrets file - See https://docs.docker.com/compose/compose-file/#secrets for more information.	Specifies the password of the database user for normal usage.
<code>env/secrets/KEYSTORE_PASSWORD</code>	Secrets file - See https://docs.docker.com/compose/compose-file/#secrets for more information.	Specifies the password of the AppWorks Platform keystore.
<code>env/secrets/LDAP_ADMIN_PASSWORD</code>	Secrets file - See https://docs.docker.com/compose/compose-file/#secrets for more information.	Specifies the password of the admin user to connect to the LDAP server.
<code>env/secrets/LDAP_CONFIG_PASSWORD</code>	https://docs.docker.com/compose/compose-file/#secrets	Specifies the password of the user to create a new database in the LDAP server.
<code>env/secrets/LICENSE_KEY</code>	Secrets file - See https://docs.docker.com/compose/compose-file/#secrets for more information.	Specifies the AppWorks Platform's license key. This is used when the value of <code>FETCH_LICENSE_KEY</code> property in the <code>.env</code> file is set to <code>false</code> .

Artifact	Type	Description
env/secrets/ LICENSE_SITE_PASSWORD	Secrets file - See https://docs.docker.com/compose/compose-file/#secrets for more information.	Specifies the AppWorks Platform license customer password.
env/secrets/ PLATFORM_PASSWORD	Secrets file - See https://docs.docker.com/compose/compose-file/#secrets for more information.	Specifies the password for the sysadmin user. Default is: Opentext2!

The following table lists the artifacts in the containerization archive, the type of the artifact, and their descriptions.



Note: The contents in these artifacts must not be modified.

Artifact	Type	Description
copy-to-image/appworks/ trustCertificates	Folder; contains certificates	Part of AppWorks Platform installation. When you add a certificate to this folder it will be imported in the security administration certificate store. This can be used if your application packages are signed with a certificate.
copy-to-image/appworks/ packages	Folder; contains AppWorks Platform application packages	Part of AppWorks Platform installation. When you add your own application packages to this folder, they are deployed inside the container. This way you can create a custom container image for your application.
docker-compose.yml	Docker compose file. See https://docs.docker.com/compose/compose-file/ for more information.	Specifies the AppWorks Platform services with the exposed port and other configuration.
docker-compose- openldap.yml	Docker compose file. See https://docs.docker.com/compose/compose-file/ for more information.	Specifies the LDAP services and its configuration.
docker-compose- postgres.yml	Docker compose file. See https://docs.docker.com/compose/compose-file/ for more information.	Specifies the PostgreSQL database services and its configuration. It is based on the public postgres:12.3-alpine container.

Artifact	Type	Description
docker-compose-mssql.yml	Docker compose file. See https://docs.docker.com/compose/compose-file/ for more information.	Specifies the MS SQL Server database services and other configuration. It is based on the public <code>mcr.microsoft.com/mssql/server:2019-GA-ubuntu-16.04</code> container.
docker-compose-oracle.yml	Docker compose file. See https://docs.docker.com/compose/compose-file/ for more information.	Specifies the Oracle database services and other configuration. It is based on the public <code>store/oracle/database-enterprise:12.2.0.1-slim</code> container.
docker-compose-jmx-monitoring.yml	Docker compose file. See https://docs.docker.com/compose/compose-file/ for more information.	Specifies the configuration to enable JMX monitoring on TomEE through <code>CATALINA_OPTS</code> .
docker-compose-remote-debugging.yml	Docker compose file. See https://docs.docker.com/compose/compose-file/ for more information.	Specifies the configuration to enable remote debugging on TomEE through <code>JPDA_OPTS</code> .
Dockerfile	Dockerfile. See https://docs.docker.com/engine/reference/builder/ for more information.	Contains the AppWorks Dockerfile for building the Application images.
helm-charts	Script	Contains Helm-charts files required to run the containers in Kubernetes.
License.txt	Text file	The OpenText End User License Agreement.

3.1 Application-specific artifacts

If you are deploying an application-specific image, before building it, you must have the following.

- Application-specific application packages (.cap)
- Application-specific configuration files. For information on the configuration files, their structure, and the content format, see: [“Understanding the configuration files” on page 23](#) and [“Externalized configuration” on page 24](#).

Chapter 4

Deployment requirements

To deploy AppWorks Platform, you need to have certain items before starting the deployment process. Ensure that the required software is installed or the relevant access details are obtained and ready for use.

While this guide provides only specific instructions to use the third-party applications as required for deploying AppWorks Platform, OpenText recommends you to refer the application's official documentation for in-depth information.

4.1 Software requirements

AppWorks Platform containerization requires the following minimal software versions:

Product	Minimal version
Docker	20.10.23
Docker-compose	1.25
Kubernetes	1.13
Helm	3.1.2

4.2 Hardware requirements

Ensure that you meet the following resource requirements:

- While the minimum memory requirements for running the AppWorks Platform, LDAP, and database containers is 6 GB, the standard AppWorks Platform container requires at least 4GB memory.
- When adding applications to the container, depending upon the load expected, you can increase the memory to 16 GB or more.
- As the OpenLDAP and database containers also require memory, when planning your application's resource requirements, ensure that Docker has enough resources available including the CPUs, swap memory, and disk space.

4.3 Docker

If you are deploying AppWorks Platform on the Docker environment, do the following:

- Install and configure Docker. You can install Docker on both Windows and Linux operating systems.

Windows	– Microsoft provides Docker for Windows to run containers. It uses Microsoft Hyper-V to run the Linux-based containers. Ensure to use Docker for Windows release 2.2 or later versions. – Install Docker and Docker Compose. See https://docs.docker.com/install/ for the installation instructions. – Ensure that Docker can access the local drive on the host where the AppWorks Platform containerization build files are located. You can perform this through the Docker App > Resources > File Sharing section.
Linux	You can install Docker on various Linux flavors. Based on the Linux flavor you use, see the corresponding flavor's Docker installation documentation for the instructions.

- Login to the OpenText registry: `docker login registry.opentext.com` using your OpenText Connect credentials.

4.4 Kubernetes

If you are deploying AppWorks Platform on the Kubernetes environment, do the following:

- Install and configure Docker.
- Install Kubernetes.
- Install Helm.
- Install `kubectl`.
- Set up LDAP using the OpenLDAP container or the OpenText CARS installer. When LDAP is deployed using Helm, the HA-enabled LDAP is automatically deployed.
- Set up a database service or server for AppWorks Platform deployment.



Note: If you are deploying AppWorks Platform on any of the cloud platforms such as Microsoft Azure, Google Cloud Platform (GCP), or Amazon Web Services (AWS), apart from the prerequisites listed above for Kubernetes, you must ensure that other requirements are met. See the corresponding topics listed below for a list of prerequisites.

- [Prerequisites for deploying AppWorks Platform on Azure](#)
- [Prerequisites for deploying AppWorks Platform on Google Cloud Platform](#)
- [Prerequisites for deploying AppWorks Platform on Amazon Web Services](#)

Chapter 5

Understanding the configuration files

When deploying the AppWorks Platform and OpenLDAP images, you have to update various files to configure the settings required for your application. Depending upon the deployment option you select, the configuration files vary. For example, if you are using Docker, you may need to update the `.env`, `secrets`, and `docker-compose` files.

This topic aims at explaining various files involved in the process and also preparing the files necessary for deployment.

5.1 Configuration files

When building the container image, the AppWorks Platform tools extract the configuration metadata from the application packages. This information is added to the `settings-all.yml` and `settings-mandatory.yml` files. These files act as a template or reference for the administrator. These files also have the specific configuration needed for the entity building blocks that are part of the application packages. These files are not shipped with the container build files because they can change based on the building blocks used in the entities.

- `settings-all.html` - contains all the possible settings for AppWorks Platform and the application packages deployed, in HTML format. You can open this file in a browser and it contains the default values for a lot of settings. AppWorks Platform uses smart defaults where possible. Therefore, as an administrator, you do not need to specify these, which results in lesser configuration.
- `settings-all.yml` - contains all possible settings for AppWorks Platform and application packages deployed in it, in YAML format.
- `settings-mandatory.yml` - contains only the settings that do not have a default for AppWorks Platform and application packages deployed in it, in YAML format. Therefore, to use these settings, you must specify values.

Obtaining the `solution.yml`, `settings-all.yml`, and `settings-mandatory.yml` files from the container image

These files are created in the `appworks` container. Therefore, you must create the Application image first. Docker does not allow to copy files from an image. Files can be copied from a running container only. Therefore, an administrator must build an image first and run it once to get the `settings-all.yml` and `settings-mandatory.yml` files for this specific solution image.

Run the following Docker commands to obtain the files:

- `docker run --entrypoint cat appworks/platform:23.4 /opt/appworks/cluster/shared/config/solution.yml > temp_solution.yml`

- `docker run --entrypoint cat appworks/platform:23.4 /opt/appworks/config/template/settings-all.html > temp_settings-all.html`
- `docker run --entrypoint cat appworks/platform:23.4 /opt/appworks/config/template/settings-all.yml > temp_settings-all.yml`
- `docker run --entrypoint cat appworks/platform:23.4 /opt/appworks/config/template/settings-mandatory.yml > temp_settings-mandatory.yml`

5.2 Externalized configuration

AppWorks Platform supports externalized configuration through MicroProfile Configuration(MPC). The configuration can be specified in any MicroProfile compatible configuration source. The two common configuration sources are environment variables and YAML files. All the settings in AppWorks Platform are available through external configuration.



Note: Externalized configuration is supported only through MPC and not from the AppWorks Platform user interface.

Mandatory service containers

AppWorks Platform has a set of mandatory service containers that are required to run the platform. These services are monitor, LDAP, XMLStore, DataSourceManagement, Platform, License, CAP, Security, Authentication, XForms, UIFoundation, Task, Tag, Preference, Events, Development, BPM, PIM, and Notification.

solution.yml file

A best practice approach for adding the required configuration settings to AppWorks Platform is by placing this configuration in a YAML file, for example, `solution.yml`. AppWorks Platform, by default, searches for a `solution.yml` in the `/opt/appworks/cluster/shared/config/`. You can use multiple configuration files and must provide the YAML configuration file names as values for the `PLATFORM_CONFIGURATION_YAMLFILES` configuration variable in the `.env` file. `PLATFORM_CONFIGURATION_YAMLFILES` accepts a comma separated list of YAML file names.

For example:

```
PLATFORM_CONFIGURATION_YAMLFILES=solution.yml, workspace.yml, otds.yml
```

Structure and usage of a YAML configuration file

A YAML configuration file should include an ordinal. This value must be specified in the property called `config_ordinal`. A YAML configuration file with a higher value takes priority over a file with a lower ordinal. When you have two configuration files that define the same property, the property value from the YAML configuration file with the highest ordinal value is considered.

If there is no `solution.yml` available in the container, a default `solution.yml` file is created in the `/opt/appworks/cluster/shared/config/` folder with the following content:

Default solution.yml sample

```
config_ordinal: 210
platform:
  services:
    instances: Audit, Rule, Scheduler
```

AppWorks Platform configuration can be provided in several ways. You can provide it through environment variables or YAML files. The sample YAML above can also be written as `PLATFORM_SERVICES_INSTANCES=Audit, Rule, Scheduler`. The YAML notation is just another common way to express the same configuration.



Note: In YAML files, indentation is used to separate the property levels. YAML files are very sensitive to spaces and tabs. The same level of properties must have the same indentation. For example, if you use two spaces for a specific level, you must use two spaces for all the properties at the same level.

As YAML files are sensitive to spaces and tabs, any difference in the number of spaces or tabs results in an incorrect configuration. While the sample code snippets provided in the documentation tried to preserve the spaces, there may be some instances where the indentation might appear lost due to lengthy values being wrapped to the next line.

In the sample YAML file, three services are specified to be started as service container. As there is no other configuration specified for them, they use the smart defaults as defined in the platform.

For example, in the `settings-all.yml` file, to use the Audit service, you can define it as follows:

```
platform:
  auditing:
    dataSource: system
    <auditComponent>: 'false'
```

The Audit service, auditing, uses a `DataSource` `system` by default. It can also have multiple auditing artifacts for which audit can be enabled. You must replace the placeholders enclosed within `<>`, for example `<auditComponent>`, with the actual name. For description of these properties, see [“Audit” on page 218](#).

The following code snippet displays the Rule service definition in the `settings-all.yml` file:

```
platform:
  ...
  rule:
    configurations:
      <configuration>:
        customMonitorListener: ''
        dataSource: system
        maxDispatcherQueueSize: '50000'
        monitorRequestAndResponse: 'false'
```

```
ruleAudit: 'false'
threadPoolSize: '5'
```

As explained earlier, the `<configuration>` property must be replaced with a name, for instance, `MyRuleConfig`. There can be multiple rule configurations that are grouped under `configurations`, each with its own name. The `settings-all.yml` file contains the available properties that can be used per configuration. The values of some properties are preset with default values, for example, 5000 for `maxDispatcherQueueSize`, `false` for `monitorRequestAndResponse`, and so on.

The `settings-all.yml` file has similar configuration for all available services.

The following is a sample code snippet for the Authenticators service:

```
platform:
  ...
  authenticators:
    defaultLogin: <no default value specified>
    defaultSigner: <no default value specified>
    <oauthAuthenticator>:
      kind: OAuth
      clientId: <no default value specified>
      publicLoginUrl: ${platform.otds.url}/otdsws/login
    <otdsAuthenticator>:
      kind: OTDS
      publicLoginUrl: ${platform.otds.url}/otdsws/login
    <platformAuthenticator>:
      kind: Platform
      loginPath: /wcp/sso/login.htm
```

- The `defaultLogin` and `defaultSigner` properties contain the name of a defined authenticator under them. There are three types of authenticators, OAuth, OTDS, and Platform, each with their own kind and properties. Therefore, to define an OTDS authenticator with the name, `myotds`, add the property as follows:

```
authenticators:
  myotds:
```

- Add the kind and properties with no default values:

```
authenticators:
  myotds:
    kind: OTDS
```

- As the OTDS authenticator uses an OTDS resource, the `settings-all.yml` file contains the following:

```
otds:
  resourceId: <no default value specified>
  url: <no default value specified>
```

- Enter the relevant details, for example:

```
otds:
  resourceId: 5b4c2203-4bf1-4b57-8b55-e65b5b7fbd28
  url: https://otds-server.opentext.net
```

- As it is possible to add multiple authenticators to the configuration, you can add another authenticator:

```
myoauth:
  kind: OAuth
```

```
clientId: jopl
publicLoginUrl: https://otds-server.opentext.net/otdsws/login
```

- The `publicLoginUrl` has a default value in the `settings-all.yml` file. If required, you can change it.

Usage of instances

The following sections explain the configuration of some instances with examples.

```
services:
  instances: Audit, Rule, Scheduler
```

As shown above, the `instances` property is required in some services as the platform otherwise cannot determine which of the next properties in the configuration to use.

Similarly, the email connector also has an `instances` property:

```
<Email>:
  kind: Email
  emailBoxes:
    instances: ''
    <emailBoxName>:
      automaticRestart: 'true'
      profile: <no default value specified>
      restartInProgress: 'false'
      triggersDefinitionXmlStoreFilePath: default_triggers
  storage:
    compressData: 'false'
    dataSource: system
    maxEmailsPerPollingCycle: '15'
    maxExecutionThreads: '5'
    maxNumberOfRows: '100'
    pollerInterval: PT10S
```

You can define multiple emailboxes, each with its own name, but must be listed in the `instances` property again.

The generic services property also follows this pattern for the methods it can handle:

```
<service>:
  kind: ${name}
  classpath: ''
  packages:
    instances: ''
    <package>:
      methodSets: ''
  requestTimeout: PT30S
  validation: 'false'
```

This generic service section is a parent class of the individual services. All services handle methodsets, can have request and response validation, and so on.

Your configuration however most likely will not contain methodsets. This is due to the auto attach feature of the platform. The platform links services and service containers. For example, a Business Process Management (BPM) web service is always handled by a BPM service, an HTTP connector service is handled by the HTTPConnector service, and UDDI services are handled by the UDDIConnector.

The `kind` property in the service determines the type of application connector. This can either be the full class name of the application connector, for example, `com.cordys.bpm.service.BPMApplicationConnector` or the simplified name as BPM.

The following table lists the name of the application connector and its corresponding Application connector class:

Name	Application connector class
Mandatory services	
Authentication	<code>com.eibus.sso.applicationconnector.SSOConnector</code>
CAP	<code>com.cordys.cap.applicationconnector.CAPConnector</code>
DataSourceManagement	<code>com.cordys.dsoconfig.DSOConfigServiceConnector</code>
Events	<code>com.eibus.applicationconnector.event.EventService</code>
LDAP	<code>com.eibus.applicationconnector.ldap.LDAPService</code>
License	<code>com.cordys.applicationconnector.license.LicenseManager</code>
monitor	<code>com.eibus.applicationconnector.monitor.Monitor</code>
UIFoundation	<code>com.cordys.ajax.internal.AVConnector</code>
Platform	<code>com.cordys.applicationserverconnector.Connector</code>
Preference	<code>com.cordys.preference.applicationconnector.PreferenceConnector</code>
Security	<code>com.eibus.secadmin.applicationconnector.SecurityAdminConnector</code>
Tag	<code>com.cordys.tagserver.connector.TagServerConnector</code>
Task	<code>com.cordys.applicationconnector.task.TaskConnector</code>
XForms	<code>com.eibus.applicationconnector.xforms.XFormConnector</code>
XMLStore	<code>com.eibus.applicationconnector.xmlstore.XMLStore</code>
BPM	<code>com.cordys.bpm.service.BPMApplicationConnector</code>
Notification	<code>com.cordys.cpc.notification.net.NotificationMapper</code>

Name	Application connector class
PIM	com.cordys.bpm.pim.service.PIMQueryConnector
Development	com.cordys.cws.internal.soap.CWSSOAPConnector
Optional services	
Audit	com.cordys.audit.connector.AuditConnector
Email	com.eibus.applicationconnector.email.MailConnector
HTTPConnector	com.opentext.applicationconnector.httpconnector.HttpConnector
UDDIConnector	com.eibus.applicationconnector.uddi.UDDIConnector
BPMArchive	com.cordys.bpm.admin.archive.service.ArchiveApplicationConnector
DocumentStore	com.cordys.documentstore.applicationconnector.DocumentStoreConnector
FTP	com.eibus.applicationconnector.ftp.FTPHandler
Magellan	com.opentext.ihub.IHubConnector
Rule	com.cordys.bre.ws.RuleService
Scheduler	com.cordys.cpc.coboc.scheduler.soap.ScheduleMapper
UserManagement	com.cordys.usermanagement.applicationconnector.UserManagementConnector
WsAppServer	com.cordys.cpc.bsf.connector.BsfConnector

The `settings-all.yml` file by default contains the configuration properties of the platform services, but when deploying an entity-based solution, the entities and used building blocks are mentioned in the `settings-all.yml` file. This helps you in defining the correct `solution.yml` file.



Note: Ensure that you copy the relevant configuration settings files generated from the container image to see the application configuration settings you need to specify.

Organization-specific configuration

The YAML configuration also can contain organization-specific configuration. For example, to define an FTP service in the acme organization only, you can use the following configuration:

```
platform:
  organizations:
    acme:
```

```
FTP:
connections:
  teleFTP:
    kind: FTP
    server: tele.ftp.server.acme.com
    userName: jopl
    password: aGVsbG8gZHVkZSA6LSk
    basePath: /home/jopl
```

The configuration settings are kept to a minimum because you must specify the non-default values only. In this sample, the password is entered directly in the configuration file as plain text. Alternatively, you can use a secrets file to keep it more secured as shown in the following sample code snippet:

- Create a YAML file, for example `docker-secrets.yml`, and add the following code.

```
version: "3.7"

services:
  appworks:
    secrets:
      - FTP_PASSWORD
secrets:
  FTP_PASSWORD:
    file: env/secrets/FTP_PASSWORD
```

- Create a `env/secrets/FTP_PASSWORD` file and type the plain text password in this file.
- Modify the FTP configuration by replacing the plain text password with a reference to the secrets file as shown in the following sample.

```
...
FTP:
connections:
  teleFTP:
    kind: FTP
    server: tele.ftp.server.acme.com
    userName: jopl
    password: ${ftp.password}
    basePath: /home/jopl
```

- Include the `docker-secrets.yml` file with the `docker-compose -f <files>` command.



Note: After providing the YAML configuration, an organization is not created automatically. You must create an organization from the Organization manager.

Organization vs shared configuration

You also can define a specific property at the shared and organization level. The most specific configuration takes precedence.

Using non-containerized LDAP and database

The default setup of the `docker-compose` files includes running OpenLDAP and PostgreSQL in a container. It is not mandatory to use this configuration. You can

also use a classic deployed CARS server and database in combination with AppWorks Platform running in a container. You must specify the LDAP and database endpoints, credentials, schema name, and so on in the configuration. The AppWorks Platform container uses this configuration to connect to CARS and the database because it connects when using a container deployment for these components.

When using the `docker-compose` command to build and start the container, you must not specify the `docker-compose` file for OpenLDAP and the database.

5.3 Configuration files for Docker

Before starting the deployment on Docker, you must configure a set of YAML files that are required by the AppWorks container to run on Docker.



Note: While it is necessary that you update the parameters of the files listed in the tables below based on your system configuration and application requirements, you can also retain the default values.

The following table lists the files are:

File name	Description
<code>.env</code>	All environment variables defined and can be used in other docker compose files.
<code>docker-compose.yml</code>	All configuration settings related to AppWorks Platform.
<code>docker-compose-postgres.yml</code>	Configuration settings required to run PostgreSQL database in a container.
<code>docker-compose-oracle.yml</code>	Configuration settings required to run Oracle database in a container.
<code>docker-compose-mssql.yml</code>	Configuration settings required to run Microsoft SQL server database in a container.
<code>docker-compose-openldap.yml</code>	Docker compose file to run OpenLDAP in a container. Note: If you are using a HA setup for OpenLDAP, you must use the <code>docker-compose-openldap-ha.yml</code> file.
<code>docker-compose-jmx-monitoring.yml</code>	Configuration settings required to enable JMX monitoring in AppWorks container.
<code>docker-compose-remote-debugging.yml</code>	Configuration settings required to enable remote debugging of AppWorks container.



Note: The `.env` and `docker-compose.yml` files are necessary for the AppWorks container deployment. The other docker files are optional and need to be updated based on your requirements only.

5.3.1 .env

The `.env` file is used to define all the environment variables required by the appworks or other images. These environment variables can be used in other YAML files. It provides information about the license, database, and ldap settings.

5.3.2 docker-compose.yml

The `docker-compose.yml` file contains the configuration settings to run the AppWorks Platform service. Most of the settings contain default values, which can be retained or updated according to your requirements.

The following table lists the parameters that are available in the `docker-compose.yml` file and their descriptions.

Parameter	Description
services/appworks	Configuration details of AppWorks Platform including secrets, image details, environment, and volumes.
volumes	List of volumes specified in AppWorks Platform.
networks	Name of the AppWorks Platform network, which is used by all the containers. The default value is <code>appworks_net</code> .

5.4 Configuration files for Helm charts

The `values.yaml` file is used to provide various configuration settings for configuring Docker images deployed in Kubernetes. These configuration values include database details, LDAP details, license details, memory and other resource requirements, and volumes.

When using the `values.yaml` file, the following properties are mandatory:

- repository
- tag
- customerName, siteName, licenseKey or customerName, siteName, sitePassword
- Namespace - by replacing `$NAMESPACE`
- storageClass
- appworks.password, keystorePassword, jmxKeystorePassword, adminPassword, configPassword, database.password, and database.adminPassword.



Note: The default values provided for the memory settings and other resource settings in the `values.yaml` file are based on minimum recommendations.

Administrators can change these values according to the application's requirements.

The following table lists the categories in the `values.yaml` file, the properties, the default values they accept, their descriptions, and sample snippets.

Category	Property and details	Sample
global	The global setting section is used to avoid redundant entries. ImageRegistry - Name of the image registry. ImageTag - Tag name of the images. ImagePullPolicy - When to pull the images. Possible values: - Always - IfNotPresent ImagePullSecret - Secret to pull the image from the registry. UseExternalSecrets - Creates an external secret when set to true. Default value is false. SecretStoreRefName - Name of the secret store. SecretStoreRefKind - Kind of the secret store. Possible values: - SecretStore - ClusterSecretStore SecretStoreRefDataFromKey - path of the secret. StorageClassName - Persistent volume storage class for AppWorks Platform and LDAP.	<pre> global: imageRegistry: &image_registry registry.opentext.com imageTag: &image_tag <tag> imagePullPolicy: &image_pull_policy Always imagePullSecret: &image_pull_secret useExternalSecrets: &use_external_secrets false secretStoreRefName: &secret_store_ref_name <secretstorename> secretStoreRefKind: &secret_store_ref_kind SecretStore secretStoreRefDataFromKey: &secret_store_ref_data_from_ key <secret-manager-name> storageClassName: &storage_class_name <storageClass> enableLivenessProbe: &enable_liveness_probe true </pre>

Category	Property and details	Sample
replica	replicaCount - Type the number of pods required for AppWorks Platform. This determines the number of nodes in the AppWorks Platform cluster. Default value is 2.	replicaCount: 2
image	registry - Name of the Docker registry where the AppWorks Platform image is available. repository - Name of the Docker registry where the AppWorks Platform image is available. tag - Tag name for the AppWorks Platform Docker image. pullPolicy - When to pull new images. Possible values: • Always • IfNotPresent pullSecret - Secret to pull images from the registry.	<pre>image: registry: registry.opentext.com repository: otawp tag: 22.2 pullPolicy: IfNotPresent</pre>
license	fetchLicenseKey - Fetch the license key from the license server. • If the value is set to false, provide the following: • customerName • siteName • licenseKey • If the value is set to true, provide the following values: • customerName • siteName • sitePassword	<pre>license: fetchLicenseKey: "false" customerName: <valid customer name> siteName: <site name> licenseKey: <valid license key> license: fetchLicenseKey: "true" customerName: <valid customer name> siteName: <site name> sitePassword: <password></pre>

Category	Property and details	Sample
appworks	password – Password of the AppWorks Platform sysadmin user. keystorePassword – Certificate keystore password serviceAnnotations - List of custom annotations that must be assigned to the appworks service. By default, serviceAnnotations is empty. However, you can add the relevant annotations to the tag as required. For example, if you want to set up ingress in GCP, you can add the annotations under serviceAnnotations as follows: <pre>serviceAnnotations: servicecloud.google.com/ neg: '{"ingress": true}'</pre> useTLSForLDAP - Flag to determine whether the AppWorks Platform connection to OpenLDAP needs to be secure or not. true - AppWorks Platform connects with OpenLDAP in secure mode. false - AppWorks Platform uses non-secure connection with OpenLDAP only if allowNonTlsConnections of ldap is set to true. Default is true. originalNodesList - List of nodes that must be migrated from the classic mode to the containerized mode.  Note: Applicable only when migrating AppWorks Platform from the classic mode	<pre>appworks: password: "mypassword" keystorePassword: "mykeystore" originalNodesList: "" serviceAnnotations: [] ldap: host: ldaphost.myorg.net port: 6366 instanceName: defaultinst domain: myorg.net user: cn=Directory Manager,o= myorg.net adminPassword: "adminPwd" configPassword: "configPwd" mdbDirectory: /opt/ cars/var/mdb enabled: true image: registry: registry.opentext.com repository: otawp- openldap tag: 22.2 persistence: storageClass: trident- nfs database: vendor: PostgreSql host: mydb.myorg.net port: "5432" createDataBase: "true" adminUserName: "admin" adminPassword: "admin" user: "dbuser" password: "password"</pre>

Category	Property and details	Sample
	to the containerized mode.	

Category	Property and details	Sample
ldap	host - LDAP host name port - LDAP port number instanceName - Name of the instance user - LDAP user name adminPassword - LDAP administrator password configPassword - LDAP config user password mdbDirectory - Directory where MDB is located allowNonTlsConnections - Flag to determine whether OpenLDAP container needs to run in secure(TLS) mode or both secure and non-secure mode. true - the OpenLDAP container accepts both secure and non-secure connection. false - OpenLDAP accepts only secure connections. Default value is false. useTlsForSyncReplication - Flag to determine whether OpenLDAP container needs to use secure or non-secure connection for replication. true - sync replication is done using a secure connection. false - sync replication is done using a secure connection only if allowNonTlsConnections of ldap is set to true. Default value is true. enabled - Flag to determine whether OpenLDAP must be deployed as a pod or not. If	

Category	Property and details	Sample
	the value is true, the OpenLDAP image will be deployed based on image details.	
ldap/image	repository – name of the OpenLDAP repository . tag- Tag name of the OpenLDAP image. registry - name of the docker registry where OpenLDAP image is available.	
ldap/persistence	storageClass - Persistence volume storage class for the OpenLDAP image.	
database	vendor – Name of the database vendor. Possible values: • PostgreSQL • Oracle • MsSqlServer • AzureSqlDb host – Database server name port – Database port databaseName – Database name tableSpaceName – Table space name. Applicable to Oracle database only. createDataBase – Whether the database must be created. adminUserName - Database administrator user name adminPassword – Database administrator user password user – Database user name password - Database user password	

Category	Property and details	Sample
container	All the properties related to resources for the container, such as memory and CPU.	<pre>container: resources: requests: memory: "4Gi"</pre>
readinessProbe	All the properties for checking the readiness of a container.	<pre>readinessProbe: initialDelaySeconds: 180 periodSeconds: 10 timeoutSeconds: 5</pre>
persistence	All properties related to volumes. The administrator must provide the storage class and can also change the size of the volume, if needed. See Volume in Docker on page 11 .	<pre>persistence: enable:true storageClass: trident- nfs</pre>
ingress	Ingress exposes HTTP and HTTPS routes from outside the cluster to services within the cluster. Traffic routing is controlled by rules defined on the Ingress resource.	<pre>ingress: enabled: true annotations: kubernetes.io/ ingress.class: nginx nginx.ingress.kubernetes.io/ ssl-passthrough: "false" kubernetes.io/ingress.allow- http: "false" host: <Public Host name></pre>

Category	Property and details	Sample
extraEnv	All environment variables and settings that must be set. For example: • TomEE memory settings • JVM settings • AppWorks Platform settings • PLATFORM_DATABASE_CONNECTIONS_SYSTEM_USESSL - when set to true, AppWorks Platform uses SSL communication with the Database server during deployment. Default: false • TRUST_DB_CERTS - when set to false, the SSL certificate of the database must be added to copy-to-image \appworks \trustCertificates folder while building the AppWorks Platform image. This property is applicable only when PostgreSQL is used as the database and above property is set to true. Default: true • SHOW_ADMIN_UI - Enables or disables administrator interfaces when all the services are configured externally using MicroProfile configuration. When set to false, several administration tasks are hidden in the AppWorks Platform Explorer. See “Displaying hidden tasks in the containerized mode” on page 121.	<pre>extraEnv: - name: SHOW_ADMIN_UI value: "true" - name: CATALINA_OPTS value: - XX:MaxRAMPercentage=75.0 extraEnv: - name: SHOW_ADMIN_UI value: "true" - name: CATALINA_OPTS value: - XX:MaxRAMPercentage=75.0 - name: PLATFORM_DATABASE_CONNECTION S_SYSTEM_USESSL value: "false" - name: TRUST_DB_CERTS value: "true"</pre>

5.4.1 azure.yaml

The `azure.yaml` file provides the azure specific configuration settings for configuring the Docker images deployed in Kubernetes. These configuration values include storageclass, ingress, and external secrets.

The following table lists the categories in the `azure.yaml` file, the properties, the default values they accept, their description, and sample snippets.

Category	Property and details	Sample
externalSecretSpec/ secretStoreRef	Configuration of the SecretStore or the ClusterSecretStore. name - Name of the SecretStore or ClusterSecretStore. kind - Default value is SecretStore. Possible values: - SecretStore - ClusterSecretStore	<pre>externalSecretSpec: secretStoreRef: name: &secret_store_name <secretstore-name> kind: &secret_store_kind SecretStore</pre>
externalSecretSpec/data	Configuration to fetch keys from the provider. secretKey - Name of the Secret. remoteRef - Configuration of the Azure key. remoteRef /key - Name of the key in Azure.	<pre>externalSecretSpec: data: - secretKey: DATABASE_ADMIN_ PASSWORD remoteRef: key: DATABASE-ADMIN-PASSWORD</pre>
secretStoreSpec/provider	Configuration of the provider.	

Category	Property and details	Sample
azurekv	Configuration of azure key vault. tenantID - Azure tenant ID. For more information, refer https://learn.microsoft.com/en-us/azure/active-directory/fundamentals/active-directory-how-to-find-tenant. vaultURL - URL of your vault instance. For more information, refer https://learn.microsoft.com/en-us/azure/keyvault/general/about-keys-secrets-certificates. authSecretRef - Points to the secret that contains the azure service principal credentials.	<pre>secretStoreSpec: provider: azurekv: # azure tenant ID, see: https://learn.microsoft.com/en-us/azure/active-directory/fundamentals/active-directory-how-to-find-tenant tenantID: "" # URL of your vault instance, see: https://learn.microsoft.com/en-us/azure/key-vault/general/aboutkeys-secrets-certificates vaultUrl: "" authSecretRef: # points to the secret that contains the azure service principal credentials clientId: name: <secret-name> #name of the secret created with clientId and clientSecret. key: ClientID clientSecret: name: <secret-name> #name of the secret created with clientId and clientSecret. key: clientSecret</pre>
authSecretRef	clientId/name - name of the Client ID. clientId/kind - Enter ClientID. clientSecret/name - name of the Client Secret. clientSecret/kind - Enter the ClientSecret.	
ldap/ externalSecretSpec / secretStoreRef	Configuration of the SecretStore or the ClusterSecretStore name – name of the SecretStore or ClusterSecretStore kind – Default value is SecretStore Possible values: • SecretStore • ClusterSecretStore	<pre>ldap: externalSecretSpec: secretStoreRef: name: *secret_store_name kind: *secret_store_kind data: remoteRef: key: LDAP-ADMIN-PASSWORD secretKey: LDAP_CONFIG_PASSWORD remoteRef: key: LDAP-CONFIG-PASSWORD</pre>

Category	Property and details	Sample
ldap/externalSecretSpec/data	Configuration to fetch keys from the provider. secretKey – Name of the Secret. remoteRef/key – Name of the key in Azure.	
ldap/persistence/storageClass	Persistent volume storage class for LDAP. Default value is azurefile.	ldap: persistence: storageClass: &storage_classazurefile
ingress	annotations - Annotations of the ingress. Default annotation is nginx.	ingress: annotations: - kubernetes.io/ ingress.class: nginx - nginx.ingress.kubernetes.io/ sslpassthrough: "false" - kubernetes.io/ ingress.allowhttp: "false" - nginx.ingress.kubernetes.io/ proxy-read-timeout: "360" - nginx.ingress.kubernetes.io/ proxy-body-size: 10m
persistence	storageClass - Persistent volume storage class for AppWorks Platform. Default value is azurefile.	persistence: storageClass: *storage_class

5.4.2 gcp.yaml

The `gcp.yaml` file provides the GCP specific configuration settings for configuring the Docker images deployed in Kubernetes. These configuration values include storageclass, ingress, and external secrets.

The following table lists the categories in the `gcp.yaml` file, the properties, the default values they accept, their description, and sample snippets.

Category	Property and details	Sample
externalSecretSpec / secretStoreRef	Configuration of the SecretStore or the ClusterSecretStore. name – name of the SecretStore or the ClusterSecretStore. kind - Default value is SecretStore. Possible values: • SecretStore • ClusterSecretStore	<pre>externalSecretSpec: secretStoreRef: name: &secret_store_name <secret-store-name> kind: &secret_store_kindSecretStore</pre>
externalSecretSpec /data	Configuration to fetch keys from the provider. secretKey - Name of the Secret. remoteRef - Configuration of the gcp key. remoteRef /key - Name of the key in gcp.	<pre>externalSecretSpec: secretStoreRef: data: - secretKey: DATABASE_ADMIN_ PASSWORD remoteRef: key: DATABASE-ADMIN-PASSWORD ...</pre>
secretStoreSpec/ provider	Configuration of the provider.	<pre>secretStoreSpec: provider: gcpsm: auth: secretRef: secretAccessKeySecretRef: name: <secret-name> # secret name containing SA key key: <key-name> # key name containing SA key projectID: <project-id></pre>
gcpsm	Configuration of the GCP Secret Manager. auth/secretRef– Configuration of the secret. projectID – GCP project ID.	
gcpsm/auth/secret Ref/ secretAccessKeySecretRef	name - Name of the secret containing SA key. key – name of the key containing SA key.	

Category	Property and details	Sample
ldap/ externalSecretSpec / secretStoreRef	Configuration of the SecretStore or the ClusterSecretStore. name – Name of the SecretStore or ClusterSecretStore. kind – Default value is SecretStore. Possible values: - SecretStore - ClusterSecretStore	<pre> ldap: externalSecretSpec: secretStoreRef: name: *secret_store_name kind: *secret_store_kind data: - secretKey: LDAP_ADMIN_PASSWORD remoteRef: key: LDAP_ADMIN_PASSWORD - secretKey: LDAP_CONFIG_PASSWORD remoteRef: key: LDAP_CONFIG_PASSWORD </pre>
ldap/externalSecret Spec/data	Configuration to fetch the keys from the provider. secretKey – Name of the Secret. remoteRef /key – Name of the key in Azure.	
ldap/persistence/storageClass	Persistent volume storage class for LDAP.	<pre> ldap: persistence: storageClass: &storage_class <storageClass> </pre>
ingress	annotations - Annotations of the ingress. Default value is GCP Ingress Controller.	<pre> ingress: annotations: - kubernetes.io/ ingress.class: gce - kubernetes.io/ ingress.allowhttp: "false" - gcp.ingress.kubernetes.io/ target - type: ip </pre>
persistence	storageClass - Persistent volume storage class for AppWorks Platform.	<pre> persistence: storageClass: *storage_class </pre>
dnsPolicy	Configuration to change DNS Policy of AppWorks Statefulset. Default value is "None".	<pre> dnsPolicy: &dns_policy "None" </pre>

dnsConfig/nameservers	List of IP addresses that are used as DNS servers for the Pod	dnsConfig: nameservers: - 10.120.0.10 #<IP address of the Kube DNS service>. You can obtain it using the following command: <code>kubectl get svc -n kube-system</code> searches: - appworks.default.svc.cluster.local
dnsConfig/ searches	List of DNS search domains for hostname lookup in the Pod	- default.svc.cluster.local - svc.cluster.local - cluster.local
dnsConfig/options	Optional list of objects, where each object may have a name property (required) and a value property (optional)	options: - name: ndots
ldap/dnsPolicy	Configuration to change DNS Policy of LDAP Statefulset. Default value is "None".	ldap: dnsPolicy: &dns_policy "None"
ldap/dnsConfig	Configuration to control DNS setting for ldap pods. Provide dnsConfig when dnsPolicy is set to "None".	
ldap/dnsConfig/nameservers	List of IP addresses that are used as DNS servers for the Pod	ldap: dnsConfig: nameservers: - 10.120.0.10 #<IP address of the Kube DNS service>. You can obtain it using the following command: <code>kubectl get svc -n kube-system</code> searches: - ldap.default.svc.cluster.local
ldap/dnsConfig/ searches	List of DNS search domains for hostname lookup in the Pod	- default.svc.cluster.local - svc.cluster.local - cluster.local
ldap/dnsConfig/options	Optional list of objects, where each object may have a name property (required) and a value property (optional)	options: - name: ndots

5.4.3 aws.yaml

The `aws.yaml` file provides the AWS specific configuration settings for configuring the Docker images deployed in Kubernetes. These configuration values include storageclass, ingress, and external secrets.

The following table lists the categories in the `aws.yaml` file, the properties, the default values they accept, their description and sample snippets.

Category	Property and details	Sample
externalSecretSpec / secretStoreRef	Configuration of the SecretStore or the ClusterSecretStore. name – name of the SecretStore or the ClusterSecretStore. kind - Default value is SecretStore. Possible values: • SecretStore • ClusterSecretStore	<pre>externalSecretSpec: secretStoreRef: name: &secret_store_name <secret-store-name> kind: &secret_store_kind SecretStore</pre>
externalSecretSpec /dataFrom	Configuration to fetch the keys from the provider. key - Name of the key in AWS.	<pre>externalSecretSpec: dataFrom: - extract: key: <secret-manager-name></pre>
secretStoreSpec/ provider	Configuration of the provider.	<pre>secretStoreSpec: provider: aws: service: SecretsManager region: <region> auth: jwt: serviceAccountRef: name: <serviceAccountName> namespace: <namespace></pre>
aws	Configuration of AWS Secret Manager. service - Enter SecretsManager. region – Region of the AWS Secret Manager auth/ jwt - Configuration of the secret.	
aws/auth/jwt/serviceAccountRef	name - name of the service account. namespace - name of the namespace.	

Category	Property and details	Sample
ldap/ externalSecretSpec / secretStoreRef	Configuration of the SecretStore or the ClusterSecretStore. name – name of the SecretStore or ClusterSecretStore. kind – Default value is SecretStore. Possible values: • SecretStore • ClusterSecretStore	<pre> ldap: externalSecretSpec: secretStoreRef: name: &secret_store_name <secret-store-name> kind: &secret_store_kind SecretStore dataFrom: - extract: key: <secret-manager- name> </pre>
ldap/externalSecretSpec /data	keys from the provider. secretKey – Name of the Secret. remoteRef /key – Name of the key in AWS.	
ldap/persistence/storageClass	Persistent volume storage class for LDAP.	<pre> ldap: persistence: storageClass: &storage_class <storageClass> </pre>
ingress	annotations - Annotations of the ingress. Default value is Amazon Load Balancer.	<pre> ingress: annotations: - kubernetes.io/ ingress.class: alb - alb.ingress.kubernetes.io/ scheme: internet-facing - alb.ingress.kubernetes.io/ targettype: ip </pre>
persistence	storageClass - Persistent volume storage class for Appworks.	<pre> persistence: storageClass: *storage_class </pre>

5.5 Application-specific configuration to the image

When adding application packages to the image, you might have to provide additional configuration to create new services, change service properties and so on. You can provide these configurations to a container through a `solutions.yml` file either by adding it to a running container or building it into a custom image. AppWorks Platform loads and acts on the changes in the `solution.yml` file.

In a production environment, it is not a usual practice to edit the files in the container. However, this might be an option in the development environment. You might add a new service or update some properties depending on your preferences, requirements, and style of working. AppWorks Platform uses MicroProfile

Configuration as the standard because this brings a very flexible and open method of configuring AppWorks Platform. The examples provided in this section are just a subset of what all is possible. This documentation does not aim to be complete in this respect but only acts as a guideline.

A couple of methods describing how to add a new version of a solution configuration to AppWorks Platform running in a container are provided below:

- [“Direct in-container editing of solution.yml” on page 49](#)
- [“Copying solution.yml into a container” on page 49](#)

5.5.1 Direct in-container editing of solution.yml

The AppWorks Platform image is based on a small Linux distribution that executes several Linux commands in it. It is possible to execute these commands directly in a container.

- Starting bash in a container

```
docker exec -it <appworks-container-name> bash
```

- Opening a file in vi in a container

You can use `vi` to edit a file directly in a container. However, to do this, you must build the image with the debug tools included. See [“Enabling debug tools in a container” on page 125](#).

```
docker exec -it <appworks-container-name> vi /opt/appworks/cluster/shared/config/solution.yml
```

The `solution.yml` file opens in a `vi` editor. After you close the editor, you are returned to the current prompt.

While the approach is very simple, you lose the configuration when you delete the container and its related volume.

5.5.2 Copying solution.yml into a container

Instead of editing the `solution.yml` file in the container, you also can edit it on the host, and then copy it into the container using the `docker cp` command.

```
docker cp solution.yml <appworks-container-name>:/opt/appworks/cluster/shared/config/solution.yml
```

With this approach, you can have a local copy of the configuration that can be stored in a repository.

5.5.3 Copying the solution.yml file in a container through the copy-to-image folder

The following examples demonstrate adding a `solution.yml` to a container image.

The container build files contain a `copy-to-image` folder with an `appworks` sub-folder. All the files and sub-folders in the `appworks` folder are copied into the container image's `appworks` folder. When you copy the `solution.yml` file to the `appworks/config` folder, the build script copies the `solution.yml` file into the container's image. However, this is copied to the `/opt/appworks/config` folder instead of the `/opt/appworks/cluster/shared/config/` folder. Therefore, if you need this file to be used by AppWorks Platform, set the following configuration property in the `.env` file:

```
PLATFORM_CONFIGURATION_YAMFILES=solution.yml
```

This fetches the configuration from the `solution.yml` file available in the `/opt/appworks/config/` folder and loads it. This approach enables the configuration to be a part of the container image only.

Using configuration variables in combination with a solution.yml file

The `solution.yml` file contains all the application-specific configuration. You can also use configuration variables in combination with the `solution.yml` file. By using this combination, you can have a fixed `solution.yml` in the Development, Test, Acceptance, and Production (DTAP) environments and also modify the configuration by changing the configuration variables. These variables can be stored in a `.env` file, a secrets file, or passed on to a container in any other way that the Docker environment supports.

The following code snippet is a sample in the docker-compose file that uses environment variables, an MPC form, and a secret.

```
version: "3.7"

services:
  appworks:
    secrets:
      - PLATFORM_SECURITY_KEYSTORE_STOREPASSWORD
    environment:
      - PLATFORM_GATEWAY_REQUESTVALIDATION=${jopl.wsdl.validation:-true}
      - PLATFORM_GATEWAY_RESPONSEVALIDATION=${jopl.wsdl.validation:-false}
    secrets:
      PLATFORM_SECURITY_KEYSTORE_STOREPASSWORD:
        file: env/secrets/PLATFORM_SECURITY_KEYSTORE_STOREPASSWORD
```

The sample shows how to use a secret, for example, the secret called `PLATFORM_SECURITY_KEYSTORE_STOREPASSWORD`, the value of which can be found in the `env/secrets/PLATFORM_SECURITY_KEYSTORE_STOREPASSWORD` file. The secret used for the `PLATFORM_SECURITY_KEYSTORE_STOREPASSWORD` configuration within the MicroProfile Configuration form is as follows:

```
platform:
  security:
```

```
keystore:
  storePassword
```

The example also defines PLATFORM_GATEWAY_REQUESTVALIDATION and PLATFORM_GATEWAY_RESPONSEVALIDATION, which are as follows in the MicroProfile Configuration form:

```
platform:
  gateway:
    requestValidation:
    responseValidation:
```

Their value is taken from a custom defined MicroProfile Configuration variable `jopl.wsdl.validation`. This variable is provided as `${jopl.wsdl.validation:true}`. The value after the colon is the default value in case the variable is not set. Therefore, in the earlier example, the default is `true` for the request validation and `false` for the response validation. The custom defined MicroProfile Configuration variable can be added to the `solution.yml` file as follows:

```
config_ordinal: 210
platform:
  services:
    instances: Audit, Rule, Scheduler
# custom configuration properties
jopl:
  wsdl:
    validation: true
```

You can pass configuration variables through the `.env` file, but another way is to add them to a `docker-compose-environment.yml` file and pass that when starting the containers.

5.6 Licensing

Running AppWorks Platform in a containerized environment requires a valid license key just as it is required to run it in a classic deployment. In the classic installation, after you provide the Customer name, Site name, and Password, the installer requests a license key. If you already have a license key, provide that in the installer. When using containers, the license information must be placed in the container configuration files.

5.6.1 Obtaining a license

The AppWorks Platform container also allows you to select one of the following two methods of obtaining a license key.

- To let the AppWorks Platform container request a license when it initializes, you must have the following information in hand:
 - License Customer Name
 - License Site Name
 - Password

- Alternatively, request a license key from OpenText up-front and provide it in the configuration.
 - When using Docker, the license key must be valid for the host name `platformcontainer.local`.
 - When using Kubernetes, the license key must be valid for the host name `ldap`. This is the default name used for LDAP in the Helm charts.

5.6.2 Applying a license when using Docker

License details are provided to the AppWorks Platform container through variables and Docker Compose secrets.

Depending on the option selected to obtain the license, you must do the following.

- If fetching a license key when the container is started for the first time:
 - Edit the following variables in the `.env` file and enter the appropriate values:
 - `FETCH_LICENSE_KEY=true`
-  **Note:** When `FETCH_LICENSE_KEY` is set to "true", ensure that you have a working internet connection.
- `LICENSE_CUSTOMER_NAME`
 - `LICENSE_SITE_NAME`
 - Edit the following secrets file and enter the password:
 - `./env/secrets/LICENSE_SITE_PASSWORD`
- If you are using a pre-generated license key:
 - Edit the following variables in the `.env` file and enter the appropriate values:
 - `FETCH_LICENSE_KEY=false`
 - `LICENSE_CUSTOMER_NAME`
 - `LICENSE_SITE_NAME`
 - Edit the following secrets file and enter the license key:
 - `./env/secrets/LICENSE_KEY`

5.6.3 Applying a license when using Kubernetes Helm chart

License details are provided to the AppWorks Platform container through Helm configuration.

Depending on the option selected to obtain the license, you must do the following.

- If fetching a license key when the container is started for the first time:
 - Enter appropriate values for the following variables in the `values.yaml` file:

- `fetchLicenseKey: "true"`



Note: When `fetchLicenseKey` is set to `"true"`, ensure that you have a working internet connection.

- `customerName`
 - `siteName`
 - `sitePassword`

- If you are using a pre-generated license key:
 - Enter appropriate values for the following variables in the `values.yaml` file:

- `fetchLicenseKey: "false"`
 - `customerName`
 - `siteName`
 - `licenseKey`

5.6.4 Renewing the AppWorks Platform license

Checking the validity of the license

You can check the validity of the license by using the License Manager in the CUSP page.

Also, in the Application Server logs you get the license key validity information as alerts.

For example:

```
<log4j:event logger="com.eibus.management.AlertSystem" timestamp="1683282048054"
level="WARN" thread="pool-1-thread-4">
<log4j:message><![CDATA[[Alert ID=Cordys.ESBServer.Messages.licenseValid] The
validity of license key is 29 day(s).]]></log4j:message>
<log4j:MDC><![CDATA[host=appworks-0 processid=1]]></log4j:MDC>
<log4j:locationInfo class="com.eibus.applicationconnector.monitor.Launcher"
method="isLicenseValid" file="Launcher.java" line="658"/>
</log4j:event>
```

Applying license on running AppWorks instance before the license expiry

You can change the license key before the license expiry date or when AppWorks Platform is up and running after the expiry by using the License Manager from the CUSP page.

1. Open the license Manger in the CUSP page.
2. In key information Panel select Change.
3. A dialog box pops up to enter new key.
4. Provide a valid license key in the license key field.

Applying license when AppWorks instance is not running and after the license is expired:

1. Go inside the LDAP pod using the following command:
 - In Kuberenetes: `Kubectl exec -it LdapPodName bash`
Example: `Kubectl exec -it ldap-0 bash`
 - In Docker: `docker exec -it LdapContainerId bash`
Example: `docker exec -it appworks_ldap_1 bash`
2. Run the following command: `export LDAPTLS_REQCERT=never`
3. Create a file in the LDAP pod named `licinfo.ldif` with the following content:

```
dn: cn=licinfo,cn=cordys,cn=defaultInst,o=opentext.net
changetype: modify
replace: licensekey
licensekey: 67ddf486-b91c531b-767d1ae1-81e15a48-25024ecc-ad075350-9992df69-
xxxxxxx(Provide a valid license key)
```

Run the following commands to create a file in the LDAP pod and add the content in the file **cat > licinfo.ldif**:

```
dn: cn=licinfo,cn=cordys,cn=InstanceName,o=Domain(exaple:dn:
cn=licinfo,cn=cordys,cn=defaultInst,o=opentext.net)
changetype: modify
replace: licensekey
licensekey: 67ddf486-b91c531b-767d1ae1-81e15a48-25024ecc-ad075350-9992df69-
xxxxxxx(Provide a valid license key)
```

4. Run the following command to add the `licinfo.ldif` file to the LDAP server:
 - In Kubernetes: `ldapmodify -H ldaps://LDAP_PODNAME:6366 -f licinfo.ldif -w adminPassword -D "ldap user"`
Example: `ldapmodify -H ldaps://ldap-0:6366 -f licinfo.ldif -w changeit -D "cn=Directory Manager,o=opentext.net"`
 - In Docker: `ldapmodify -H ldaps://LDAP_CONTAINER_ID:6366 -f licinfo.ldif -w LDAP_ADMIN_PASSWORD -D "cn=Directory Manager,o=opentext.net"`
Example: `ldapmodify -H ldaps://appworks_ldap_1:6366 -f licinfo.ldif -w dmanager -D "cn=Directory Manager,o=opentext.net"`
5. Restart the AppWorks pod.

- In Kubernetes, down scale and up scale to restart the pod by using the following command:

```
kubectl scale statefulset appworks --replicas=0  
kubectl scale statefulset appworks --replicas=1
```

- In Docker, Docker restart the appworksContainerId.

For example: `docker restart appworks_appworks_1`

Chapter 6

Building the Application image

A container image is required to run a container. Multiple containers can be based on the same container image. The `docker-compose build` command is used to build the container image. This build command uses the contents of the `Dockerfile` or `Dockerfile_init` that loads various build files and prepares a specific AppWorks Platform image.

The AppWorks Platform and OpenLDAP images are available in the OpenText registry. You can use these images to deploy AppWorks Platform or build application-specific images.

You can build the application images in the following ways:

- Extend the existing AppWorks Platform or application image to build a new application image.
- Build a separate image for the application using an init container.

Adding application-specific packages to the image

In a classic deployment, after developing an application, the Application Deployer is used to deploy the application packages in an environment. However, with container-based deployment, the application packages are not added after the image is created. If additional files are required, you must add them to the container when building the image. Therefore, when you deploy your application packages in a container, the application packages must be added during the container image build time.

Building an application-specific container might involve adding one or more of the following:

- application-specific application packages
- application-specific files such as JARs, web files, or property files
- application-specific configuration for used building blocks, specific services, email, and other integrations
- application-specific TomEE configuration, HTTPS, or enabling CORS

6.1 Extend the existing AppWorks Platform or application image to build a new application image

To build and deploy an advanced or application-specific AppWorks Platform and OpenLDAP container images:

1. Download `appworks-platform-23.4-containerization.tar.gz` and extract it to a folder. The name of this folder will be used as a prefix for the appworks, ldap, and database container names. See [“Container archive contents” on page 15](#) for more information on the contents of the archive.
2. Add your application packages to the `copy-to-image/appworks/packages/shared` folder.



Note: As a best practice, all applications packages must be signed. However, if the application packages are unsigned, you must allow them by setting the `loadUnsignedCaps` property to true. See [“Signing application packages” on page 121](#) for instructions to add signed or unsigned application-specific packages.

3. Navigate to the `copy-to-image/appworks` folder and add your application-specific configuration (`solution.yml`). See the [“Copying the solution.yml file in a container through the copy-to-image folder” on page 50](#) for instructions.
4. To deploy the platform skipped packages in the shared space, add the cap template file in the `copy-to-image\appworks\captmp\templates\customtemplates` folder. The `cap template.xml` file contains the list of the platform skipped packages in the shared space, with the `skip` value set as `skip="false"` to deploy the caps.

```
<capcommands timeout="300000">
  <cap name="Cordys BOP Language Pack_ja-JP" skip="false">
    <DeployCAP xmlns="http://schemas.cordys.com/cap/1.0" revertOnFailure="false">
      <url>http://server:port/cordys/wcp/capcontent/packages/
com.cordys.web.cap.CAPGateway.wcp?capName=Cordys BOP Language Pack_ja-JP</url>
    </DeployCAP>
  </cap>
  <cap name="Cordys BOP Language Pack_ko-KR" skip="false">
    <DeployCAP xmlns="http://schemas.cordys.com/cap/1.0" revertOnFailure="false">
      <url>http://server:port/cordys/wcp/capcontent/packages/
com.cordys.web.cap.CAPGateway.wcp?capName=Cordys BOP Language Pack_ko-KR</url>
    </DeployCAP>
  </cap>
</capcommands>
```

5. Build your application-specific image using the existing AppWorks Platform image available in the OpenText registry as follows:
`docker-compose build`

6.2 Build a separate image for the application using an init container.

To build Init container images:

1. Download `appworks-platform-22.4-containerization.tar.gz` and extract it to a folder.
2. Navigate to the extracted folder and add all your application specific configurations to the `copy-to-image` folder.
3. Navigate to the `copy-to-image/appworks` folder and do the following:
 - a. Add your application packages to the `copy-to-image/appworks/packages/shared` or `copy-to-image/appworks/packages/organization` folder.
 - b. Add your application-specific configuration `yml` to the `copy-to-image/appworks/config` folder.
 - c. Add any other application specific configurations that you want under `appworks` install directory.
4. **Optional** If you want to customize or add any configurations for TomEE installation, add your configuration files to the `copy-to-image/tomee` folder.
5. Build your application-specific init container image using the following command:
`docker build --file Dockerfile_init --tag <init-image-tag>`

You must rebuild the init container image, if there are any modifications to the `copy-to-image` folder content.

The Application image is built and stored in the local Docker repository.



Note: You must rebuild the container image whenever the following artifacts are modified:

- build files
- configuration properties in the `solution.yml` file.
- application packages

To upgrade the application init containers

1. Navigate to the `helm-charts/appworks` folder.
2. Open the `values.yaml` file and update the new tag details for the corresponding init container images and perform a Helm upgrade.
 - After the Helm upgrade is completed, With the helm upgrade, the pods restart.
 - After each pod restarts, the init image also restarts with a new image tag.

- When the pods are ready, the upgraded applications/configurations are applied on the AppWorks Platform container.



Note: If you deploy multiple init images with the same AppWorks container and if the images use the same file names under the same folder paths while building the init images, the configuration files might be overwritten.

Example: If Init image1 adds a customized `server.xml` file under `copy-to-image/tomee/conf/server.xml` and Init image2 also adds the same file with a different customization at the same path `copy-to-image/tomee/conf/server.xml`. Then deploying these init images only one configuration is applied on AppWorks container's TomEE instance.

The last init container's configuration in the `values.yaml` file will take the precedence.

Based on your need, you can use these images and do the following:

- Deploy applications on Docker. See [“Deploying Application images on Docker” on page 61](#).
- Deploy applications on Kubernetes. See [“Deploying Application images on Kubernetes” on page 69](#).
- Push the application image to the Microsoft Azure registry and deploy it. See [“Deploying Application images on Microsoft Azure” on page 73](#).
- Push the application image to the Google Cloud Platform registry and deploy it. See [“Deploying Application images on Google Cloud Platform” on page 79](#).
- Push the application image to the Amazon Web Services registry and deploy it. See [“Deploying Application images on Amazon Web Services” on page 83](#).

Chapter 7

Deploying Application images on Docker

AppWorks Platform provides a set of files that enable you to configure and deploy a standard AppWorks Platform environment that includes OpenLDAP and PostgreSQL running in their own containers. While this enables you to quickly set up and run the default containers on Docker, if required, you can also modify the configuration settings to use other databases or LDAP applications.

7.1 Prerequisites

Ensure that the deployment requirements specified in “[Deployment requirements](#)” on page 19 are met.



Note: Login to the OpenText registry: `docker login registry.opentext.com` using your OpenText Connect credentials.

7.2 Deployment of default containers in Docker

For a simplified deployment process, AppWorks Platform provides default containers that can be configured to run in a Docker environment. Most of the configuration settings for these containers are preset and provide a standard AppWorks Platform environment. PostgreSQL and OpenLDAP are configured as the default database and LDAP applications.

You can start the containers using the `docker-compose` command. Docker-compose accepts a set of compose files as input. The minimal set of compose files that must be provided when starting the containers are:

- `docker-compose.yml` - contains the configuration settings to run the AppWorks Platform service.
- `docker-compose-openldap.yml` - contains the configuration settings to run the OpenLDAP service.
- `docker-compose-postgres.yml` - contains the configuration settings to run the PostgreSQL database service.

To run the default containers in Docker:

1. Navigate to the folder where the `appworks-platform-23.4-containerization.tar.gz` is extracted.
2. Add the Application and OpenLDAP image details to the `docker-compose.yml` and `docker-compose-openldap.yml` files respectively.
3. Modify the relevant configuration files to apply license information. See “[Licensing](#)” on page 51.

4. Run the `docker-compose` command to start the containers:`docker-compose -f docker-compose.yml -f docker-compose-openldap.yml -f docker-compose-postgres.yml up -d`
5. Check the health status of the containers using the `docker` command: `docker ps`



Note: This might approximately take 10 minutes to complete. Ensure that the status of all the containers is healthy.

6. Sign into AppWorks Platform explorer - `http://localhost:8080/home/system` using the following default credentials:
 - User name - `sysadmin`
 - Password - As provided in the secrets file for `PLATFORM_PASSWORD`



Note: If you want to add your own configuration and application packages to the container image, see [“Externalized configuration” on page 24](#) and [“Application-specific configuration to the image” on page 48](#).

7.3 Advanced deployment of Applications in Docker

Apart from the simplified deployment described above, you can also do the following:

- Configure to use other databases such as Oracle, MS SQL Server, or Azure SQL Database.
- Enable JMX monitoring and remote debugging options.
- Configure to connect to a non-containerized OpenLDAP installed through OpenText CARS if required.

To perform the advanced deployment of Applications in Docker:

1. Navigate to the folder where the `appworks-platform-23.4-containerization.tar.gz` is extracted.
2. Add the Application and OpenLDAP image details to the `docker-compose.yml` and `docker-compose-openldap.yml` files respectively.
3. Based on your requirement, modify the database-specific `docker-compose` configuration files. See [“Using a specific database” on page 63](#) for database-specific configuration details.
4. Apply license information. See [“Licensing” on page 51](#).
5. Run the `docker-compose` command to start the containers:`docker-compose -f docker-compose.yml -f docker-compose-openldap.yml -f docker-compose-<database>.yml up -d`

Example: If using Oracle, run the following command: `docker-compose -f docker-compose.yml -f docker-compose-openldap.yml -f docker-compose-oracle.yml up -d`

6. **Optional** If required, you can enable the remote debugging facility to the container by adding `docker-compose-remote-debugging.yml` option to the `docker-compose` command.
7. **Optional** If required, you can enable JMX configuration in AppWorks Platform by adding the `docker-compose-jmx-monitoring.yml` option.



Note: The following command can be used to start the container with the OpenLDAP application, Oracle database, remote debugging option, and JMX monitoring enabled: `docker-compose -f docker-compose.yml -f docker-compose-openldap.yml -f docker-compose-oracle.yml -f docker-compose-remote-debugging.yml up -d`
The `-d` command runs it in a detached mode.

8. Check the health status of the containers using the `docker` command: `docker ps`



Note: This might approximately take 10 minutes to complete. Ensure that the status of all the containers is healthy.

9. Sign into AppWorks Platform explorer - `http://localhost:8080/home/system` using the following default credentials:
 - User name - `sysadmin`
 - Password - As provided in the secrets file for `PLATFORM_PASSWORD`

7.4 Using a specific database

By default, the AppWorks Platform containerization uses the PostgreSQL container image. If required, you can change the database to one of the supported databases - Oracle, MS SQL Server, or Azure SQL Database. The Oracle and MS SQL Server databases can be run either in a container environment or a non-container environment and the Azure SQL database is available as a service in the Microsoft Azure environment. However, when using a non-container database environment, you can omit the `docker-compose` file for the database service while starting the containers. In that scenario, the configuration attempts to connect the database service and use it.

For details on using an existing database, see the [Prerequisites for an existing database](#) section in the AppWorks Platform Installation Guide on My Support.

The information on different configuration variables required per database are explained below:

PostgreSQL

The `docker-compose` file for PostgreSQL, `docker-compose-postgres.yml`, loads a public available container image.

To use an already existing database server, specify the following configuration variables. These can be set in the `.env` file.

Configuration Variable	Description	Example
DATABASE_USER	Database used to connect to the PostgreSQL database.	appworks
DATABASE_PASSWORD (this is a secret)	Password of the database user.	Jo4nD0e!
DATABASE_NAME	Name of the database to connect to.	appworks
DATABASE_HOST	Database server name.	database
DATABASE_CREATE	Database to be created by AppWorks Platform. If <code>true</code> , AppWorks Platform creates the database.	true
DATABASE_ADMIN_USER	Name of the database admin user.	postgres
DATABASE_ADMIN_PASSWORD (this is a secret)	Password of the database admin user.	J@neD0e!
PLATFORM_DATABASE_CONNECTIONS_SYSTEM_USESSL	Communication mode between the Platform and database server. When set to <code>true</code>, AppWorks Platform uses SSL communication with the Database server during deployment. Default: false	true
TRUST_DB_CERTS	SSL certificate of the database. When set to <code>false</code>, the SSL certificate of the database must be added to <code>copy-to-image\appworks\trustCertificates</code> folder while building the AppWorks Platform image. This property is applicable only when PostgreSQL is used as the database and above property is set to <code>true</code>. Default: true	

Oracle

The docker-compose file for Oracle, `docker-compose-oracle.yml`, loads a public available container image.

To use an already existing database server, specify the following configuration variables. These can be set in the `.env` file.

Configuration Variable	Description	Example
PLATFORM_DATABASE_CONNECTIONS_SYSTEM_CONNECTIONSTRING	Database connection string	jdbc:oracle:thin:@otora.lab.opentext.com:1521/orcl.lab.opentext.com
DATABASE_CREATE	Database to be created by AppWorks Platform. If <code>true</code> , AppWorks Platform creates the database.	false
DATABASE_HOST	Database server name	otora.lab.opentext.com
DATABASE_USER	Database user to connect to the Oracle database	awpora
DATABASE_PASSWORD	Password of the database user	Oradoc_db1
DATABASE_NAME	Name of the database to connect to.	awpora
DATABASE_ADMIN_USER	Name of the database admin user.	sys as sysdba
DATABASE_ADMIN_PASSWORD	Password of the database admin user.	Oradoc_db1
ORACLE_TABLESPACE_NAME	Name of the Oracle tablespace to use.	SYSTEM
ORACLE_TABLESPACE_PATH	Path of the table space.	/tablespace

MS SQL Server

The docker-compose file for MS SQL Server, `docker-compose-mssql.yml`, loads a public available container image. By default, the `sa` user is used as `DATABASE_USER`. If needed, create another user before starting the AppWorks Platform container.

! Important

The EULA of MS SQL Server must be manually accepted. This can be done by adding `ACCEPT_EULA=Y` in the `.env` file.

To use an already existing database server, specify the following configuration variables. These can be set in the `.env` file.

Configuration Variable	Description	Example
DATABASE_CREATE	Database to be created by AppWorks Platform. If <code>true</code> , AppWorks Platform creates the database.	true

Configuration Variable	Description	Example
DATABASE_HOST	Database server name.	database
DATABASE_USER	Database user to connect to the MS SQL Server database.	sa
DATABASE_PASSWORD(this is a secret)	Password of the database user.	Jo4nD0e!
DATABASE_NAME	Name of the database to connect to.	appworks
DATABASE_ADMIN_USER	Name of the database admin user.	sa
DATABASE_ADMIN_PASSWORD(this is a secret)	Password of the database admin user.	J@neD0e!

Azure SQL Database

Azure SQL Database is available as a service in Microsoft Azure environment. The following configuration variables can be set in the `.env` file.

Configuration Variable	Description	Example
DATABASE_CREATE	Database to be created by AppWorks Platform. If <code>true</code> , AppWorks Platform creates the database.	<code>true</code>
DATABASE_HOST	Database server name.	database
DATABASE_USER	Database user to connect to the Azure SQL Database.	sa
DATABASE_PASSWORD (this is a secret)	Password of the database user.	Jo4nD0e!
DATABASE_NAME	Name of the database to connect to.	appworks
DATABASE_ADMIN_USER	Name of the database admin user.	AzureAdmin
DATABASE_ADMIN_PASSWORD(this is a secret)	Password of the database admin user.	J@neD0e!

7.5 Accessing AppWorks Platform

During the initial startup of a new AppWorks Platform container, all the artifacts are not yet deployed in LDAP and databases. They deploy when the first node in a cluster starts, which usually takes around 10 minutes. During this time, the status of the AppWorks container is displayed as `unhealthy`, indicating that it is not yet ready to take the requests.

- Run `docker ps` to see the container status.

After the container initialization step is complete, the status of the AppWorks container switches to `healthy`.

The default access URL AppWorks Platform is - `http://localhost:8080/home/system`. Use this URL to open AppWorks Platform. The default username to log in is `sysadmin` and the password is as specified in the `PLATFORM_PASSWORD` configuration variable of the secrets file. The default port is 8080 as specified in the `PLATFORM_PORT` configuration variable.

7.6 Stopping containers

- To stop the containers, use the `docker-compose` command and run it in the `docker` folder:

```
docker-compose stop
```

This stops the containers but will not delete them. It is advised to stop the containers before stopping the host Operating System.

7.7 Removing containers

- To stop and remove the containers, use the following command:

```
docker-compose down
```

- To force the deletion of volumes add `-v` or `--volumes`:

```
docker-compose down --volumes
```

7.8 Accessing log files

Use the `docker-compose logs` command to see all the logging performed for the containers. For example:

- To print all logging available in the container until now, use:

```
docker-compose logs appworks
```

- To show the last 40 lines of logging, use:

```
docker-compose logs --tail 40 appworks
```

- To follow the logs as they are generated, use:

```
docker-compose logs --follow appworks
```

Chapter 8

Deploying Application images on Kubernetes

Kubernetes is a portable, extensible open-source platform orchestration engine for automating deployment, scaling, and managing containerized applications. It can be considered as a containerized, microservices, and portable cloud platform. It provides a container-centric management environment.

You can use Kubernetes to deploy the AppWorks Platform cluster using the AppWorks Platform Docker images and Helm charts.

8.1 Prerequisites

Ensure that all “[Deployment requirements](#)” on page 19 are met.

8.2 Deploying Application images in Kubernetes

You can deploy the Application, OpenLDAP and Init images on Kubernetes using the Helm chart scripts.

To deploy the Application, OpenLDAP and Init images using Helm charts:

1. Download `appworks-platform-23.4-containerization.tar.gz` and extract it to a folder. The name of this folder will be used as a prefix for the appworks, ldap, and database container names.
2. Navigate to the `helm-charts/appworks` folder.
3. Open the `values.yaml` file and do the following:

Application and OpenLDAP images	• Enter the Application image details. • Enter the LDAP image details. • Enter the license details. • Replace \$NAMESPACE with your Kubernetes cluster namespace. • Replace <storageClass> with the storage provider class name. • Enter the number of AppWorks pods that you require in the replicaCount parameter. • Enter the number of LDAP pods that you require in the ldap.replicaCount parameter. Example: <pre>image: registry: registry.opentext.com repository: otawp tag: 22.4</pre>  Note: The default LDAP replica count value is 1 but you can increase it to its maximum value: 2. If the replica count is 2, the data between the two LDAP pods is sync replicated by the master-master sync replication. • Enter Appworks password, LDAP password, and database password.
--	--

Init images	• Enter the <code>init</code> image details. • Enter the license details. • Replace <code>\$NAMESPACE</code> with your Kubernetes cluster namespace. • Replace <code><storageClass></code> with the storage provider class name. • Enter the number of AppWorks pods that you require in the <code>replicaCount</code> parameter. • Enter the number of LDAP pods that you require in the <code>ldap.replicaCount</code> parameter. Example: <pre>initContainers: - name: app1 image: registry: artifactory.otxlab.net repository: appworks_dev_himalaya/init/app1 tag: '1.0' pull policy: Always</pre>  Note: The default LDAP replica count value is 1 but you can increase it to its maximum value: 2. If the replica count is 2, the data between the two LDAP pods is sync replicated by the master-master sync replication. • Enter Appworks password, LDAP password, and database password.
-------------	---

See “[Configuration files for Helm charts](#)” on page 32 for more information on the contents in various `values.yaml` files.

4. To configure the Azure, GCP, and AWS yaml files:

Azure	For Azure specific default settings, update <code>helm-charts/appworks/platform/azure.yaml</code> See “azure.yaml” on page 41 for more information on the contents in the <code>azure.yaml</code> file.
GCP	For GCP specific default settings, update <code>helm-charts/appworks/platform/gcp.yaml</code> See “gcp.yaml” on page 43 for more information on the contents in the <code>gcp.yaml</code> file.

AWS	For AWS specific default settings, update <code>helm-charts/appworks/platform/aws.yml</code> See “aws.yml” on page 46 for more information on the contents in the <code>aws.yml</code> file.
-----	--

5. Change the working directory to the `helm-charts` folder and run the following commands:

- `helm dep update ./appworks`
- `helm install appworks ./appworks files.`

Azure	To apply the <code>azure.yml</code>, run the following command: <pre>helm install appworks -f ./appworks/platform/azure.yml ./appworks</pre>
GCP	To apply the <code>gcp.yml</code>, run the following command: <pre>helm install appworks -f ./appworks/platform/gcp.yml ./appworks</pre>
AWS	To apply the <code>aws.yml</code>, run the following command: <pre>helm install appworks -f ./appworks/platform/aws.yml ./appworks</pre>



Note: After you run the `helm install` command with each AppWorks pod, the `init` containers run first and add their application configuration to the main AppWorks container. Any application packages or configurations added as a part of `init` images are deployed on the AppWorks platform container.

6. Run the following command to obtain the status of each pod:
`kubectl get pods`
7. If the `ldap.replicaCount` is greater than one, after all the AppWorks pod are live and ready, you must delete all the LDAP pods sequentially. The new LDAP pods are created and content is synced between them.

Chapter 9

Deploying Application images on Microsoft Azure

You can create a Kubernetes cluster, connect it to Microsoft Azure, and then deploy your application images on it.

9.1 Prerequisites

Before deploying Application images on Microsoft Azure, ensure that you have the following:

- The deployment requirements specified in “[Deployment requirements](#)” on page 19 are met.
- The Azure CLI tool installed on the computer where the image is available. You must use this tool to execute the commands provided in the sections below.
 - For installation instructions on Windows, see <https://docs.microsoft.com/en-us/cli/azure/install-azure-cli-apt?view=azure-cli-latest>
 - For installation instructions on Linux and other operating systems, see <https://docs.microsoft.com/en-us/cli/azure/install-azure-cli?view=azure-cli-latest>
- An Azure logon account.
- Access to an Azure container registry.
- The required Application image.
- OpenLDAP for AppWorks Platform image or the OpenText CARS installer.
- A Kubernetes cluster (AKS) on Azure for AppWorks Platform deployment. For instructions on creating a Kubernetes cluster, see <https://docs.microsoft.com/en-us/azure/aks/kubernetes-walkthrough-portal#create-an-aks-cluster>



Note: A Node pool with sufficient compute power for your project must be configured. For example: Two nodes with 2vCPUs and 8 GB memory on each node.

- A database service/server on Azure for AppWorks Platform.
- Ingress installed and configured under the same namespace of the AKS cluster.
- Azure provides azurefile as the default storage. You must change the mount option of the azurefile storage class to include the `nobrl` option.
- Open the azurefile using the following command: `kubectl edit storageclass azurefile`
- Navigate to `mountoptions`, and then add the `- nobrl` option.

Example:

```
mountoptions:  
- nobrl
```

9.2 Deploying application images on Azure

Deploying application images on Azure comprises the following steps:

1. “Pushing the Application and OpenLDAP images to the Azure container registry” on page 74
2. “Deploying the Application image on Azure” on page 75

9.2.1 Pushing the Application and OpenLDAP images to the Azure container registry

1. Log into the Azure container registry using the following command:
`az acr login --name <name>`
2. Tag the Application and OpenLDAP images using the following command:

- `docker tag registry.opentext.com/otawp-openldap:<version> <Azure container registry name>/<Tag name>`

Example: `docker tag registry.opentext.com/otawp-openldap:22.2 awpce.azurecr.io/otawp-openldap:22.2`

- `docker tag registry.opentext.com/otawp:<version> <Azure container registry name>/<Tag name>`

Example: `docker tag registry.opentext.com/otawp:23.4 awpce.azurecr.io/otawp:22.2`

3. Push the image to the ACR using the following command:
`docker push <Azure container registry name>/<Tag name>`

Example:

- `docker push awpce.azurecr.io/otawp-openldap:22.2`
- `docker push awpce.azurecr.io/otawp:22.2`

4. Verify whether the image is pushed into ACR. For instructions, see <https://docs.microsoft.com/en-us/azure/container-registry/container-registry-get-started-portal#list-container-images>

9.2.2 Deploying the Application image on Azure

You can deploy the Application, OpenLDAP and Init images on Kubernetes using the Helm chart scripts.

To deploy the Application, OpenLDAP and Init images using Helm charts:

1. Download `appworks-platform-23.4-containerization.tar.gz` and extract it to a folder. The name of this folder will be used as a prefix for the appworks, ldap, and database container names.
2. Navigate to the `helm-charts/appworks` folder.
3. Open the `values.yaml` file and do the following:

Application and OpenLDAP images

- Enter the Application image details.
- Enter the LDAP image details.
- Enter the license details.
- Replace `$NAMESPACE` with your Kubernetes cluster namespace.
- Replace `<storageClass>` with the storage provider class name.
- Enter the number of AppWorks pods that you require in the `replicaCount` parameter.
- Enter the number of LDAP pods that you require in the `ldap.replicaCount` parameter.

Example:

```
image:
  registry: registry.opentext.com
  repository: otawp
  tag: 22.4
```



Note: The default LDAP replica count value is 1 but you can increase it to its maximum value: 2. If the replica count is 2, the data between the two LDAP pods is sync replicated by the master-master sync replication.

- Enter Appworks password, LDAP password, and database password.

Init images	• Enter the init image details. • Enter the license details. • Replace \$NAMESPACE with your Kubernetes cluster namespace. • Replace <storageClass> with the storage provider class name. • Enter the number of AppWorks pods that you require in the replicaCount parameter. • Enter the number of LDAP pods that you require in the ldap.replicaCount parameter. Example: <pre>initContainers: - name: app1 image: registry: artifactory.otxlab.net repository: appworks_dev_himalaya/init/app1 tag: '1.0' pull policy: Always</pre>  Note: The default LDAP replica count value is 1 but you can increase it to its maximum value: 2. If the replica count is 2, the data between the two LDAP pods is sync replicated by the master-master sync replication. • Enter Appworks password, LDAP password, and database password.
-------------	--

See “[Configuration files for Helm charts](#)” on page 32 for more information on the contents in various values.yaml files.

4. To configure the Azure, GCP, and AWS yaml files:

Azure	For Azure specific default settings, update helm-charts/appworks/platform/azure.yaml See “azure.yaml” on page 41 for more information on the contents in the azure.yaml file.
GCP	For GCP specific default settings, update helm-charts/appworks/platform/gcp.yaml See “gcp.yaml” on page 43 for more information on the contents in the gcp.yaml file.

AWS	For AWS specific default settings, update <code>helm-charts/appworks/platform/aws.yml</code> See “aws.yml” on page 46 for more information on the contents in the <code>aws.yml</code> file.
-----	--

5. Change the working directory to the `helm-charts` folder and run the following commands:

- `helm dep update ./appworks`
- `helm install appworks ./appworks files.`

Azure	To apply the <code>azure.yml</code>, run the following command: <pre>helm install appworks -f ./appworks/platform/azure.yml ./appworks</pre>
GCP	To apply the <code>gcp.yml</code>, run the following command: <pre>helm install appworks -f ./appworks/platform/gcp.yml ./appworks</pre>
AWS	To apply the <code>aws.yml</code>, run the following command: <pre>helm install appworks -f ./appworks/platform/aws.yml ./appworks</pre>



Note: After you run the `helm install` command with each AppWorks pod, the `init` containers run first and add their application configuration to the main AppWorks container. Any application packages or configurations added as a part of `init` images are deployed on the AppWorks platform container.

6. Run the following command to obtain the status of each pod:


```
kubectl get pods
```
7. If the `ldap.replicaCount` is greater than one, after all the AppWorks pod are live and ready, you must delete all the LDAP pods sequentially. The new LDAP pods are created and content is synced between them.

After all the pods are running and ready, you can start using AppWorks Platform.

Chapter 10

Deploying Application images on Google Cloud Platform

Google Kubernetes Engine (GKE) is a managed Kubernetes service provided by Google and is used to create a Kubernetes cluster on Google Cloud Platform (GCP).

10.1 Prerequisites

Before deploying the Application images on GCP, ensure that you have the following:

- The deployment requirements specified in “**Deployment requirements**” on page 19 are met.
- The latest version of the Google Cloud SDK installed on your computer including the gcloud command-line tool. You must use this tool to execute the commands provided in the sections below.
- A GCP cluster created and configured to deploy AppWorks Platform. For instructions on creating a GCP cluster, see <https://cloud.google.com/kubernetes-engine/docs/how-to/creating-a-regional-cluster>



Note: A Node pool with sufficient compute power for your project must be configured. For example: Two nodes with 2vCPUs and 8 GB memory on each node.

- Logon credentials to log into GCP.
- A project in GCP. This will be used as the primary location to hold all artifacts in GCP.
- An external storage configured with the capability to dynamically provision both ReadWriteOnce(RWO) Persistent Volumes (PV) and ReadWriteMany(RWX) Persistent volumes (PV).
- Ingress installed and configured under the same namespace of the GKE cluster.

Deploying AppWorks Platform on GCP comprises the following steps:

1. “**Pushing the Application and OpenLDAP images to Google Container Registry**” on page 80
2. “**Deploying the Application image on Google Cloud Platform**” on page 80

10.2 Pushing the Application and OpenLDAP images to Google Container Registry

To push the Application and OpenLDAP images to Google Container Registry:

1. Log into the Google Cloud Platform using the Google cloud SDK:

```
gcloud auth login
```
2. Run the following command to configure Docker to use gcloud as a credential helper:

```
gcloud auth configure-docker
```
3. Tag the Application and OpenLDAP images in the following format:
 - `docker tag registry.opentext.com/otawp:<version> <GCP hostname>/<GCP Projectname>/<Image>`

Example: `docker tag registry.opentext.com/otawp:22.2 gcr.io/crafty-biplane-278109/otawp:22.2`
 - `docker tag registry.opentext.com/otawp-openldap:22.2 <GCP Projectname>/<Image>`

Example: `docker tag registry.opentext.com/otawp-openldap:22.2 gcr.io/crafty-biplane-278109/otawp-openldap:22.2`
4. Push the tagged image to Google container registry.
 - `docker push <Application_image-id> <GCP hostname>/<GCP Project name>/<Image>`

Example:
 - `docker push gcr.io/crafty-biplane-278109/otawp-openldap:22.2`
 - `docker push gcr.io/crafty-biplane-278109/otawp:22.2`

10.3 Deploying the Application image on Google Cloud Platform

You can deploy the Application and OpenLDAP images on Kubernetes using the Helm chart scripts.

To deploy the Application and OpenLDAP images using Helm charts:

1. Download `appworks-platform-23.4-containerization.tar.gz` and extract it to a folder. The name of this folder will be used as a prefix for the appworks, ldap, and database container names.
2. Navigate to the `helm-charts/appworks` folder.

3. Open the `values.yaml` file and do the following:

- Enter the Application image details.
- Enter the LDAP image details.
- Enter the license details.
- Replace `$NAMESPACE` with your Kubernetes cluster namespace.
- Replace `<storageClass>` with the storage provider class name. OpenText recommends to use NFS-server provisioner as the storage class.

See [“Configuration files for Helm charts” on page 32](#) for more information on the contents in various `values.yaml` files.

4. This step is applicable only if you are deploying AppWorks Platform on GKE.

- a. Open the `helm-charts\appworks\templates\values.yml` file.
- b. Search for `dnsConfig` and update the code in the following format:

```
dnsPolicy: "None"
dnsConfig:
  nameservers:
    - <IP address of the Kube DNS service>. You can obtain it using the
      following command: kubectl get svc -n kube-system
  searches:
    - appworks.<namespace>.svc.cluster.local
    - <namespace>.svc.cluster.local
    - svc.cluster.local
    - cluster.local
  options:
    - name: ndots
```

5. Change the working directory to the `helm-charts` folder and run the following commands:

- `helm dep update ./appworks`
- `helm install appworks ./appworks`

6. Run the following command to obtain the status of each pod:

```
kubectl get pods
```

After all the pods are running and ready, you can start using the Application.

Chapter 11

Deploying Application images on Amazon Web Services

You can deploy your application images using Amazon Elastic Kubernetes Service (EKS), the Kubernetes container orchestration platform.

11.1 Prerequisites

Before deploying the Application images on Amazon Web Services (AWS), ensure that you have the following:

- The deployment requirements specified in “[Deployment requirements](#)” on page 19 are met.
- The AWS CLI tool installed on the computer where the image is available. You must use this tool to execute the commands provided in the sections below. See <https://docs.aws.amazon.com/cli/latest/userguide/install-windows.html> for more information.
- The AWS IAM authenticator for Kubernetes installed to provide authentication to the Kubernetes cluster.
- The eksctl tool for working with EKS clusters.
- The cURL tool to transfer data from or to a server.
- The Wget utility for retrieving files using HTTP, HTTPS, FTP and FTPS protocols.
 - To configure AWS CLI to connect to your AWS environment, run the following command: `aws configure`. The command will prompt you to enter your ID, secret key, region, and default output format. For more information on the `aws configure` command, see <https://docs.aws.amazon.com/cli/latest/reference/configure>.
- An Amazon EKS cluster for AppWorks Platform deployment. For instructions on creating a Kubernetes cluster <https://docs.aws.amazon.com/eks/latest/userguide/create-cluster.html>.



Notes

- Before creating the cluster, ensure that you have run the AWS configure command and provided your ID, secret key, region, and default output format.
- A Node pool with sufficient compute power for your project must be configured. For example: Two nodes with 2vCPUs and 8 GB memory on each node.

- Access to Amazon Elastic Container Registry (ECR).
- An external storage configured with the capability to dynamically provision both ReadWriteOnce(RWO) Persistent Volumes (PV) and ReadWriteMany(RWX) Persistent volumes (PV).
- Ingress installed and configured under the same namespace of the EKS cluster.

11.2 Pushing the Application and OpenLDAP images to ECR

1. Log into ECR using the following command:

```
aws ecr get-login-password --region <region> | docker login --username  
<user name> --password-stdin <location of the ECR on Amazon cloud>
```

2. Tag the Docker images using the following command:

```
docker tag registry.opentext.com/otawp:<version> <AppWorks  
Platform_image-id> <ECR hostname>/<ECR project name><image>.
```

Example: `docker tag registry.opentext.com/otawp:22.2 048736801737.dkr.ecr.us-east-2.amazonaws.com/otawp:22.2`

3. Push the tagged image to ECR.

```
docker push <ECR hostname>/<ECR name>:tag
```

Example: `docker push 048736801737.dkr.ecr.us-east-2.amazonaws.com/otawp:22.`

11.3 Deploying the Application and OpenLDAP images on Amazon Web Services

You can deploy the Application, OpenLDAP and Init images on Kubernetes using the Helm chart scripts.

To deploy the Application, OpenLDAP and Init images using Helm charts:

1. Download `appworks-platform-23.4-containerization.tar.gz` and extract it to a folder. The name of this folder will be used as a prefix for the appworks, ldap, and database container names.
2. Navigate to the `helm-charts/appworks` folder.
3. Open the `values.yaml` file and do the following:

Application and OpenLDAP images	• Enter the Application image details. • Enter the LDAP image details. • Enter the license details. • Replace \$NAMESPACE with your Kubernetes cluster namespace. • Replace <storageClass> with the storage provider class name. • Enter the number of AppWorks pods that you require in the replicaCount parameter. • Enter the number of LDAP pods that you require in the ldap.replicaCount parameter. Example: <pre>image: registry: registry.opentext.com repository: otawp tag: 22.4</pre>  Note: The default LDAP replica count value is 1 but you can increase it to its maximum value: 2. If the replica count is 2, the data between the two LDAP pods is sync replicated by the master-master sync replication. • Enter Appworks password, LDAP password, and database password.
---------------------------------	--

Init images	• Enter the init image details. • Enter the license details. • Replace \$NAMESPACE with your Kubernetes cluster namespace. • Replace <storageClass> with the storage provider class name. • Enter the number of AppWorks pods that you require in the replicaCount parameter. • Enter the number of LDAP pods that you require in the ldap.replicaCount parameter. Example: <pre>initContainers: - name: app1 image: registry: artifactory.otxlab.net repository: appworks_dev_himalaya/init/app1 tag: '1.0' pull policy: Always</pre>  Note: The default LDAP replica count value is 1 but you can increase it to its maximum value: 2. If the replica count is 2, the data between the two LDAP pods is sync replicated by the master-master sync replication. • Enter Appworks password, LDAP password, and database password.
-------------	--

See “[Configuration files for Helm charts](#)” on page 32 for more information on the contents in various values.yaml files.

4. To configure the Azure, GCP, and AWS yaml files:

Azure	For Azure specific default settings, update helm-charts/appworks/platform/azure.yaml See “azure.yaml” on page 41 for more information on the contents in the azure.yaml file.
GCP	For GCP specific default settings, update helm-charts/appworks/platform/gcp.yaml See “gcp.yaml” on page 43 for more information on the contents in the gcp.yaml file.

AWS	For AWS specific default settings, update <code>helm-charts/appworks/platform/aws.yml</code> See “aws.yml” on page 46 for more information on the contents in the <code>aws.yml</code> file.
-----	---

5. Change the working directory to the `helm-charts` folder and run the following commands:

- `helm dep update ./appworks`
- `helm install appworks ./appworks files.`

Azure	To apply the <code>azure.yml</code> , run the following command: <code>helm install appworks -f ./appworks/platform/azure.yml ./appworks</code>
GCP	To apply the <code>gcp.yml</code> , run the following command: <code>helm install appworks -f ./appworks/platform/gcp.yml ./appworks</code>
AWS	To apply the <code>aws.yml</code> , run the following command: <code>helm install appworks -f ./appworks/platform/aws.yml ./appworks</code>



Note: After you run the `helm install` command with each AppWorks pod, the `init` containers run first and add their application configuration to the main AppWorks container. Any application packages or configurations added as a part of `init` images are deployed on the AppWorks platform container.

6. Run the following command to obtain the status of each pod:
`kubectl get pods`
7. If the `ldap.replicaCount` is greater than one, after all the AppWorks pod are live and ready, you must delete all the LDAP pods sequentially. The new LDAP pods are created and content is synced between them.

After all the pods are running and ready, you can start using AppWorks Platform.



Note: Based on your application requirements, if you want to add additional configurations to the container, see [“Application-specific configuration to the image” on page 48](#).

Chapter 12

Deploying Application images on Red Hat OpenShift

You can deploy your application images on the Red Hat OpenShift environment on various cloud platform.



Note: The following steps explain the procedure to set up the OpenShift cluster on Amazon Web Services (AWS). See the Red Hat OpenShift documentation for specific procedures to setup the OpenShift cluster on other cloud platforms.

12.1 Prerequisites

Before deploying the Application images on Red Hat OpenShift, do the following:

- Ensure that the deployment requirements specified in “**Deployment requirements**” on page 19 are met.
- Download the supported version of the Helm package from the Helm website. See the product Release Notes (<https://github.com/helm/helm/releases>) for information about the supported versions.
- Setup the Red Hat account from the Red Hat website (<https://console.redhat.com/>).
- Download and configure the OpenShift CLI (OC) from the Red Hat website. See Red Hat OpenShift Documentation (https://docs.openshift.com/container-platform/4.10/cli_reference/openshift_cli/getting-started-cli.html) for more information.
- Download and configure the ROSA CLI from the Red Hat website (<https://console.redhat.com/openshift/downloads>).
- Download and configure the AWS CLI from Amazon website (<https://aws.amazon.com/cli/>).
- Create a ROSA (Red Hat OpenShift Service on AWS) cluster. See Red Hat OpenShift Documentation (https://www.rosaworkshop.io/rosa/1-account_setup/) for more information.
- Download the `appworks-<version>.tar.gz` Helm chart from OpenText My Support and unzip the Helm package to a folder.
- Configure an external storage with the capability to dynamically provision both ReadWriteOnce(RWO) Persistent Volumes (PV) and ReadWriteMany(RWX) Persistent volumes (PV).
- Install and configure Ingress under the OpenShift cluster.

- Ensure that you have the required database server credentials to deploy AppWorks Platform.
- When a user logs in to Red Hat OpenShift, by default, the user is added to the restricted security context constraints (SCC). For some privilege operations such as `chmod`, you must provide specific privileges to the pods. Use the following command to add all authenticated users to anyuid SCC instead of restricted:

```
oc adm policy add-scc-to-group anyuid system:authenticated
```

12.2 Deploying application images on Red Hat OpenShift

Deploying application images on Red Hat OpenShift comprises the following steps:

1. [“Pushing the application and OpenLDAP images to Image Stream” on page 90](#)
2. [“Deploying the Application image on OpenShift” on page 91](#)

12.2.1 Pushing the application and OpenLDAP images to Image Stream

To push the application and OpenLDAP images to Image Stream:

1. Create an ImageStream repository to store Docker images on the OpenShift service.
2. Log into the ImageStream repository using the following command format:

```
docker login -u <username> -p $(oc whoami -t) <public image repository>.
```

Example:

```
$docker login -u openshifttestawp -p $(oc whoami -t) default-route-openshift-image-registry.apps.awp-cluster.6mjz.p1.openshiftapps.com/openshiftapps.com/awp/awp-is
```

3. Download the Docker images from OpenText Container Registry.
4. Tag the Docker image to the OpenShift ImageStream repository to specific image using the following command format:

```
docker tag <source image> <destination image>
```

Example: AppWorks Platform -

```
docker tag artifactory.otxlab.net/appworks_dev_himalaya/wip-build/appworks/platform:latest default-route-openshift-image-registry.apps.awp-cluster.6mjz.p1.openshiftapps.com/awp/awp-is:platform
```

LDAP -

```
docker tag artifactory.otxlab.net/appworks_dev_himalaya/wip-build/appworks/openldap:latest default-route-openshift-image-registry.apps.awp-cluster.6mjz.p1.openshiftapps.com/awp/awp-is:ldap
```

5. Upload the Docker image to the OpenShift ImageStream using the following command format:

```
docker push <image>
```

Example: AppWorks Platform - docker push default-route-openshift-image-registry.apps.awp-cluster.6mjz.p1.openshiftapps.com/awp/awp-is:platform

LDAP - docker push default-route-openshift-image-registry.apps.awp-cluster.6mjz.p1.openshiftapps.com/awp/awp-is:ldap

12.2.2 Deploying the Application image on OpenShift

You can deploy the Application and OpenLDAP images on OpenShift using the Helm chart scripts.

To deploy the Application and OpenLDAP images using Helm charts:

1. Download `appworks-platform-23.4-containerization.tar.gz` and extract it to a folder. The name of this folder will be used as a prefix for the appworks, ldap, and database container names.
2. Navigate to the `helm-charts/appworks` folder.
3. Open the `values.yaml` file and do the following:
 - Enter the Application image details.
 - Enter the LDAP image details.
 - Enter the license details.
 - Enter the database details.
 - Replace `$NAMESPACE` with your OpenShift cluster namespace.
 - Replace `<storageClass>` with the storage provider class name.
 - Enter the number of AppWorks pods that you require in the `replicaCount` parameter.
 - Enter the number of LDAP pods that you require in the `ldap.replicaCount` parameter.

See [“Configuration files for Helm charts” on page 32](#) for more information on the contents in various `values.yaml` files.

4. Change the working directory to the `helm-charts` folder and run the following commands:
 - `helm dep update ./appworks`
 - `helm install appworks ./appworks`
5. Run the following command to obtain the status of each pod:
`kubectl get pods`

After all the pods are running and ready, you can start using the Application.

Chapter 13

Deploying applications to an organization

In some cases, you need to deploy an application package only to a specific organization. This can be done right from the initial start of the container, but also later, when the AppWorks Platform container is already running.

To deploy applications to an organization:

1. Add the application package to the build folder `copy-to-image/appworks/packages/organizationcap`.
2. Specify the application packages to be deployed in:
`platform.organizations.<organization>.package.deployList`

Example:

```
platform:
  organizations:
    acme:
      package:
        deployList: OpenText Party Management, OpenText Salesforce Integrator, OpenText
Document Generation
    contoso:
      package:
        deployList: OpenText Document Generation
```



Note: When the AppWorks Platform container starts, administrator must log into AppWorks Platform and create an organization using the Organization Manager task with the same name as specified in the YAML file. After organization is created, AppWorks Platform deploys the packages to the organization. If the organization exists already, then the administrator must access Organization Manager, select the required organization, and then click `Restart organization creation initialization` to deploy the packages. If it results in an error, then the error details are available in the Organization Manager administration task.

Chapter 14

Configuring OTDS with AppWorks Platform

Installing and configuring OTDS with AppWorks Platform involves the following steps:

- “Configuring AppWorks with OTDS” on page 95
 - “Configuring OTDS resource in AppWorks Platform” on page 95
 - “Consolidating OTDS users with AppWorks Platform” on page 96
- “Installing and configuring OTDS” on page 96
 - “Installing OTDS” on page 96
 - “Configuring OTDS” on page 99

Prerequisites:

Before you begin, ensure to check the following:

- OTDS is deployed.
- All required configurations are done in OTDS . Refer to the OTDS documentation for details or “Installing OTDS” on page 96.
- OTDS resource for AppWorks is configured.

14.1 Configuring AppWorks with OTDS

14.1.1 Configuring OTDS resource in AppWorks Platform

In solution.yml of the application in AppWorks Platform, add OTDS resource details as shown below. The secretKey is Base64 encoded with a string data type.

```
platform:
  otds:
    url: http://:<otds
    resourceId: <OTDS resource id for AppWorks> # Ex: '282a3745-
e633-45c8-87da-33adce2246ae'
    secretKey: 'c2RnYXNnYXNnYXNrYXNrbmdobg==' # base64 input
```

14.1.2 Consolidating OTDS users with AppWorks Platform

To automate user consolidation with AppWorks Platform, add the following details to `solution.yml`:

1. Assign OTDS push role:

```
platform:
  organizations:
    <org>:
      otds:
        pushUser: <sysadmin> # note user has to be present
```

2. Specify the user for consolidation:

```
platform:
  organizations:
    <org>:
      otds:
        userForConsolidation: otadmin@otds.admin # admin user in otds
```

To manually consolidate OTDS users with AppWorks Platform:

1. To push users from OTDS to AppWorks Platform first, assign the OTDS push roles role to the sysadmin (or any other AppWorks Platform user) that you provided in the OTDS bootstrap YAML in the resource.
2. Sign in to OTDS to consolidate users.
Only the first user consolidation is manual. Subsequent user and role consolidation from OTDS to AppWorks Platform is performed automatically. OTDS Bootstrap does not support license configuration.

14.2 Installing and configuring OTDS

14.2.1 Installing OTDS

You can install OTDS in the following ways:

- “Installing OTDS using Docker” on page 96
- “Installing OTDS in Kubernetes through Helm charts” on page 99

14.2.1.1 Installing OTDS using Docker

You can install OTDS through `docker-compose`. The OTDS images can be found in <http://knowledge.opentext.com>.

To install OTDS:

1. Create a text file with the name `docker-compose-otds.yml` and paste the following sample code:

```
version: "3.7"
services:
  opendj:
    image: artifactory.otxlab.net/ot2-paas-dev/otds/otds-opendj:20_dev
```



```

environment:
  OPENDJ_PASS: changeit
  OTDS_PASS: livelink
hostname: opendj
ports:
  - 1389:1389
networks:
  - otds_net
restart: always
volumes:
  - otds_data:/opt/opendj/data
otdsws:
  image: artifactory.otxlab.net/ot2-paas-dev/otds/otds-server:20_dev environment:
    OTDS_DIRECTORY_BOOTSTRAP_CRYPTSECRET: T2NCUFNKQmRGVUpaUkpJSw==
    OTDS_DIRECTORY_BOOTSTRAP_PASSWORD: changeit
    OTDS_DIRECTORY_BOOTSTRAP_SERVERLIST: ldap://opendj:1389
  hostname: otdsws
  ports:
    - 8085:8080
  networks:
    - otds_net
  restart: always
  depends_on:
    - opendj
  volumes:
    - ./config.yml:/opt/config/bootstrap/config.yml
volumes:
  otds_data:
networks:
  otds_net:

```

2. Create a text file with the name `config.yml` and add the required bootstrap configuration:

```

---
!!com.opentext.otds.configurator.Configuration
systemAttributes:
  - name: otdsautoconfig.test.1
    value: test1
trustedSites:
  - https://autoConfiguredSite1/mypath
  - https://autoConfiguredSite2/mypath
#Disable the http.negotiate auth handler
authHandlers:
  - _name: http.negotiate
    _id: http.negotiate
    _description: Handles "Negotiate" authentication with the browser.
    _class: com.opentext.otds.as.drivers.http.NegotiateHttpAuthenticationHandler
    _enabled: false
    _priority: 20
    _scope:
    _properties:
      - _key: com.opentext.otds.as.drivers.http.negotiate.enablemobile
        _value: 'false'
    _authPrincipalAttrNames:
      - oTExternalID4
partitions:
  - name: autoConfigPartition
oauthClients:
  - name: autoConfigClient1
    secret: eecec09ebfb5196bb1dd
    confidential: true
users:
  - name: autoConfigUser1
    userPartitionID: autoConfigPartition
    values:
      - name: userPassword
        values:

```

```

- Password@123

groups:
- name: autoConfigGroup1
  userPartitionID: autoConfigPartition
  users:
  - autoConfigUser1@autoConfigPartition
  groups:
  - autoConfigGroup2@autoConfigPartition
- name: autoConfigGroup2
  userPartitionID: autoConfigPartition

roles:
- name: autoConfigRole1
  userPartitionID: autoConfigPartition
  users:
  - autoConfigUser1@autoConfigPartition
  groups: - autoConfigGroup1@autoConfigPartition
resources:
- resourceName: autoConfigResource1
  description: auto-configured resource 1
  logoutURL: http://mylogouturl
  logoutMethod: GET
  userAttributeMapping:
  - sourceAttr:
    - oTExternalID3
    destAttr: __NAME__
    mappingFormat: "%s"
  groupAttributeMapping:
  - sourceAttr:
    - cn
    destAttr: __NAME__
    mappingFormat: "%s"
accessRoles:
- name: autoConfigAccessRole
  accessRoleMembers:
  userPartitions:
  - userPartition: autoConfigPartition
  users:
  - name: autoConfigUser1
    userPartition: autoConfigPartition
  groups:
  - name: autoConfigGroup1
    userPartition: autoConfigPartition
  roles:
  - name: autoConfigRole1
    userPartition: autoConfigPartition
  resources:
  - resourceName: autoConfigResource1

```

3. Place both the created files in the same directory.

4. To install OTDS, run the following command:

```
docker-compose -f docker-compose-otds.yml up -d
```

5. To see the running containers, run the following command:

```
docker container ls
```

Two containers related to OTDS can be seen running: `dockercompose_otdsws_1` (front end access) and `dockercompose_opendj_1` (database). The `opendj` container takes time to configure.

6. Open `localhost:8085` in a browser. If the URL is not accessible, try restarting the `otdsws` container, for example, run the following command:

```
docker container restart dockercompose_otdsws_1
```

14.2.1.2 Installing OTDS in Kubernetes through Helm charts

You can install OTDS in Kubernetes through Helm charts.

To install OTDS:

1. Download the latest OTDS Helm chart from OpenText Knowledge Center and extract the `otds` folder.
2. Edit the `otds/values.yml` file and ensure that `enableBootstrapConfig` is set to `true`.
3. Edit the `otds/charts/otdsws/config.yml` file, which is mapped to bootstrap configurations.
4. Install the OTDS Helm chart using the following command:

```
helm install otds ./otds/
```

Using the bootstrap YAML, you can create `trustedSites`, `authHandlers`, `partitions`, `oauthClients`, `users`, `groups`, `roles`, `resources`, and `accessRoles`. You can set additional properties for the above objects that are not mentioned in the `config.yml` file. See OTDS Swagger documentation. You can take the JSON representation of the above objects and converted them to YAML.

14.2.2 Configuring OTDS

14.2.2.1 Configuring partitions and users

After installing OTDS, you can configure partitions and users in the Bootstrap YAML using the following sample code:

```
partitions:
- name: appworksPartition1
- name: appworksPartition2
users:
- name: user1 # create more users like this
  userPartitionID: appworksPartition1
  values:
    - name: userPassword
      values:
        - Password@123
- name: user2
  userPartitionID: appworksPartition1
  values:
    - name: userPassword
      values:
        - Password@123
- name: user3
  userPartitionID: appworksPartition2
  values:
    - name: userPassword
      values:
        - Password@123
```

14.2.2.2 Configuring access roles for users

You can configure access roles using the following sample code:

```
accessRoles:
- name: autoConfigAccessRole
  accessRoleMembers:
    userPartitions:
      - userPartition: appworksPartition1
      - userPartition: appworksPartition2
    users:
      - name: otadmin@otds.admin
        userPartition: otds.admin
    groups:
      - name: otadmins@otds.admin
        userPartition: otds.admin
    roles:
      - name: testRole@appworksPartition1
        userPartition: appworksPartition1
  resources:
    - resourceName: appworksResource1
    - resourceName: appworksResource2
  attributeList:
    - name: pushAllGroups # include groups
      values:
        - 'True'
```

14.2.2.3 Configuring resources

A list of resources are provided in the Bootstrap YAML file under `resources`. Multi-tenancy configuration is not supported by OTDS Bootstrap `config.yml`.

There are two ways in which you can configure resources:

- Copying the resourceID from the OTDS user interface
- Generating a random GUID for the resourceID

Copying the resourceID from the OTDS user interface:

When OTDS starts, copy the resourceID from the OTDS user interface to the AppWorks Platform configuration files as shown in the sample code below:

```
resources:
- resourceName: appworksResource
  description: OTDS will generate resource ID and Appworks will activate it
  impersonateAnonymousList:
  pcCreatePermissionAllowed: true
  pcModifyPermissionAllowed: true
  pcDeletePermissionAllowed: false
  allowImpersonation: true
  displayName: appworksResource
  connectorid: rest
  resourceType: rest
  connectorName: REST (Generic)
  connectionParamInfo:
    - name: fBaseURL
      value: http://172.21.240.1:8080/home/system/app/otdspush
    - name: fUsername
      value: sysadmin
    - name: fPassword
      value: cordys123
```

Generating a random GUID for the resourceID:

A random GUID is generated to provide the resourceID. When the resourceID is provided, OTDS also requests a secretKey, which implies that the resource is preconfigured.

A secretKey is a Base64 encoded format of any string that has a length 16 characters. In the listed sample code, a string of 16 characters `sdgasgaskngn` is used with the following Base64 format: `c2RnYXNnYXNnYXNrbmdobg==`

In the OTDS Bootstrap YAML, the secretKey is Base64 encoded with a binary data type.

```
resources:
- resourceName: appworksResourceAuto
  description: Resource ID and secret Key are preconfigured
  impersonateAnonymousList:
  pcCreatePermissionAllowed: true
  pcModifyPermissionAllowed: true
  pcDeletePermissionAllowed: false
  allowImpersonation: true
  displayName: autoIDResource
  resourceID: '845088eb-6d5e-4d29-b022-9ea622c1ad7c' # Generate your own GUID
  secretKey: !!binary c2RnYXNnYXNnYXNrbmdobg== # 16 character length of string in Base64
  connectorid: rest
  resourceType: rest
  connectorName: REST (Generic)
  connectionParamInfo:
  - name: fBaseURL
    value: http://172.21.240.1:8080/home/system/app/otdspush
  - name: fUsername
    value: sysadmin
  - name: fPassword
    value: cordys123
```

Example 14-1: Bootstrap example

```
---
!!com.opentext.ots.configurator.Configuration
systemAttributes:
- name: otdsautoconfig.test.1
  value: test1

trustedSites: &appworksURI
- http://localhost:8080/
- http://172.31.176.1:8080/
- http://appworks:8080/
- http://host.docker.internal:8080/

partitions:
- name: appworksPartion

oauthClients:
- name: appworksOauth
  secret: eecec09ebfb5196bb1dd
  confidential: true
  redirectURLs: *appworksURI

users:
- name: appworksUser
  userPartitionID: appworksPartion
  values:
  - name: userPassword
    values:
    - Opentext1!

groups:
- name: appworksGroup
```

```

userPartitionID: appworksPartition
users:
- appworksUser@appworksPartition
- name: Cordys@Work#Administrator
  userPartitionID: appworksPartition
  users:
  - appworksUser@appworksPartition
  - otadmin@otds.admin

resources:
- resourceName: appworksResourceAuto
  description: ''
  impersonateAnonymousList:
  pcCreatePermissionAllowed: true
  pcModifyPermissionAllowed: true
  pcDeletePermissionAllowed: false
  allowImpersonation: true
  displayName: autoIDResource
  resourceID: '845088eb-6d5e-4d29-b022-9ea622c1ad7c' # Generate your own GUID
  secretKey: !!binary c2RnYXNnYXNrbmdobg== # 16 character length of string in Base64
  connectorid: rest
  resourceType: rest
  connectorName: REST (Generic)
  connectionParamInfo:
  - name: fBaseUrl # OTDS Push URL
    value: http://host.docker.internal:8080/home/system/app/otdspush
  - name: fUsername # User with OTDS Push Role
    value: sysadmin
  - name: fPassword # User password
    value: cordys123
  userAttributeMapping: &userAttributeMapping #reference variable to reuse mapping in
next resource
- sourceAttr:
  - oTExternalID3
  destAttr: __NAME__
  mappingFormat: "%s"
- sourceAttr:
  - displayName
  destAttr: DisplayName
  mappingFormat: "%s"
- sourceAttr:
  - mail
  destAttr: Email
  mappingFormat: "%s"
- sourceAttr:
  - oTTelephoneNumber
  destAttr: Telephone
  mappingFormat: "%s"
- sourceAttr:
  - oTMobile
  destAttr: Mobile
  mappingFormat: "%s"
- sourceAttr:
  - oTFacsimileTelephoneNumber
  destAttr: Fax
  mappingFormat: "%s"
- sourceAttr:
  - oTStreetAddress
  - l
  - st
  - postalCode
  - c
  destAttr: Address
  mappingFormat: "%s%n%s %s %s%n%s"
- sourceAttr:
  - oTCompany
  destAttr: Company
  mappingFormat: "%s"
- sourceAttr:
  - ds-pwp-account-disabled

```

```

    destAttr: AccountDisabled
    mappingFormat: "%s"
- sourceAttr:
  - c
  destAttr: Identity-CountryOrRegion
  mappingFormat: "%s"
- sourceAttr:
  - gender
  destAttr: Identity-Gender
  mappingFormat: "%s"
- sourceAttr:
  - displayName
  destAttr: Identity-DisplayName
  mappingFormat: "%s"
- sourceAttr:
  - oTStreetAddress
  destAttr: Identity-Address
  mappingFormat: "%s"
- sourceAttr:
  - l
  destAttr: Identity-City
  mappingFormat: "%s"
- sourceAttr:
  - mail
  destAttr: Identity-Email
  mappingFormat: "%s"
- sourceAttr:
  - givenName
  destAttr: Identity-FirstName
  mappingFormat: "%s"
- sourceAttr:
  - sn
  destAttr: Identity-LastName
  mappingFormat: "%s"
- sourceAttr:
  - initials
  destAttr: Identity-MiddleNames
  mappingFormat: "%s"
- sourceAttr:
  - oTMobile
  destAttr: Identity-Mobile
  mappingFormat: "%s"
- sourceAttr:
  - postalCode
  destAttr: Identity-PostalCode
  mappingFormat: "%s"
- sourceAttr:
  - st
  destAttr: Identity-StateOrProvince
  mappingFormat: "%s"
- sourceAttr:
  - title
  destAttr: Identity-title
  mappingFormat: "%s"
- sourceAttr:
  - physicalDeliveryOfficeName
  destAttr: Identity-physicalDeliveryOfficeName
  mappingFormat: "%s"
- sourceAttr:
  - oTFacsimileTelephoneNumber
  destAttr: Identity-oTFacsimileTelephoneNumber
  mappingFormat: "%s"
- sourceAttr:
  - notes
  destAttr: Identity-notes
  mappingFormat: "%s"
- sourceAttr:
  - oTCompany
  destAttr: Identity-oTCompany
  mappingFormat: "%s"

```

```

- sourceAttr:
  - oTDepartment
  destAttr: Identity-oTDepartment
  mappingFormat: "%s"
- sourceAttr:
  - birthDate
  destAttr: Identity-Birthday
  mappingFormat: "%s"
- sourceAttr:
  - cn
  destAttr: Identity-UserName
  mappingFormat: "%s"
- sourceAttr:
  - Description
  destAttr: Identity-UserDescription
  mappingFormat: "%s"
- sourceAttr:
  - oTTelephoneNumber
  destAttr: Identity-Phone
  mappingFormat: "%s"
- sourceAttr:
  - displayName
  destAttr: Identity-IdentityDisplayName
  mappingFormat: "%s"
groupAttributeMapping: &groupAttributeMapping
- sourceAttr:
  - cn
  destAttr: __NAME__
  mappingFormat: '%js:function format(name) { return name.replace(/&/g,"-and-");
    }'
- sourceAttr:
  - description
  destAttr: Description
  mappingFormat: "%s"
- sourceAttr:
  - description
  destAttr: Identity-Description
  mappingFormat: "%s"
- sourceAttr:
  - displayName
  destAttr: Identity-DisplayName
  mappingFormat: "%s"

accessRoles:
- name: appworksResourceAuto
  accessRoleMembers:
    userPartitions:
      - userPartition: appworksPartion
    users:
      - name: otadmin
        userPartition: otds.admin
    groups:
      - name: Cordys@Work#Administrator
        userPartition: appworksPartion
  resources:
    - resourceName: appworksResourceAuto
  attributeList:
    - name: pushAllGroups
      values:
        - 'True'

```



Chapter 15

MicroProfile Health

MicroProfile Health of AppWorks Platform enables the platform's services to report and publish their health status. For example, a container is reported as healthy if a service returns its status as UP. Similarly, a container is reported as unhealthy if a service reports it as DOWN. MicroProfile Health reports an individual service status at the endpoint and indicates the overall status as UP if all the services are UP.

15.1 Health check probes

AppWorks Platform's MicroProfile Health uses the Readiness and Liveness probes from Kubernetes to check the health of containers.

See <https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/> for information on using these probes.



Note: Ensure that the names of the service containers provided in the list of components match with the names of corresponding service containers in the default `solution.yml` file or the file you use to provide the configuration details.

15.1.1 Readiness probe

The readiness probe is used by Kubernetes to determine whether a container is ready to accept requests. When a container starts, the readiness check reports with a DOWN status until all the service containers are ready and can handle requests. Only after they are up and ready to handle requests, the readiness probe responds with an UP status. The readiness probe is responsible for determining that your application is ready to accept requests. The traffic is routed to other healthy containers if the container is not ready.

Endpoint URL: `http://localhost:8080/home/system/app/mp/health/ready`

Configuration in Kubernetes

```
readinessProbe:
  httpGet:
    path: /home/system/app/mp/health/ready
    port: 8080
  initialDelaySeconds: 10
  periodSeconds: 10
  timeoutSeconds: 5
```

Configuration in AppWorks Platform

As part of readiness probes, AppWorks Platform performs the following checks:

- Are all the CAPs deployed?

- Are the Post-deployment actions, such as activating the OTDS resource and pushing application roles to OTDS complete?
- Are the services up and running?

The MicroProfile Configuration (MPC) file of AppWorks Platform can be updated to change these default checks. The following MPC code snippet defines the AppWorks Platform's health ready check:

```
platform:
  management:
    health:
      ready:
        components: <List of components>
        availability: true / false
  organizations:
    <org_name>:
      management:
        health:
          ready:
            components: <List of components>
            availability: true / false
```

Property	Description
components	Contains a list of the components used to determine the health. For example: - To check the application package deployment and post-deployment actions, provide the component value as initialization. This is applicable for the System organization only. - To check the services, provide the component value in the following format separated by commas - service.<name of the service>, that is Service.LDAP, Service.Repository, and so on.  Note: Ensure that the names of the service containers provided in the list of components match the names of the corresponding service containers in the <configuration> file. - To check the services in an organization, provide the component value in the following format separated by commas - service.<name of the service> Data type: List Default value: - default services - all - application package deployment status - post-deployment actions
availability	If true - returns the readiness of the components provided in the components property. If false - excludes the components provided. Data type: Boolean Default value: true

Output

```
{ "checks":[ {
  "name":"Initialization",
  "state":"UP"
},
{
  "name":"service.<organization>.<service_name>",
  "state":"UP"
}
}
```

```
},  
"status":"UP"  
}
```

The following table provides a sample readiness probe and its outcome:

Organization	Probe sample	Outcome
system org1	<pre>platform: management: health: ready: components: initialization, service.Business Process Management,service.Auditing availability: true organizations: org1: management: health: ready: components: service.XForms,service.Audit i ng availability: true</pre>	As the availability property is set to true for the System organization, though the Business Process Management service is DOWN, the check on this service is included and the overall status is returned as DOWN. <pre>{ "status": "DOWN", "outcome": "DOWN", "checks": [{ "name": "Initialization", "state": "UP" }, { "name": "service.system.monitor", "state": "UP" }, { "name": "service.system.Collaborativ e Workspace", "state": "UP" }, { "name": "service.system.Event Handling", "state": "UP" }, { "name": "service.system.Platform", "state": "UP" }, { "name": "service.system.LDAP", "state": "UP" }, { "name": "service.system.Security Administration", "state": "UP" }, { "name": "service.system.CAP", "state": "UP" }, { "name": "service.system.Data Transformation", "state": "UP" }, { "name": "service.system.Repository",</pre>

Organization	Probe sample	Outcome
		<pre> "state": "UP" }, { "name": "service.system.XForms", "state": "UP" }, { "name": "service.system.Notification ", "state": "UP" }, { "name": "service.system.Single Sign-On", "state": "UP" }, { "name": "service.system.Logging", "state": "UP" }, { "name": "service.system.CoBOC", "state": "UP" }, { "data": { "error": "Configuration Error: Configuration Error: Exception occurred during BPM service boot com.cordys.bpm.service.syste m.BPMApplicationConnector.op en (BPMApplicationConnector.jav a:65) com.eibus.soap.Processor.ope nApplicationConnector(Proces sor.java:1038) com.eibus.soap.Processor.ope nApplicationConnectors(Proce ssor.java:1028) com.eibus.soap.Processor.ope n(Processor.java:863)" }, "name": "service.system.Business Process Management", "state": "DOWN" }, { "name": "service.org1.XForms", "state": "UP" }, { </pre>

Organization	Probe sample	Outcome
system org1	platform: management: health: ready: components: service.LDAP,service.CAP,service.Business Process Management availability: false organizations: org1: management: health: ready: components: service.XForms,service.Auditing availability: true	As the availability property is set to false for the System organization, though the Business Process Management service is DOWN, the check on this service is excluded and the overall status is returned as UP. { "status": "UP", "outcome": "UP", "checks": [{ "name": "Initialization", "state": "UP" }, { "name": "service.system.monitor", "state": "UP" }, { "name": "service.system.Collaborative Workspace", "state": "UP" }, { "name": "service.system.Event Handling", "state": "UP" }, { "name": "service.system.Platform", "state": "UP" }, { "name": "service.system.LDAP", "state": "UP" }, { "name": "service.system.Security Administration", "state": "UP" }, { "name": "service.system.CAP", "state": "UP" }, { "name": "service.system.Data Transformation", "state": "UP" }, { "name": "service.system.Repository",

Organization	Probe sample	Outcome
		<pre> "state": "UP" }, { "name": "service.system.XForms", "state": "UP" }, { "name": "service.system.Notification ", "state": "UP" }, { "name": "service.system.Single Sign- On", "state": "UP" }, { "name": "service.system.Logging", "state": "UP" }, { "name": "service.system.CoBOC", "state": "UP" }, { "data": { "error": "Configuration Error: Configuration Error: Exception occured during BPM service boot com.cordys.bpm.service.syste m.BPMApplicationConnector.op en(BPMApplicationConnector.j ava:65) com.eibus.soap.Processor.ope nApplicationConnector(Proces sor.java:1038) com.eibus.soap.Processor.ope nApplicationConnectors(Proce ssor.java:1028) com.eibus.soap.Processor.ope n(Processor.java:863)" }, "name": "service.system.Business Process Management", "state": "DOWN" }, { "name": "service.org1.XForms", "state": "UP" }, { "name": "service.org1.Auditing", "state": "UP" } </pre>

Organization	Probe sample	Outcome
		<pre>] } </pre>

15.1.2 Liveness probe

The liveness probe is used by Kubernetes to check whether a specific container is running without issues. This probe is used to detect issues in the container such as internal deadlocks that do not show any progress though the container is running. The liveness probe determines when a container must be restarted.

Endpoint URL: `http://localhost:8080/home/system/app/mp/health/live`

By default, the liveness probe is enabled for the AppWorks Platform configuration in Kubernetes. If needed, an administrator can disable it by providing the following details in `values.yaml`.

Configuration in Kubernetes

```

livenessProbe:
  enabled: false
  initialDelaySeconds: 180
  periodSeconds: 10
  timeoutSeconds: 5

```

Configuration in AppWorks Platform

As part of liveness probes, AppWorks Platform performs the following checks and serves the requests only when all the checks are successful.

- Are all the CAPs deployed ?
- Are the Post-deployment actions, such as activating the OTDS resource and pushing application roles to OTDS complete?

The MicroProfile Configuration (MPC) file of AppWorks Platform can be updated to change these default checks. The following MPC code snippet defines the AppWorks Platform's health liveness check:

```

platform:
  management:
    health:
      live:
        components: <List of components>
        availability: true / false
  organizations:
    <org_name>:
      management:
        health:
          live:
            components: <List of components>
            availability: true / false

```

Property	Description
components	List of components to determine the container's health. • To check the application package deployment and post-deployment actions, provide initialization as the component's value. • To check the application package deployment and post-deployment actions, provide the component value as initialization. This is applicable for the System organization only. Data type: List Default value: • application package deployment status • post-deployment actions
availability	If true - returns the readiness of the components provided in the components property. If false - excludes the components provided. Data type: Boolean Default value: false

Output

```
{
  "checks": [
    {
      "name": "Initialization",
      "state": "UP"
    }
  ],
  "status": "UP"
}
```

Organization name	MPC	Description
system org1	<pre> platform: management: health: live: components: service.XForms, service.Auditing availability: false org1: management: health: live: components: service.XForms,service.Audit ing,service.Email availability: false </pre>	As the availability property is set to false for the System organization, though the Business Process Management service is DOWN, the check on this service is excluded and the overall status is returned as UP. <pre> { "status": "UP", "outcome": "UP", "checks": [{ "name": "Initialization", "state": "UP" }, { "name": "service.system.monitor", "state": "UP" }, { "name": "service.system.Collaborativ e Workspace", "state": "UP" }, { "name": "service.system.Event Handling", "state": "UP" }, { "name": "service.system.Platform", "state": "UP" }, { "name": "service.system.LDAP", "state": "UP" }, { "name": "service.system.Security Administration", "state": "UP" }, { "name": "service.system.CAP", "state": "UP" }, { "name": "service.system.Data Transformation", "state": "UP" }, { "name": "service.system.Repository", </pre>

Organization name	MPC	Description
		<pre> "state": "UP" }, { "name": "service.system.XForms", "state": "UP" }, { "name": "service.system.Notification ", "state": "UP" }, { "name": "service.system.Single Sign- On", "state": "UP" }, { "name": "service.system.Logging", "state": "UP" }, { "name": "service.system.CoBOC", "state": "UP" }, { "data": { "error": "Configuration Error: Configuration Error: Exception occured during BPM service boot com.cordys.bpm.service.syste m.BPMApplicationConnector.op en(BPMApplicationConnector.j ava:65) com.eibus.soap.Processor.ope nApplicationConnector(Proces sor.java:1038) com.eibus.soap.Processor.ope nApplicationConnectors(Proce ssor.java:1028) com.eibus.soap.Processor.ope n(Processor.java:863)" }, "name": "service.system.Business Process Management", "state": "DOWN" }, { "name": "service.org1.XForms", "state": "UP" }, { "name": "service.org1.Auditing", "state": "UP" } </pre>

Organization name	MPC	Description
		} }

Organization name	MPC	Description
system org1	<pre> platform: management: health: live: components: initialization, service.monitor,service.LDAP ,service.CAP,service.Busines s Process Management availability: true organizations: org1: management: health: live: components: service.XForms,service.Audit ing availability: false </pre>	As the availability property is set to true for the System organization, though the Business Process Management service is DOWN, the check on this service is included and the overall status is returned as DOWN. <pre> { "status": "DOWN", "outcome": "DOWN", "checks": [{ "name": "Initialization", "state": "UP" }, { "name": "service.system.monitor", "state": "UP" }, { "name": "service.system.Collaborativ e Workspace", "state": "UP" }, { "name": "service.system.Event Handling", "state": "UP" }, { "name": "service.system.Platform", "state": "UP" }, { "name": "service.system.LDAP", "state": "UP" }, { "name": "service.system.Security Administration", "state": "UP" }, { "name": "service.system.CAP", "state": "UP" }, { "name": "service.system.Data Transformation", "state": "UP" }, { "name": "service.system.Repository", </pre>

Organization name	MPC	Description
		<pre> "state": "UP" }, { "name": "service.system.XForms", "state": "UP" }, { "name": "service.system.Notification ", "state": "UP" }, { "name": "service.system.Single Sign- On", "state": "UP" }, { "name": "service.system.Logging", "state": "UP" }, { "name": "service.system.CoBOC", "state": "UP" }, { "data": { "error": "Configuration Error: Configuration Error: Exception occurred during BPM service boot com.cordys.bpm.service.syste m.BPMApplicationConnector.op en(BPMApplicationConnector.j ava:65) com.eibus.soap.Processor.ope nApplicationConnector(Proces sor.java:1038) com.eibus.soap.Processor.ope nApplicationConnectors(Proce ssor.java:1028) com.eibus.soap.Processor.ope n(Processor.java:863)" }, "name": "service.system.Business Process Management", "state": "DOWN" }, { "name": "service.org1.XForms", "state": "UP" }, { "name": "service.org1.Auditing", "state": "UP" } </pre>

Organization name	MPC	Description
		} }

Chapter 16

Administration tasks

16.1 Displaying hidden tasks in the containerized mode

When using AppWorks Platform in the containerized mode, the following administration tasks that are used to modify several configuration settings are hidden in the Explorer:

- Application Deployer
- Audit Configuration
- Database Metadata Manager
- External Services Configuration
- FTP Configuration manager
- Google Maps Key Manager
- HTTP Connection manager
- iHub Connection Manager
- Log Viewer
- Security Administration
- Synchronize service containers
- Web Gateway Monitor

To display these administration tasks, modify the value of the configuration variable `SHOW_ADMIN_UI` to `true` in the `.env` file. This enables usage of the administration tasks.

16.2 Signing application packages

To sign an application package, a code signing certificate is needed. You can buy a certificate or create one yourself.

To generate a self-signed certificate:

1. Generate the Java Keystore with a private key and certificate:

```
keytool -genkey -alias my-code-signing -keyalg RSA -keysize 2048 -keystore my-code-signing.jks -dname "CN=JOPL Code Signing" -ext "EKU=codeSigning"
```

2. Export the certificate.

```
keytool -export -alias my-code-signing -file my-code-signing.crt -keystore my-code-signing.jks
```

3. Use the AppWorks Platform Package Signer to sign your application packages.
 - a. In Folder/Application Packages, browse and enter the folder location where your application packages are present.
 - b. In Keystore Location, browse and enter the Java Keystore location.
 - c. In Keystore Password, type the password to access the Java Keystore location.
 - d. In Alias, type the alias used to generate and export the certificate.
 - e. Click **Sign** to sign the application packages.
4. Copy the signed application packages in the `.../cap/packages/shared` folder and build the container image.

Allowing unsigned application packages

It is possible to use an unsigned application package in the image, although it is not advised to use it in a production system. To allow unsigned packages, use the following configuration:

```
package:
  security:
    loadUnsignedCaps: true
```

16.3 Running command line tools in the containerized mode

AppWorks Platform has a few command line tools, such as Organizational Removal Tool, BPM Archival tool, and otdsconfigurator. These tools can be run in the containerized mode. Start a shell in the containerized mode, and then start the command line tool similar to starting it on a normal Linux computer. To open a shell in the containerized mode, use the following command:

Starting bash in a container

```
docker exec -it <appworks-container-name> bash
```

For detailed information about the different command line tools, see the *AppWorks Administration guide*.

For detailed information about running Management Console in the containerized mode, see the Accessing Management Console topic in the *AppWorks Administration guide*.

16.4 Changing AppWorks Platform log levels and writing logs to file for backup

Sometimes, it is required to change the Log4J log level, for example for debug level logging. This can be done by overwriting the `/opt/appworks/config/Log4j2Configuration.xml` file. You can either directly edit, copy it into the container, or build a new image with an updated `Log4j2Configuration.xml` file in it. See [“Application-specific configuration to the image” on page 48](#). To write the log files for backup use the configuration given below:

```
<?xml version="1.0" encoding="UTF-8"?>
<Configuration status="WARN"
packages="com.eibus.util.logger.appender,com.cordys.logger.internal,com.cordys.cap.logger
.appender">
    <Properties>
        <Property name="logsFolderLocation">/opt/appworks/node/
logs</Property>
    </Properties>
    <Appenders>
        <Console name="console" target="SYSTEM_OUT"
        immediateFlush="true">
            <XMLLayout locationInfo="true" />
        </Console>
        <ProcessNamedDailyRollingFileAppender name="file"
        fileName="${logsFolderLocation}%N.xml" filePattern="${logsFolderLocation}%N.xml.%d{yyyy-
MM-dd-HH}-%i">
            <XMLLayout locationInfo="true" />
            <Policies>
                <SizeBasedTriggeringPolicy
                size="10 MB" />
            </Policies>
            <DefaultRolloverStrategy max="5">
                <Delete basePath="$
{logsFolderLocation}">
                    <IfFileName
                    glob="*.xml.*" />
                </IfFileName>
            </Delete>
        </DefaultRolloverStrategy>
        </ProcessNamedDailyRollingFileAppender>
    </Appenders>
    <Loggers>
        <Logger name="com.eibus.management.AlertSystem"
        level="info">
            <AppenderRef ref="console" />
            <AppenderRef ref="file" />
        </Logger>
        <Logger name="com.eibus.management.ManagedComponent"
        level="info" />
        <Logger name="com.cordys.cap.applicationconnector.CAPMethod
$CapLogger" additivity="false" level="info">
            <AppenderRef ref="console" />
            <AppenderRef ref="file" />
        </Logger>
        <Logger name="com.cordys.cap.framework.PostDeploymentTask"
        level="info" />
        <Logger
        name="com.cordys.audit.internal.CordysAuditRecorder" level="info" />
        <Root level="error">
            <AppenderRef ref="console" />
            <AppenderRef ref="file" />
        </Root>
    </Loggers>
</Configuration>
```

16.5 Changing AppWorks Platform log levels

By default, the AppWorks Platform does not provide any logging backup solutions in the container environment. If the pods are down or restarted, the AppWorks Platform loses the logs for debugging. So, it is advised that you use some third-party logging backup solutions like GrayLog or Splunk.

16.6 Configuring HTTPS in Kubernetes

You can secure a load balancer by using an ingress controller. See the TLS section in <https://kubernetes.io/docs/concepts/services-networking/ingress>.

16.7 Configuring TomEE

If an administrator wants a specific configuration in TomEE, such as enabling additional logging or enabling CORS, or adding another filter to TomEE, this can be done by editing the `server.xml` or `logging.properties` in the `copy-to-image/tomee/conf` folder.

16.7.1 Securing a TomEE deployed application

Use the semaphore value provided by TomEE to secure the application from Rate limit Attack, for more information see Semaphore Valve at https://tomcat.apache.org/tomcat-8.0-doc/config/valve.html#Semaphore_Valve.

16.8 Enabling CORS

If you developed your own front-end application that uses AppWorks Platform REST services from another server, you must enable CORS. See <https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS>.

To enable CORS, see “CORS” on page 266.

16.9 Enabling Proxy SSL offloading

To enable SSL proxy offloading, you must configure the following settings in the `values.yml` file.

```
security:
  remoteIpFilter:
    enabled:true/false
  overrideProxies: true/false
```

Property name	Description
enabled	Enables SSL offloading when set to true.

Property name	Description
overrideProxies	By default, the application server allows the following IP ranges for internal proxies: • 0/8 • 192.168/16 • 169.254/16 • 127/8 • 0:0:0:0:0:0:0:1 When overrideProxies is set to true, the following IP ranges are allowed: • 100.64.0.0/10 • 10/8 • 192.168/16 • 169.254/16 • 127/8 • 172.16.0.0/12 • 0:0:0:0:0:0:0:1

16.10 Enabling debug tools in a container

It is handy to have some utilities available in the container to check DNS, verify a process, or edit a file. By passing the build argument, `INSTALL_DEBUG_TOOLS=true`, when building with `docker-compose` command, tools such as Vi (Vim), `dnsutils`, `libxi6`, `libxrender1`, `libxtst6`, `net-tools`, `procps`, and `telnet` are installed.

```
docker-compose -f docker-compose.yml -f docker-compose-openldap.yml -f docker-compose-postgres.yml build --build-arg INSTALL_DEBUG_TOOLS=true
```

16.11 Enabling the PSL upgrade feature

AppWorks Platform provides users with a graphical representation of process models to view their progress. Additionally, the process administrators can view details of completed activities and reset current execution points. Application packages having the process models also pack the Published Source Layer(PSL) artifact content. As the PSL content is heavy, when upgrading the applications, there might be a decline in performance. Therefore, to minimize the performance decline, address upgrading the PSL data is now provided as an option rather than a default.

1. Open the `solutions.yml` file.
2. Add the following code snippet to deploy the PSL content:

```
package:
  deployPslArtifact: true //Set the value to true to deploy the PSL content.
```

3. Based on your requirement, enter the property value for the PSL upgrade data deployment behavior as follows:

```
cws:
pslDeploymentBehaviour: immediate
```

The acceptable values are:

Behavior	Description
onDemand	You can access the Deployed Process Models task and view or upgrade the PSL data. This is the default option.
autoUpdate	You can set a property in the CWS properties file to automatically upgrade PSL data along with the application package deployment without any dependency. The application package deployment will continue if the PSL data upgrade fails. The system admin can view the progress of the PSL data upgrade.
immediate	You can set a property in CWS properties file to upgrade the PSL data along with the application package deployment with a dependency. The application package deployment fails if the PSL data upgrade fails.

16.12 Disabling TLS for AppWorks Platform and OpenLDAP communication

SSL is enabled by default but you can disable it for the OpenLDAP container using the following instructions.



Note: This setting will be applicable only for a fresh install and not for upgraded environments.

16.12.1 Enabling OpenLDAP to expose TLS disabled connection

OpenLDAP pod or container always exposes a TLS enabled connection at port 6366. The following setting will allow the container to expose the TLS disabled connection at port 6389.

Docker

Set `LDAP_ALLOW_NON_TLS_CONNECTIONS` in `.env` file as `true`. By default it is read as `false`.

```
LDAP_ALLOW_NON_TLS_CONNECTIONS=true
```

Helm charts

Set `ldap.allowNonTlsConnections` as `true` in the charts. Its default value is `false`.

```
ldap:
  allowNonTlsConnections: true
```

Verification

1. Log into the OpenLDAP pod/container.

```
[kubect1/docker] exec -it <ldap-pod-or-container-name> bash
```

2. Run the search command on TLS enabled port.

```
/opt/cars/bin/ldapsearch -H ldaps://<ldap-host-name>:6366 -w <ldap-admin-
password> -D "cn=Directory Manager,o=openext.net" -b "o=openext.net" | grep
"dn:"
```

3. Run the search command on TLS disabled port.

```
/opt/cars/bin/ldapsearch -H ldap://<ldap-host-name>:6389 -w <ldap-admin-
password> -D "cn=Directory Manager,o=openext.net" -b "o=openext.net" | grep
"dn:"
```

4. Both the above commands should fetch some results.

16.12.2 Switch Appworks to connect to OpenLDAP on TLS disabled port

By default, the communication between AppWorks and OpenLDAP container is TLS enabled. This can be switched to a non-TLS connection. For this, the OpenLDAP container must expose the TLS disabled port (6389) also using `LDAP_ALLOW_NON_TLS_CONNECTIONS` env variable.

Docker

Set `USE_TLS_FOR_LDAP` in `.env` file to false to use Non-TLS connection between AppWorks and OpenLDAP. By default it is read as true.

```
LDAP_ALLOW_NON_TLS_CONNECTIONS=true (This is to expose Non-TLS port in OpenLDAP
container) USE_TLS_FOR_LDAP=false (This is to use Non-TLS communication between
AppWorks and OpenLDAP)
```

Helm charts

Set `appworks.useTLSForLDAP` as false in the charts. Its default value is true. `ldap.allowNonTlsConnections` must be true for OpenLDAP to expose TLS disabled port.

```
appworks:
  useTLSForLDAP: false
ldap:
  allowNonTlsConnections: true
```

16.12.3 Enable TLS disabled connection between multiple OpenLDAP pods or containers

Docker

Set `LDAP_USE_TLS_FOR_SYNC_REPLICATION` in `.env` file as `false`. By default it is read as `true`. `LDAP_ALLOW_NON_TLS_CONNECTIONS` must be `true` for OpenLDAP to expose TLS disabled port.

```
LDAP_ALLOW_NON_TLS_CONNECTIONS=true
LDAP_USE_TLS_FOR_SYNC_REPLICATION=false
```

Helm charts

Set `ldap.useTlsForSyncReplication` as `false` in the charts. Its default value is `true`. `ldap.allowNonTlsConnections` must be `true` for OpenLDAP to expose TLS disabled port.

```
ldap:
  allowNonTlsConnections: true
  useTlsForSyncReplication: false
```

16.13 Enabling detailed error messages

When any SOAP request results in an error, the response contains details of the error including stack traces which can create security concerns. AppWorks Platform removes stack traces from the error messages by default in SOAP responses. So, AppWorks Platform completely removes the `cordys:FaultRelatedException` tag from the SOAP fault in the SOAP response.

You can disable this default behavior by adding `-Dgateway.fault.stripstacktrace=false` property as follows:

```
extraEnv:
- name: CATALINA_OPTS
  value: -XX:MaxRAMPercentage=75.0 -XX:+UseG1GC -XX:ErrorFile=/opt/appworks/node/
hs_err_pid%p.log -Dgateway.fault.stripstacktrace=false
```

This provides the complete stack trace in the error.

Chapter 17

Migrating from classic to containerized deployment

You can migrate AppWorks Platform, which is installed using the classic installation procedure to the containerized deployment model. After AppWorks Platform is migrated to the containerized deployment model, for any further upgrade, you can use container images.

17.1 Migration overview

This topic covers migration of AppWorks Platform installed using the classic deployment to the containerized deployment. It does not cover database or LDAP migration to the containerized deployment.



Note: To migrate data from the classic deployment to the containerized deployment or cloud, check with the database vendor for the required tools.

AppWorks Platform in the containerized mode does not support applications using data transformation, Master Data Management (MDM), and Business Activity monitoring (BAM).

AppWorks Platform in the containerized mode runs all service containers inside TomEE. All service containers run in one JVM. This may impact transactions and other functionality. Therefore, you must validate existing applications or solutions thoroughly.

The document store connector in the Repository service group is not available after migration to the containerized mode.

17.2 Prerequisites to migrate to the containerized deployment

Ensure that the following prerequisites are met before migrating the classic deployment to the containerized deployment.

1. Upgrade the AppWorks Platform cluster to AppWorks Platform 23.4. See the *AppWorks Migration Guide*.
2. Move all the service containers to TomEE (application server). Assign the Application Server OS process to all the service containers except Monitor service.
3. Undeploy the following application packages:
 - Cordys Data Transformation

- Cordys Data Transformation Modeler
 - Cordys CoBOC
 - Cordys BAM MDM Model
 - Cordys BAM Runtime
 - Cordys MDM Runtime
 - Cordys MSMQTransport
 - Cordys Orchestrator 4.2
 - Cordys BAM Modeler
 - Cordys Business Activity Monitoring
 - Cordys MDM Modeler
 - Cordys MDM
 - Cordys Object Template Modeler
4. Validate whether your applications are working as expected with these changes.

 **Note:** If your applications have dependencies with any of the listed application packages, migration is not supported for your environment.
 5. Back up the configuration of the Document Store inside the Repository service container because this service container is removed after the migration to the containerized deployment.
 6. After upgrading to AppWorks Platform 23.4, and before switching to the containerized mode, if you made any changes to the CWS projects, ensure that you take the following precautions for SVN and CWS as described in the *AppWorks Migration Guide*.
 - a. Ensure that workspaces have no local changes and are in sync with the file system and Subversion (SVN).
 - b. Validate the projects.
 - c. Run the CWSWorkspaceUpgradeAnalyzer tool to identify workspace documents with long path issues.
 7. If you have any exported content present in your installation, back up the exported data content to migrate it to the Docker instance. Take a backup of:

```
${CORDYS_HOME}/webroot/wcpdev
```

17.3 Migration tasks

Perform the following tasks to migrate to the container-based deployment:

- “Preparing the Docker image” on page 131
- “Deploying AppWorks Platform with applications” on page 132

17.3.1 Preparing the Docker image

To prepare the Docker image:

1. Download the AppWorks Platform containerization files from OpenText My Support. Extract the files in the archive to a new folder, for example, awp.
2. Add application packages:

- a. Collect all the application packages that are deployed into AppWorks Platform.

- b. Copy the shared packages to:

```
<archive-extracted-folder>/copy-to-image/appworks/packages/shared
```

- c. Copy the organization packages to:

```
<archive-extracted-folder>/copy-to-image/appworks/packages/organization
```

Example: The copy-to-image folder hierarchy might be as follows:

```
+-- copy-to-image
|  +- appworks
|  |  +- packages
|  |  |  +- organization
|  |  |  |  --- My Company CBBISEPProject 2.0.0.cap
|  |  |  |  --- Opentext Employee Management 1.0.2.cap
|  |  |  |  --- Test Connectors.cap
|  |  +- tomee
|  |  |  +- conf
|  |  |  |  --- logging.properties
|  |  |  |  --- server.xml
```

To deploy application packages in a specific organization, use the `package.deployList` configuration property to specify which application packages must be deployed. Specify this configuration for the organization in which these application packages must be deployed. Following is the sample code to deploy the three application packages in the previous example in the acme organization.

```
platform:
  organizations:
    acme:
      package:
        deployList: My Company CBBISEPProject 2.0.0, Opentext Employee
Management 1.0.2, Test Connectors
```



Note: Ensure that all the deployed applications and the AppWorks Platform packages are of the same version and build with those provided to build the Docker image. Application packages can be

built on any version of AppWorks Platform, but the package of the version, which is deployed must be available in the image.

3. Build the Docker image. See [“Building the Application image” on page 57](#).

17.3.2 Deploying AppWorks Platform with applications

17.3.2.1 Deploying AppWorks Platform with applications on Docker

To deploy AppWorks Platform with applications on the Docker environment:

1. Provide LDAP and database related details in the `.env` file.
2. Provide license details in the `.env` file.
3. Provide the list of nodes in the cluster in the variable `ORIGINAL_NODES_LIST` with nodes separated by a colon (:), starting with the primary node.



Note: The node, which has `IS_DISTRIBUTED_INSTALLATION=false` in `_uninst/installvariables.properties`, is the primary node.

See [“Deploying Application images on Docker” on page 61](#).

17.3.2.2 Deploying AppWorks Platform with applications on Kubernetes

To deploy AppWorks Platform with applications on Kubernetes using Helm:

1. Provide LDAP and database related details in the `values.yml` file.
2. Provide license details in the `values.yml` file.
3. Provide the list of nodes in the cluster in the variable `originalNodesListwith` in the `values.yml` file with nodes separated by a colon (:), starting with the primary node.

See [“Deploying Application images on Kubernetes” on page 69](#).

17.4 Post migration tasks

After the migration, perform the following task:

- The Repository service container has been removed. Therefore, recreate the Document Store service container in the system organization either from YAML or from the user interface.
- For your CWS projects:
 - When you are using SVN, perform a new SVN checkout of your workspaces in the Docker container.

- When you are not using SVN, copy the files from the synchronization folder of the classic environment to the correct location in the synchronization folder in the Docker container, and sync them into CWS.

Chapter 18

Updating the Application container images

The Application container images can be updated from one version to another in the following scenarios:

- The application or its configuration has changed but the AppWorks Platform version does not change.
- The AppWorks Platform version has changed due to a hotfix or a newer release.

18.1 Updating an application or its configuration to a new version

As you start working with the application, based on your requirements, you might want to update or enhance the application. When running applications with containers, if you want to make an update, you must build a new container image with the relevant changes and run it. Similar to adding a new application package, AppWork Platform's upgrade logic handles the process of adding the upgraded package.

When updating your application, you can reuse the folder structure that is used to build the previous version of the container.

18.2 Updating AppWorks Platform to a new version

OpenText provides new releases or fix-packs of AppWorks Platform regularly. The new AppWorks Platform version image will be available in OpenText registry and the corresponding helm charts and docker related files are available on My Support as AppWorks Platform containerization.tar.gz archives. You can use these images and archives to build new versions of your application-specific containers.

To update AppWorks Platform to a new version:

1. Build a new application-specific container image with your application packages and configuration on the new AppWorks Platform version.
2. Update the deployment environment on Docker or Kubernetes.

Required tasks

- Download the latest AppWorks Platform container archive from My Support, for example, appworks-platform-23.4-containerization.tar.gz, and extract it to a new folder.



Note: The name of the folder must match the name of the folder in the existing version. For example, if the docker-compose files of AppWorks

Platform 21.3 are stored in `/opt/Appworks213/docker`, ensure that the AppWorks Platform 23.4 container files are extracted to `/opt/docker`. The name of this folder will be used as a prefix for the appworks, ldap, and database container names such as `docker_appworks_1`, `docker_database_1` and `docker_ldap_1`. See [“Container archive contents” on page 15](#) for more information on the contents of the archive.

- Before you begin the update process, ensure that the payload validation is switched off. If payload validation is enabled either at the gateway level or a specific service level, do the following:
 - If payload validation is enabled at the WebGateway level, you must disable it as shown below:

```
platform:
  gateway:
    requestValidation: 'false'
    responseValidation: 'false'
```

- If payload validation is enabled for one or more specific services, then you must disable it by setting the validation property for the specific service to false as shown below:

```
platform:
  services:
    instances: ''
    <service>:
      kind: ${name}
      validation: 'false'
```

Recommended tasks

Before you update AppWorks Platform to a new version, OpenText recommends you to perform the following tasks.

- Validate your design-time projects in CWS.
Ensure that all the CWS projects in the workspaces are validated using the Validate option in the project's context menu. Fix the errors, if any, before the upgrade.
- Commit local changes to SVN. If you use SVN for your development environment, ensure that:
 - there are no local changes present in the workspaces with team development configured to SVN as the SVN locks might hinder the upgrade process. Commit, revert, or delete the local changes.



Note: Release the SVN locks, if available, even if there are no local changes.

- all the workspaces are synchronized to the file system and SVN to avoid any merge conflicts.
 - the update permissions on the SVN repository are removed to prevent check-ins until the upgrade is complete.

- Back up the AppWorks Platform and OpenText CARS databases and volumes.

It is important to take a backup of your data. This allows you to refer a previous version if the upgrade fails. You must back up the following artifacts because the AppWorks Platform upgrade modifies or replaces them and you cannot roll back these artifacts automatically if the upgrade fails.

Artifact	Description
OpenText CARS database	See the Backing up OpenText CARS database topic in the <i>OpenText CARS Installation and Administration Guide</i> on OpenText My Support. Using OpenLDAP: If an OpenLDAP image is used, then you must backup the persistent volumes used by it. An OpenLDAP image uses one persistent volume and can be backed up using the following command: <pre>docker cp <openldap-container-name>:<path-to-volume-on-container> <path-to-backup-folder></pre> For example, <code>docker cp docker_ldap_1:/opt/cars/var /ldapbackup/cars</code>
AppWorks Platform database	Back up the AppWorks Platform database on the application server using database tools. Contact your database administrator.

Artifact	Description
File system /Persistent Volumes	AppWorks Platform in container mode stores files in persistent volumes. AppWorks Platform uses three volumes, one node-specific volume and two shared or cluster volumes To back up the files in these volumes, use the following commands: Using Docker <pre>docker cp <container-name>:<path-to-volume-on-container> <path-to-backup-folder></pre> For Example: – docker cp docker_appworks_1:/opt/appworks/cluster /appworks-backup/cluster – docker cp docker_appworks_1:/opt/appworks/webroot/organization /appworks-backup/organization – docker cp docker_appworks_1:/opt/appworks/node /appworks-backup/node Using Kubernetes <pre>kubectl cp <namespace>/<pod-name>:<path-to-volume-on-container> <path-to-backup-folder></pre> For example: – kubectl cp mycompany-apps-1/appworks-0:/opt/appworks/cluster /appworks-backup/cluster – kubectl cp mycompany-apps-1/appworks-0:/opt/appworks/webroot/organization /appworks-backup/organization Also, you must back up the node-specific volume of each node in the cluster as follows: – kubectl cp mycompany-apps-1/appworks-0:/opt/appworks/node /appworks-backup/node0 – kubectl cp mycompany-apps-1/appworks-1:/opt/appworks/node /appworks-backup/node1

18.2.1 Building a new image with application packages

To build a new Application image with updated application packages:

1. Create the new application packages or update the application packages as required.
2. Add the new or updated application packages to the build folder:
 - If the application packages must be deployed to the shared space, add the new or updated application packages to the `.../copy-to-image/appworks/packages` folder.
 - To add signing certificates, place them in the `copy-to-image/appworks/trustCertificates` folder. On container startup, these certificates will be automatically added to the trust store.
 - If your applications must be deployed to a specific organization:
 - Add the new or updated application packages to the build folder `.../copy-to-image/appworks/packages/organization` to deploy or upgrade the packages to the organization.
 - Specify the application package names to be deployed in the configuration property: `platform.organizations.<organization>.package.deployList` See [“Application-specific configuration to the image” on page 48](#) and [“Deploying applications to an organization” on page 93](#) for more information.
3. Update the application-specific configuration files and add it to the image. See the [“Copying the solution.yml file in a container through the copy-to-image folder” on page 50](#) for more information.
4. Merge the configuration data from the existing image configuration files into the new container build configuration files.

Important

Do not copy or replace the files. Merge them as the new configuration files might contain updates.

5. Open the `docker-compose.yml` and type a name for the AppWorks image in the specified format
6. Rebuild the Application image with the new versions of application packages and configurations using the command:

```
docker-compose build
```

Optional: To build a new OpenLDAP image with updated application packages:



Note: Only required if you are upgrading from 21.2 or earlier versions.

1. Take a backup of the LDAP contents and certificates from the existing LDAP container/pod.

- a. Create folders to take backup in the extracted zip.
 - `mkdir -p /opt/AppWorks222/docker/cars/ldif`
 - `mkdir -p /opt/AppWorks222/docker/cars/certs`
 - b. Take the LDAP data backup as backup.ldif as shown in the samples below:
 - Sample command for version AppWorks Platform 20.2 and 20.3:


```
[docker/kubect1] exec <existing openldap-container/pod-name> slapd -T cat -n 2 -l backup.ldif
```
 - Sample command for version AppWorks Platform 20.4:


```
[docker/kubect1] exec <existing openldap-container/pod name> -- /opt/cars/sbin/slapd -T cat -n 2 -F /opt/cars/var/slapd.d -l backup.ldif
```
 - Sample command for version AppWorks Platform 21.1:


```
[docker/kubect1] exec <existing openldap-container/pod name> -- /opt/cars/sbin/slapd -T cat -n 2 -F /opt/cars/var/slapd.d -l backup.ldif
```
 - Sample command for version AppWorks Platform 21.2:


```
[docker/kubect1] exec <existing openldap-container/pod name> -- /opt/cars/sbin/slapd -T cat -n 1 -F /opt/cars/var/slapd.d -l backup.ldif
```
 - c. Run the following command to add the backup.ldif file to the <path-to-zip-extracted>/cars/backup/ folder.


```
[docker/kubect1] cp <existing openldap-container/pod-name>:backup.ldif /opt/AppWorks/docker/cars/ldif
```
 - d. Copy the certificates from the existing LDAP container to <path-to-zip-extracted>/cars/certs/ folder as shown in the samples below.
 - Sample command for version AppWorks Platform 20.2 and 20.3:


```
[docker/kubect1] cp <existing openldap-container/pod-name>:/container/service/slapd/assets/certs /opt/AppWorks/docker/cars/certs
[docker/kubect1] cp <existing openldap-container/pod-name>:/container/service/:ssl- tools/assets/default-ca/default-ca.pem /opt/AppWorks/docker/cars/certs/ca.crt
```
 - Sample command for version AppWorks Platform 20.4 and later versions:


```
[docker/kubect1] cp <existing openldap-container/pod-name>:/opt/cars/var/certs/ca.crt /opt/AppWorks/docker/cars/certs/ca.crt
[docker/kubect1] cp <existing openldap-container/pod-name>:/opt/cars/var/certs/ldap.crt /opt/Appworks213/docker/cars/certs/ldap.crt
[docker/kubect1] cp <existing openldap-container/pod-name>:/opt/cars/var/certs/ldap.key /opt/Appworks213/docker/cars/certs/ldap.key
```
2. Create a Dockerfile in /opt/Appworks222/docker/cars to create a new OpenLDAP image on top of the 23.4 release image.


```
FROM registry.opentext.com/otawp-openldap:22.2
COPY --chown=openldap:openldap ldif /opt/cars/ldif
COPY --chown=openldap:openldap certs /opt/cars/certs
```
 3. Build the OpenLDAP image.

```
cd /opt/Appworks222/docker/cars
docker build -t <registry>/<ldap-repository>:<ldap-tag>
```

The latest Application and OpenLDAP images are built. If Kubernetes is used, add the resulting container image to a container artifactory.

18.3 Updating AppWorks Platform on Docker

You can update an existing AppWorks Platform instance to a new version on Docker.

To update the existing Application image on Docker:

1. Open the `docker-compose.yml` file and add the Application image name in the specified format.
2. Open the `docker-compose-openldap.yml` file and add the OpenLDAP image name in the specified format.
3. Run the upgrade command as shown below:

```
cd opt/Appworks222/docker

## if using docker-compose-openldap.yml
docker-compose --verbose -f docker-compose.yml -f docker-compose-remote-
debugging.yml -f docker-compose-jmx-monitoring.yml -f docker-compose-postgres.yml -
f docker-compose-openldap.yml up -d --no-deps --build appworks ldap

## if using docker-compose-openldap-ha.yml (Only from 21.3 and previous version
should be installed with docker-compose-openldap-ha.yml)
docker-compose --verbose -f docker-compose.yml -f docker-compose-remote-
debugging.yml -f docker-compose-jmx-monitoring.yml -f docker-compose-postgres.yml -
f docker-compose-openldap-ha.yml up -d --no-deps --build appworks ldap ldap2
```



Note: The `-f docker-compose-postgres.yml`, `-f docker-compose-remote-debugging.yml`, and `-f docker-compose-jmx-monitoring.yml` parameters are not mandatory and can be removed from the above command. These are required only in the following cases:

- If the configuration to enable remote debugging on TomEE through JPDA_OPTS is required, use `docker-compose-remote-debugging.yml`.
- If the configuration to enable JMX monitoring on TomEE through CATALINA_OPTS is required, use `docker-compose-jmx-monitoring.yml`.
- If you are using a containerized database and want to upgrade it, use `docker-compose-postgres.yml`. For other databases, you can use the applicable `docker-compose-oracle.yml` or `docker-compose-msmsql.yml` files.

The AppWorks Platform instance is updated to AppWorks Platform 23.4.

18.4 Updating the Application image on Kubernetes using Helm charts

Based on the existing version of the AppWorks Platform container, you can do one of the following:

- Update AppWorks Platform 20.2, 20.3, and 21.3 and later versions to AppWorks Platform 23.4 .
- Update AppWorks Platform 20.4, 21.1 and 21.2 to AppWorks Platform 23.4.

18.4.1 Updating AppWorks Platform 20.2, 20.3, and 21.3 and later versions to 23.4 using Helm charts

1. Navigate to the helm-charts folder and open the values.yaml file.
2. Update the AppWorks Platform image, storage class name, and license details in /opt/AppWorks222/docker/helm-charts/appworks/values.yaml as shown in the sample below:

```
image:
  registry: artifactory.otxlab.net
  repository: appworks-central-qa/appworks/platform
  tag: 23.4
```

3. **Optional** Required only if the existing instance is 20.2 or 20.3. Update the new OpenLDAP image details in /opt/AppWorks222/docker/helm-charts/appworks/values.yaml as shown in the sample below:

```
ldap:
  ...
  image:
    registry: artifactory.otxlab.net
    repository: appworks-central-qa/appworks/openldap
    tag: 23.4
```

4. Update the /opt/AppWorks222/docker/helm-charts/appworks/values.yaml with the following:
 - Set ldap.enabled=true
 - Set 'ldap.dnsConfig.searches' as 'ldap.\$clusterId.svc.cluster.local'
 - Ensure that the replica count for LDAP is 1. If necessary, it can be upscaled based on the requirement.
 - Set ldap.podManagementPolicy= "OrderedReady"
5. If upgrading from 21.3 or later versions, ensure that the value for the following two properties are same as it was when installing AppWorks Platform. If you need to change these properties in the upgraded instance, you must follow the instructions provided in the [“Optional: To build a new OpenLDAP image with updated application packages:”](#) on page 139 section above, and then perform the steps provided in the [“Updating AppWorks Platform 20.4, 21.1, and 21.2 to AppWorks Platform 23.4 using Helm-charts”](#) on page 144.

- `ldap.persistence.ldapDataPVCSize`- This property specifies the size of `appworks-ldap-data-directory` PVC. You can use `kubectl get pvc` to get the size of PVC. The default values for the PVC size in corresponding releases are:
 - 21.3 and 21.4 - 256Mi
 - 22.1 and later versions - 5Gi
 - `ldap.persistence.ldapSharedPVCSize` - This property specifies the size of `appworks-ldap-shared` PVC. You can use `kubectl get pvc` to get the size of PVC. The default values for the PVC size in corresponding releases are:
 - 21.3 and 21.4 - 10Gi
 - 22.1 and later versions - 256Mi
6. Downscale Appworks to 0 pods. Wait for all the AppWorks pods to terminate.


```
kubectl scale statefulsets appworks --replicas=0
```
 7. Switch back to the appworks namespace if you are using multiple namespaces.


```
kubectl config set-context --current --namespace=<appworks-namespace>
```
 8. Execute the following command to obtain <NAME>:


```
helm list
```
 9. Run the following commands:
 - a. Navigate to the helm charts folder: `cd /opt/AppWorks/docker/helm-charts`
 - b. Update the charts: `helm dep update ./appworks`
 - c. Delete the LDAP service. Only required if the current AppWorks Platform version is 20.2 or 20.3: `kubectl delete svc ldap`
 - d. Upgrade the Appworks instance: `helm upgrade <NAME> ./appworks`
For example: `$ helm upgrade appworks ./appworks`

The AppWorks Platform instance is updated to AppWorks Platform 23.4.



Note: You can setup HA for LDAP only when all the pods are ready and live. To setup HA, follow the steps listed in “[Appendix E – Setting up High Availability with LDAP pods](#)” on page 285.

18.4.2 Updating AppWorks Platform 20.4, 21.1, and 21.2 to AppWorks Platform 23.4 using Helm-charts

1. Navigate to the helm-charts folder and open the values.yaml file.
2. Update the AppWorks Platform image, storage class name, and license details in /opt/AppWorks222/docker/helm-charts/appworks/values.yaml as shown in the sample below:

```
image:
  registry: artifactory.otxlab.net
  repository: appworks-central-qa/appworks/platform
  tag: 23.4
```

3. Update the new ldap image details in /opt/AppWorks222/docker/helm-charts/appworks/values.yaml as shown in the sample below:

```
ldap:
  ...
  image:
    registry: artifactory.otxlab.net
    repository: appworks-central-qa/appworks/openldap
    tag: 23.4
```

4. Update the /opt/AppWorks222/docker/helm-charts/appworks/values.yaml with the following:
 - Set ldap.enabled=true
 - Set 'ldap.dnsConfig.searches' as 'ldap.\$clusterId.svc.cluster.local'
 - Ensure that the replica count for LDAP is 1. If necessary, it can be upscaled based on the requirement.
5. Switch back to the appworks namespace if you are using multiple namespaces.
 - `kubectl config set-context --current --namespace=<appworks-namespace>`
6. Delete the existing pods for LDAP as follows:
 - `kubectl get pods -l app=ldap -o jsonpath="{.items[0].metadata.name}"`
 - `kubectl delete pod <pod-name>`
7. Delete the existing replica set, if it exists for LDAP as follows:
 - `kubectl get replicaset -l app=ldap -o jsonpath="{.items[0].metadata.name}"`
 - `kubectl delete replicaset <replicaset-name>`
8. Delete the existing deployment, if it exists for LDAP as follows:
 - `kubectl get deployment -l app=ldap -o jsonpath="{.items[0].metadata.name}"`
 - `kubectl delete deployment <deployment-name>`

9. Delete the existing stateful-set, if it exists for LDAP as follows:

- `kubectl get statefulset -l app=ldap -o jsonpath="{.items[0].metadata.name}"`
- `kubectl delete statefulset <statefulset-name>`

10. Delete the existing service for LDAP as follows:

- `kubectl get service -l app=ldap -o jsonpath="{.items[0].metadata.name}"`
- `kubectl delete service <service -name>`

11. Delete the existing PVCs for LDAP as follows:

- `kubectl get pvc -l app=ldap -o jsonpath="{.items[0].metadata.name}"`
- `kubectl delete persistentvolumeclaim <pvc-name>`
- `appworks-ldap-shared kubectl delete persistentvolumeclaim appworks-ldap-shared`

12. Downscale Appworks to 0 pods. Wait for all the Appworks pods to terminate.

- `kubectl scale statefulsets appworks --replicas=0`

13. Execute the following command to obtain <RELEASE_NAME>:

- `helm list`

For example:

```
$ helm list
NAME      NAMESPACE  REVISION  UPDATED   STATUS  CHART          APP VERSION
appworks  appworks    1         ....    deployed  appworks-21.2+25  21.2+25
```

14. Run the following commands:

- Navigate to the helm charts folder: `cd /opt/AppWorks/docker/helm-charts`
- Update the helm charts: `helm dep update ./appworks`
- Upgrade the Appworks instance: `helm upgrade <NAME> ./appworks`
For example: `$ helm upgrade appworks ./appworks`

The AppWorks Platform instance is updated to AppWorks Platform 23.4.



Note: You can setup HA for LDAP only when all the pods are ready and live. To setup HA, follow the steps listed in “[Appendix E – Setting up High Availability with LDAP pods](#)” on page 285.

Chapter 19

Appendix A – MicroProfile Configuration

As a good practice, when developing an application, you must provide options to change an application's runtime behavior without having to change the code, for example, changing the language of the application, updating business rules, and modifying the user name and password of a database connection. MicroProfile Configuration (MPC) enables you to externalize these configurations, allowing you to make the changes to an application's runtime behavior with ease.

AppWorks Platform supports externalized configuration through MPC, that is, configuration can be specified in any MicroProfile compatible configuration source. All the configuration settings in AppWorks Platform are made available through external configuration.

19.1 MicroProfile properties configuration

The properties extracted in the MPC documentation `settings-all.yml` can be configured in the container in different ways as follows:

- YAML files
- `wcp.properties` files
- Environment variables
- Secrets in a vault (if configured)

Example: To configure the AppWorks Gateway password, the key is `platform.appworksgateway.password`.

19.1.1 YAML files

Each dot splits the key into levels of hierarchy in the YAML file. The keys are case-sensitive.

Following is the sample code.

```
platform:
  appworksgateway:
    password: changeit
```

19.1.2 wcp.properties

Following is the sample code.

```
platform.appworksgateway.password=changeit
```

19.1.3 Environment variables

Convert the keys to uppercase characters and replace all the non-alphanumeric characters with an underscore '_'.

Following is the sample code.

```
# inside helm chart values yaml set the value under extraEnv.  
extraEnv:  
  - name: CATALINA_OPTS  
    value: -XX:MaxRAMPercentage=75.0  
  - name: PLATFORM_APPWORKSGATEWAY_PASSWORD  
    value: changeit
```

19.1.4 Secrets in a vault

Convert the keys to uppercase characters and replace all the non-alphanumeric characters with an underscore '_'.

PLATFORM_APPWORKSGATEWAY_PASSWORD is the key.



Note: There are many configuration properties in the Platform, such as database passwords, OTDS resource secret keys, and passwords in the connector configuration. OpenText recommends storing them in a vault when a vault is configured.

```
useExternalSecrets: false  
createSecretStore: true  
useSecretStore: true  
externalSecretSpec:  
  secretStoreRef:  
    name: <secret-store-name>  
    kind: SecretStore  
  target:  
    name: appworks  
  dataFrom:  
    - extract:  
        key: <secret-manager-name>  
    data:  
      - secretKey: <secret-key-name>  
        remoteRef:  
          key: <provider-secret-key-name>  
  secretStoreSpec:  
    provider: <provider-details>
```

The following table describes the configuration properties of External Secrets:

Property Name	Description
useExternalSecrets	When <code>true</code>, secrets are taken from external providers. When <code>false</code>, the following values must be provided: • <code>appworks.password</code> • <code>appworks.keystorePassword</code> • <code>ldap.adminPassword</code> • <code>ldap.configPassword</code> • <code>database.password</code> • <code>database.adminPassword</code> Data Type: Boolean Default value: <code>false</code>
createSecretStore	When <code>true</code>, a <code>SecretStore</code> or <code>ClusterSecretStore</code> is created. When <code>false</code>, you need to point to existing <code>SecretStore/ClusterSecretStore</code>. Data Type: Boolean Default value: <code>true</code>
useSecretStore	When <code>true</code>, a <code>SecretStore</code> is created. When <code>false</code>, a <code>ClusterSecretStore</code> is created. Data Type: Boolean Default value: <code>true</code>
secretStoreRef	Configuration of the <code>SecretStore</code> or the <code>ClusterSecretStore</code> .
name	Name of the <code>SecretStore</code> or the <code>ClusterSecretStore</code>. Data Type: String Default value: None
kind	Enter <code>SecretStore</code> or <code>ClusterSecretStore</code>. Data Type: String Default value: <code>SecretStore</code>
target	Configuration of the target secret.

Property Name	Description
name	Name of the Secret. Data Type: String Default value: appworks
dataFrom	Configuration to fetch all properties from the Provider key.
key	Path to the provider key.
data	Configuration to fetch key from the provider data.
secretKey	Name of the Secret.
remoteRef	Configuration of the provider key.
key	Name of the provider key.
secretStoreSpec	Configuration of the external provider.
provider	Based on the external provider authentication, you must enter the provider configuration. Refer https://external-secrets.io/v0.5.7/ for the external provider authentication configuration information.

19.2 MicroProfile properties

The following tables list the MicroProfile configuration properties of various components:

19.2.1 Database connection

```
platform:
  organizations:
    <Organization Name>:
      database:
        dataSources:
          defaultDataSource:
            connection: <my_conn>
          xdsDataSource:
            connection: <my_conn>
        connections:
          <my_conn>:
            kind: PostgreSQL
            jdbcDriver: org.postgresql.Driver
            jdbcXaDriver: org.postgresql.xa.PGXADatasource
            connectionString: jdbc:postgresql://database:5432/appworks
            database: appworks
            userName: postgres
            password: mypassword
            useSsl: 'false'
```

Property name	Description
kind	Database vendor name. Possible values are PostgreSQL, Oracle, AzureSqlDb, and MsSqlServer. Data type: String Default value: None
jdbcDriver	Fully qualified JDBC driver class. Data type: String Default value: None
jdbcXaDriver	Fully qualified JDBC XA driver class. Data type: String Default value: None
connectionString	JDBC connection string as required by the database vendor/JDBC driver. Data type: String Default value: None
database	Database name. Data type: String Default value: None
userName	Database user name. Data type: String Default value: None
password	Database password. Data type: String Default value: None
useSsl	Whether to use SSL or non-SSL communication while using this Database connection. Data type: Boolean Default value: false

19.2.2 Datasource properties

```

datasources:
  <dataSource name>:
    connection: system
    connectionPoolRefreshInterval: PT30S
    cursorCacheRefreshInterval: PT30S
    cursorCacheSize: '1000'
    defaultCursorSize: '2147483647'
    maximumReadConnections: '5'
    maximumWriteConnections: '5'
    minimumReadConnections: '1'
    minimumWriteConnections: '1'
    optimizeQueryForCursor: 'true'
    queryCacheRefreshInterval: PT30S
    queryCacheSize: '1000'
    readEmptyStringAsNull: 'false'
    reconnectionAttempts: '1'
    reconnectionInterval: PT5S
    requestThreshold: '1000'
    useCaseConversion: 'true'
    writeEmptyStringAsNull: 'false'
    xmlEncoding: 'false'
    xsiNilForNullData: 'true'
    precedence: data

```

Property name	Description
<dataSource name>	Name of the data source. Data type: String Default value: None
connection	Name of the database connection. Data type: String Default value: None
connectionPoolRefreshInterval	Refresh interval to refresh the connection pool to remove idle connections. Data type: Duration Default value: PT30S
cursorCacheRefreshInterval	Refresh interval to refresh the cursor cache to close idle cursors. Data type: Duration Default value: PT30S
cursorCacheSize	Size of cursor cache. Data type: Integer Default value: 1000

Property name	Description
defaultCursorSize	Maximum number of rows that can be fetched as result of a query. Data type: Integer Default value: 2147483647
minimumReadConnections	Minimum number of read connections in the connection pool. Data type: Integer Default value: None
minimumWriteConnections	Minimum number of write connections in the connection pool. Data type: Integer Default value: None
optimizeQueryForCursor	Whether the query will be modified to add database-specific parameters when the request is of cursor vtype. Data type: Boolean Default value: true
queryCacheRefreshInterval	Refresh interval to refresh the query cache to close unused queries. Data type: Duration Default value: PT30S
queryCacheSize	Size of query cache. Data type: Integer Default value: 1000
maximumReadConnections	Maximum number read connections. Data type: Integer Default value: 5
readEmptyStringAsNull	Whether to read an empty string (")as NULL or not. Data type: Boolean Default value: false

Property name	Description
reconnectionAttempts	Number of times the re-connection to database must be tried when the connection is broken. Data type: Integer Default value: 1
reconnectionInterval	Interval between two reconnection attempts. Data type: Duration Default value: PT5S
requestThreshold	Data type: Integer Default value: 1000
useCaseConversion	Whether to convert the table name and field names to lowercase. Data type: Boolean Default value: false
maximumWriteConnections	Maximum number read connections. Data type: Integer Default value: 5
writeEmptyStringAsNull	Whether to update an empty string("") as NULL. Data type: Boolean Default value: false
xmlEncoding	Whether to encode r=table name and column names in the request. Data type: Boolean Default value: false
xsiNilForNullData	Whether to use null="true" or xsi:nil in the response. Data type: Boolean Default value: true
precedence	Data type: String Default value: data

19.2.3 Business Process Management

```
platform:
  organizations:
    <Organization Name>:
      services:
        instances: <BPMForPlatform>
        <BPMForPlatform>:
          kind: BPM
          activateDebugPoints: 'false'
          agingTime: PT1H
          attachmentsTimeout: PT1M
          autoRecoverProcessInstances: 'false'
          caseNetworkSyncLevel: '1'
          clientTimeout: PT30S
          dataSource: system
          deleteAbortedInstanceWithNoMonitoring: 'true'
          doCacheAssignments: 'true'
          enableAging: 'true'
          enableCasePessimisticLocking: 'true'
          enableCrashRecovery: 'true'
          enableNonProcessModel: 'false'
          enablePriority: 'true'
          enableProcessMonitoring: 'true'
          enableRuleEngine: 'false'
          maxInstanceCount: '0'
          maxInstanceInterval: PT1S
          maxIterationCount: '0'
          orchestratorQueueThreshold: '5'
          priorityAlgorithm: Aging
          processModelCacheSize: '200'
          ruleConfiguration: <ruleConfiguration>
          schedulerConfiguration: <schedulerConfiguration>
          threadPoolSizeForLongLivedProcess: '5'
          validateProcessModelOnDeployment: 'true'
          webServiceExecutionDurationWarningThreshold: '1000'
```

Property name	Description
activateDebugPoints	Debug points are usually defined on activities at the time of designing a business process model. This is done to identify and suitably address any bottlenecks that the process may encounter at the time of execution. Identifying the debug points during runtime is possible if the debug points are activated at the Business Process Management Service Container level. When a debug point is encountered, the business process goes into a debug ready state in the Process Instance Manager. Data type: Boolean Default value: false

Property name	Description
enablePriority	Execute business process models based on priority. The type of algorithm list appears when you select Enable Priorities. Priorities can be set while modeling a process or during runtime. Data type: Boolean Default value: true
priorityAlgorithm	Priority algorithm that you apply for executing processes. The following list describes the types of algorithms that you can use to execute processes. • Aging: Processes that have the highest priority are executed first, followed by those with lower priority. • PseudoTimeSlicing: A set of processes is executed from each priority in the priority queue until all the processes are executed. • Stage 1: Five processes of high priority are executed. • Stage 2: Four processes of above-normal priority are executed. • Stage 3: Three processes of normal priority are executed. • Stage 4: Two processes of below-normal priority are executed. • Stage 5: One process of low priority will execute. • Stage 6: Stage1 to Stage 5 is repeated until all the processes are executed. Data type: com.cordys.bpm.config.PriorityAlgorithmType Default value: Aging Dependent properties: enablePriority

Property name	Description
enableAging	Whether you want to change the priority of process instances periodically to a higher priority so that all processes in the priority queue can be eventually executed or if you do not want to change priorities of the process instances in the priority queue. Data type: Boolean Default value: true Dependent properties: priorityAlgorithm == Aging
agingTime	Frequency (in hours) at which the priority of the process instance is assigned a higher priority. Data type: Duration Default value: PT1H Dependent properties: enableAging
attachmentsTimeout	Timeout (in minutes) while uploading attachments. Data type: Duration Default value: PT1M
autoRecoverProcessInstances	Resumes processes in the event of a database connection failure. When set to true, the processes in running or queued state is resumed. Data type: Boolean Default value: false
caseNetworkSyncLevel	When there are service containers configured in a load balancing mode, pessimistic locking is extended to multiple service containers. The information between the container is shared using sync-up messages. In a load-balanced scenario, locking across containers is not enabled by default. This property must be set to value '2' to enable locking. This prevents the same instance from being executed simultaneously on two service containers. With this setting, MoD experiences performance issues. Data type: Integer Default value: 1

Property name	Description
clientTimeout	To set the timeout for a web service used as a BPM activity. Data type: String Default value: data
dataSource	Data source configuration name. Data type: com.opentext.dso.config.DataSourceConfiguration Default value: Not applicable
deleteAbortedInstanceWithNoMonitoring	Whether aborted instances can be cleaned when process-monitoring is set to OFF. Data type: Boolean Default value: true
doCacheAssignments	Whether the assignment is cacheable. Data type: Boolean Default value: true
enableCasePessimisticLocking	Whether case pessimistic locking is on or off. Data type: Boolean Default value: true
enableCrashRecovery	Crash recovery is the ability to restore a process instance from the point of execution failure without having to restart it. If a business process encounters an issue due to system or processor failure, you can restore the system using the crash recovery feature during runtime. Data type: Boolean Default value: true
enableNonProcessModel	Specifies the non-BPM model. Only prior to the C3 version, this is set to true, except in this case, it is set to false. Data type: Boolean Default value: false

Property name	Description
enableOrchestratorDebug	Specifies the Orchestrator debug mode. Data type: Boolean Default value: false
enableProcessMonitoring	Enables process monitoring at the SOAP Processor level. This overrides all settings made at the process and activity levels. Data type: Boolean Default value: true
enableRuleEngine	Enables support for decision tables in business process models. Data type: Boolean Default value: false
maxInstanceCount	Specifies the value of a maximum number of instances that can be triggered in the given duration. Data type: Integer Default value: 0 (means unlimited)
maxInstanceIntervalCount	Specifies the size of the interval to calculate the rate. The default value is 1 second if not specified. Data type: Duration Default value: PT1S
maxIterationCount	Specifies the value of iteration count after which, the process instance will be suspended. Data type: Integer Default value: 0 (means unlimited)
multiDocEnabled	Set the configuration option to having a nom Document per thread. Data type: Boolean Default value: true

Property name	Description
orchestratorQueueThreshold	Specifies the maximum number of process instance objects in the queue after which the BPM Engine sets the configuration option to have a nom Document per thread.ine deletes the process instance XML while adding the process instance object in the queue and loads it from DB when the process instance gets the execution thread. Data type: Integer Default value: 5
processModelCacheSize	Specifies the number of business process models that can be cached. Data type: Long Default value: 200
ruleConfiguration	Creates a Rule Engine Configuration object with a configuration node and managed component. Data type: com.cordys.bre.config.ConnectorConfigurati on Default value: Not applicable
schedulerConfiguration	Creates a Scheduler Configuration object with config node and managed component. Data type: com.cordys.scheduler.config.ConnectorConfi guration Default value: Not applicable
threadPoolSizeForLongLivedProcess	Number of business processes that can run simultaneously. An increase in the number of threads increases the number of processes running in parallel, but at the cost of consuming more memory. Data type: Integer Default value: 5
validateProcessModelOnDeployment	Validates the BPM model while deploying. Data type: Boolean Default value: true

Property name	Description
webServiceExecutionDurationWarningThreshold	Threshold value to log a warning message for the Web services that consume more time than the threshold duration. Data type: Long Default value: 1000

19.2.4 Notification

```
platform:
  organizations:
    <Organization Name>:
      services:
        instances: <NotificationForPlatform>
        <NotificationForPlatform>:
          kind: Notification
          autoClaimOrRevokeInInbox: 'true'
          autoRefresh: 'true'
          dataSource: system
          defaultDelivery: <no default value specified>
          defaultDispatchTimeout: PT0S
          hideSubCaseModelsInCaseInbox: 'false'
          inboxGridView:
            addMemo: 'true'
            addReminders: 'true'
            addTags: 'true'
            assign: 'true'
            claim: 'true'
            complete: 'false'
            delegate: 'true'
            forward: 'true'
            modifySystemAttributes: 'true'
            pause: 'true'
            resume: 'true'
            revoke: 'true'
            showAttachmentButton: 'true'
            skip: 'true'
            start: 'true'
            stop: 'true'
            suspend: 'true'
            viewHistory: 'true'
          inboxTaskView:
            addMemo: 'true'
            addReminders: 'true'
            assign: 'true'
            claim: 'true'
            complete: 'true'
            delegate: 'true'
            forward: 'true'
            modifySystemAttributes: 'true'
            pause: 'true'
            resume: 'true'
            revoke: 'true'
            showAttachmentButton: 'true'
            skip: 'true'
            start: 'true'
            stop: 'true'
            suspend: 'true'
            viewHistory: 'true'
          inboxViewMode: Refresh
          mailId: <no default value specified>
          mailText: This mail is generated by Notification Service. Click on the
hyperlink
```

```

    to access the message.
    numberOfThreads: '5'
    refreshInterval: PT1M
    showAllFolders: 'true'
    showMemoAsTooltip: 'false'
    showNotifications: 'true'
    urlPrefix: <no default value specified>
    useCustomStateModel: 'false'
    userTeamCachePerThread: 'true'

```

Property name	Description
addMemo Parent property: inboxGridView	Add Memo button shows on the toolbar of My Inbox grid view. Data type: Boolean Default value: true
addReminders Parent property: inboxGridView	Whether the Add reminders button shows on the toolbar of My Inbox grid view. Data type: Boolean Default value: true
assign Parent property: inboxGridView	Whether the Assign button shows on the toolbar of My Inbox grid view. Data type: Boolean Default value: true
claim Parent property: inboxGridView	Whether the Claim button shows on the toolbar of My Inbox grid view. Data type: Boolean Default value: true
complete Parent property: inboxGridView	Whether the Complete button shows on the toolbar of My Inbox grid view. Data type: Boolean Default value: false
delegate Parent property: inboxGridView	Whether the Delegate button shows on the toolbar of My Inbox grid view. Data type: Boolean Default value: true
forward Parent property: inboxGridView	Whether the Forward button shows on the toolbar of My Inbox grid view. Data type: Boolean Default value: true

Property name	Description
modifySystemAttributes Parent property: inboxGridView	Whether the Modify System Attributes button shows on the toolbar of My Inbox grid view. Data type: Boolean Default value: true
pause Parent property: inboxGridView	Whether the Pause button shows on the toolbar of My Inbox grid view. Data type: Boolean Default value: true
resume Parent property: inboxGridView	Whether the Resume button shows on the toolbar of My Inbox grid view. Data type: Boolean Default value: true
revoke Parent property: inboxGridView	Whether the Revoke button shows on the toolbar of My Inbox grid view. Data type: Boolean Default value: true
showAttachmentButton Parent property: inboxGridView	Whether the Show attachment button shows on the toolbar of My Inbox grid view. Data type: Boolean Default value: true
skip Parent property: inboxGridView	Whether the Skip button shows on the toolbar of My Inbox grid view. Data type: Boolean Default value: true
start Parent property: inboxGridView	Whether the Start button shows on the toolbar of My Inbox grid view. Data type: Boolean Default value: true
stop Parent property: inboxGridView	Whether the Stop button shows on the toolbar of My Inbox grid view. Data type: Boolean Default value: true

Property name	Description
suspend Parent property: inboxGridView	Whether the Suspend button shows on the toolbar of My Inbox grid view. Data type: Boolean Default value: true
viewHistory Parent property: inboxGridView	Whether the View History button shows on the toolbar of My Inbox grid view. Data type: Boolean Default value: true
addTags Parent property: inboxGridView	Whether the Add Tags button shows on the toolbar of My Inbox grid view. Data type: Boolean Default value: true
addMemo Parent property: inboxTaskView	Whether the Add Memo button shows on the toolbar of My Inbox task view. Data type: Boolean Default value: true
addReminders Parent property: inboxTaskView	Determines whether the Add Reminders button shows on the toolbar of the My Inbox task view. Data type: Boolean Default value: true
assign Parent property: inboxTaskView	Determines whether the Assign button shows on the toolbar of the My Inbox task view. Data type: Boolean Default value: true
claim Parent property: inboxTaskView	Whether the Claim button shows on the toolbar of the My Inbox task view. Data type: Boolean Default value: true
complete Parent property: inboxTaskView	Determines whether the Complete button shows on the toolbar of the My Inbox task view. Data type: Boolean Default value: true

Property name	Description
delegate Parent property: inboxTaskView	Whether the Delegate button shows on the toolbar of the My Inbox task view. Data type: Boolean Default value: true
forward Parent property: inboxTaskView	Whether the Forward button shows on the toolbar of the My Inbox task view. Data type: Boolean Default value: true
modifySystemAttributes Parent property: inboxTaskView	Whether the Modify System Attributes button shows on the toolbar of My Inbox task view. Data type: Boolean Default value: true
pause Parent property: inboxTaskView	Whether the Pause button shows on the toolbar of My Inbox task view. Data type: Boolean Default value: true
resume Parent property: inboxTaskView	Determines whether the Resume button shows on the toolbar of My Inbox task view. Data type: Boolean Default value: true
revoke Parent property: inboxTaskView	Whether the Revoke button shows on the toolbar of My Inbox task view. Data type: Boolean Default value: true
showAttachmentButton Parent property: inboxTaskView	Whether the Show attachment button shows on the toolbar of the My Inbox task view. Data type: Boolean Default value: true
skip Parent property: inboxTaskView	Whether the Skip button shows on the toolbar of the My Inbox task view. Data type: Boolean Default value: true

Property name	Description
start Parent property: inboxTaskView	Determines whether the Start button shows on the toolbar of the My Inbox task view. Data type: Boolean Default value: true
stop Parent property: inboxTaskView	Whether the Stop button shows on the toolbar of the My Inbox task view. Data type: Boolean Default value: true
suspend Parent property: inboxTaskView	Determines whether the Suspend button shows on the toolbar of My Inbox task view. Data type: Boolean Default value: true
viewHistory Parent property: inboxTaskView	Whether the View History button shows on the toolbar of My Inbox task view. Data type: Boolean Default value: true
autoClaimOrRevokeInInbox	Enables a user to claim the task as soon as it opens. If the user opens and claims the task without any action, the task reverts to the New state automatically. Data type: Boolean Default value: true
autoRefresh	If true, refreshes the Inbox automatically. Data type: Boolean Default value: true
dataSource	Data source configuration name. Data type: com.opentext.dso.config.DataSourceConfiguration Default value: Not applicable
defaultDelivery	Mail or CPCINBOX. If the target is not specified in the request, the NA value is assumed. Data type: String Default value: Not applicable

Property name	Description
defaultDispatchTimeout	Time after which tasks that are not completed are reassigned to the next available recipient. Data type: Duration Default value: PT0S
hideSubCaseModelsInCaseInbox	Types of case models to display in the Case(s) tab of My Inbox Make this true to hide the sub-case models on the Case tab of My Inbox. By default, this is set to false and all the sub-case models belonging to a case are displayed in the Inbox. Data type: Boolean Default value: false
inboxViewMode	How to check for new messages. • To check for new messages (in the Inbox), click Refresh. This option is selected by default. • To be alerted automatically when you receive a new message, click Push, select the event processor from the Choose Target window, and click OK. Data type: String Default value: refresh
numberOfThreads	Number of messages that can be triggered simultaneously. The default value is 5. If you increase the number of threads, the number of messages that can be triggered simultaneously also increases. However, this uses more memory. Data type: Integer Default value: 5
refreshInterval	Time interval (in minutes) between two successive auto-refresh operations. Data type: Duration Default value: PT1M

Property name	Description
showAllFolders	Show Empty Work Lists - Displays the worklists that do not contain tasks. Data type: Boolean Default value: true
showMemoAsTooltip	Enables showing the memo associated with a task as a tooltip.  Note: In Refresh mode, the tooltip information that is associated with the selected task is cached. The cache is updated only when the Inbox is refreshed. Data type: Boolean Default value: false
showNotifications	Displays the Notifications option in My Inbox. Data type: Boolean Default value: true
useCustomStateModel	Use a custom state model. Data type: Boolean Default value: false
userTeamCachePerThread	Cache user team mapping per thread. Data type: Boolean Default value: true

19.2.5 Scheduling

```
platform:
  organizations:
    <Organization Name>:
      scheduler:
        configurations:
          <schedulerConfiguration>:
            dataSource: system
            eventCursorSize: '5000'
            loadScheduleEventsAsync: 'true'
            missedSchedulesTriggerType: one
        services:
          instances: <SchedulerForPlatform>
          <SchedulerForPlatform>:
            kind: Scheduler
            configuration: <schedulerConfiguration>
```


Property name	Description
configurations	Set of configurations of the scheduler.
dataSource	Data source configuration name. Data type: com.opentext.dso.config.DataSourceConfiguration Default value: Not applicable
eventCursorSize	Cursor size for the load schedule events. Data type: Integer Default value: 5000
loadScheduleEventsAsync	If the schedule events must be loaded asynchronously on the starting of the engine. Data type: Boolean Default value: true
missedSchedulesTriggerType	How missed schedule events must be triggered. This takes only enumerated properties (MissedScheduleTriggerType) of the following three options. • one • none • all Data type: MissedScheduleTriggerType Default value: one

19.2.6 RPA

```
<RPA>:
  kind: RPA
  rpaServiceBaseUrl: ''
```

Property name	Description
rpaServiceBaseUrl	URL of the RPA service in the following format: http://<RPA web server host name>:<RPA port no>/rpaservice For example: http://kancm.lab.com:8081/rpaservice Data type: String Default value: None

19.2.7 Rule Management

```
platform:
  organizations:
    <Organization Name>:
      services:
        instances: <PIMForPlatform>
        <PIMForPlatform>:
          kind: PIM
          dataSource: system
```

Property name	Description
configurations	Set of configurations of the rule.
customMonitorListener	Name for the custom rule monitor listener configuration. Data type: String Default value: Not applicable
dataSource	Data source configuration name. Data type: com.opentext.dso.config.DataSourceConfiguration Default value: Not applicable
maxDispatcherQueueSize	Maximum size of the rule action dispatcher queue that must be held. Data type: Integer Default value: 50000
monitorRequestAndResponse	Configuration to monitor (persist) request and response data of external rule actions. Data type: Boolean Default value: false
ruleAudit	Whether to audit the rule monitor. Data type: Boolean Default value: false
threadPoolSize	Rule action thread pool size. Data type: Integer Default value: 5

19.2.8 PIMQuery Connector

```
platform:
  organizations:
    <Organization Name>:
      services:
        instances: <PIMForPlatform>
        <PIMForPlatform>:
          kind: PIM
          dataSource: system
```

Property name	Description
dataSource	Data source configuration name. Data type: com.opentext.dso.config.DataSourceConfiguration Default value: Not applicable
multiDocEnabled	Configuration option for having a nom Document per thread. Data type: Boolean Default value: true

19.2.9 ArchiveApplicationConnector

```
platform:
  organizations:
    <Organization Name>:
      services:
        instances: <BPMArchiveForPlatform>
        <BPMArchiveForPlatform>:
          kind: BPMArchive
          dataSource: system
```

Property name	Description
archiveMultiDocEnabled	Configuration option for having a nom Document per thread. Data type: Boolean Default value: true
dataSource	Data source configuration name. Data type: com.opentext.dso.config.DataSourceConfiguration Default value: Not applicable

19.2.10 Lifecycle building block

```
platform:
  organizations:
    <Organization Name>:
      entityRuntime:
        settings:
          entityTask:
            maxNumOfAllTargetsCacheEntries: '1000'
            maxNumOfIdToNameCacheEntries: '10000'
            timeToLiveForAllTargetsCacheEntry: PT30S
            timeToLiveForIdToNameCacheEntry: PT30S
          services:
```

Property name	Description
timeToLiveForAllTargetsCacheEntry	How long entries of the 'All targets' cache must live. Data type: Duration Default value: PT30S
maxNumOfAllTargetsCacheEntries	Maximum number of entries in the cache to hold. Data type: Long Default value: 1000
timeToLiveForIdToNameCacheEntry	How long the entries of the 'Id to Name' cache must live. Data type: Duration Default value: PT30S
maxNumOfIdToNameCacheEntries	Maximum number of entries the cache must hold. Data type: Long Default value: 10000

19.2.11 EIS Connector

The applicable properties that can be set for the EIS connector in a shared organization are provided in the following sample configuration settings:

```
platform:
  eisRepositoryConfigurations:
    SampleConnEisConfig:
      driverClass: com.microsoft.sqlserver.jdbc.SQLServerDriver
      username: sa
      password: $9Lserver
      jdbcURL: jdbc:sqlserver://
localhost:1433;sendStringParametersAsUnicode=false;databaseName=PPRe1
    SampleConnEisConfig2:
      driverClass: com.microsoft.sqlserver.jdbc.SQLServerDriver
      username: sa
      password: $9Lserver
```

```

        jdbcURL: jdbc:sqlserver://
localhost:1433;sendStringParametersAsUnicode=false;databaseName=PPRe12
organizations:
  system:
    entityRuntime:
      solutions:
        MyCompanyMyCompanyTestEISMPC:
          eisRepositoryReferences:
            TestEISRefMPC:
              connectorId: 00505601-247D-A1EA-8AC0-F1E67AF640F6
              configurationId: SampleConnEisConfig2

```

The applicable properties that can be set for the EIS connector in a specific organization in the MPC file are provided in the following sample configuration settings:

```

platform:
  eisRepositoryConfigurations:
    SampleConnEisConfig:
      driverClass: com.microsoft.sqlserver.jdbc.SQLServerDriver
      username: sa
      password: $9Lserver
      jdbcURL: jdbc:sqlserver://
localhost:1433;sendStringParametersAsUnicode=false;databaseName=PPRe1
    SampleConnEisConfig2:
      driverClass: com.microsoft.sqlserver.jdbc.SQLServerDriver
      username: sa
      password: $9Lserver
      jdbcURL: jdbc:sqlserver://
localhost:1433;sendStringParametersAsUnicode=false;databaseName=PPRe12
  organizations:
    system:
      entityRuntime:
        solutions:
          MyCompanyMyCompanyTestEISMPC:
            eisRepositoryReferences:
              TestEISRefMPC:
                connectorId: 00505601-247D-A1EA-8AC0-F1E67AF640F6
                configurationId: SampleConnEisConfig2

```

The following table describes various property names, their data types, the default values they accept, and their descriptions.

Property name	Description
eisRepositoryConfigurations	Set of EIS repository references.
SampleConnEisConfig	Name of the EIS repository configuration available in the System resource manager. Data type: String Default value: Not applicable
driverClass,username,password,jdbcURL	Depends on the EIS connector properties. Data type: None Default value: Dynamic
MyCompanyMyCompanyTestEISMPC	Name of the solution.
eisRepositoryReferences	

Property name	Description
TestEISRefMPC	Name of the EIS repository reference.
connectorId	ID of the EIS connector generated after publishing in XML store. Data type: String Default value: None
configurationId	ID of the EIS repository configuration generated in XML store Dependent properties: SampleConnEisConfig. Data type: String Default value: None

19.2.12 Email configuration

The applicable properties that can be set for configuring your email in a shared organization are provided in the sample configuration settings:

```
platform:
  entityRuntime:
    solutions:
      <project name>:
        emailConfigurations:
          <email configuration name>:
            fromEmailId: <no default value specified>
            fromDisplayName: <no default value specified>
            replyToEmailId: <no default value specified>
            useLoggedInUserNameAsEmailFromDisplayName:<no default value specified>
            emailProfileUserId: <no default value specified>
            prefixes:
              prefix-1:
                prefix : <no default value specified>
                entityName : <no default value specified>
                solutionName : <no default value specified>
              prefix-2:
                prefix : <no default value specified>
                entityName : <no default value specified>
                solutionName : <no default value specified>
```

The applicable properties that can be set for configuring your email in a shared organization are provided in the following sample configuration settings:

```
platform:
  organizations:
    <organization name>:
      entityRuntime:
        solutions:
          <project name>:
            emailConfigurations:
              <email configuration name>:
                fromEmailId: <no default value specified>
                fromDisplayName: <no default value specified>
                replyToEmailId: <no default value specified>
                useLoggedInUserNameAsEmailFromDisplayName:<no default value specified>
                emailProfileUserId: <no default value specified>
                prefixes:
```

```

prefix-1:
  prefix : <no default value specified>
  entityName : <no default value specified>
  solutionName : <no default value specified>
prefix-2:
  prefix : <no default value specified>
  entityName : <no default value specified>
  solutionName : <no default value specified>

```

The following table describes various property names, their data types, the default values they accept, and their descriptions.

Property name	Description
<organization name>	Name of the organization. Data type: String Default value: None
<project name>	Name of the project. Data type: String Default value: None
emailConfigurations	Set of email configurations. Data type: String Default value: None
<email configuration name>	Name of the email configuration used in the project. Data type: String Default value: None
fromEmailId	Email ID with which the SET PROFILE is done from WEB SERVICE INTERFACE EXPLORER. Data type: String Default value: None
replyToEmailId	Email ID with which the SET PROFILE is done from WEB SERVICE INTERFACE EXPLORER. Data type: Text Default value: None

Property name	Description
useLoggedInUserNameAsEmailFromDisplayName	Shows the email id as USERNAME when set to True. Data type: Boolean Default value: false
emailProfileUserId	Logged in User ID with which the SET PROFILE is done from WEB SERVICE INTERFACE EXPLORER. Data type: Text Default value: None
prefixes	Set of Business IDs given in entities. Data type: Text Default value: None
prefix-1,prefix-2	To configure multiple business IDs. Data type: Text Default value: None
prefix	Business ID is set under IDENTITY BB in the Entity. Data type: Text Default value: None
entityName	Name of the entity that the user wants to map the business ID prefix. Data type: Text Default value: None Dependent properties: prefix
solutionName	Name of the solution that has the entity. Data type: Text Default value: None Dependent properties: entityName

19.2.13 CWS settings

```
platform:
  cws:
    buildOutputBackupFolder: ${platform.install.dir}/cws/build.bck
    buildOutputFolder: ${platform.install.dir}/cws/build
    careIsvpInstallTimeout: PT30S
    checkoutFolder: <no default value specified>
    excludedFilenameExtensionsForSynchronization: .svn:.bak:.cvs:.swp
    excludedFilenamePrefixesForSynchronization: '~'
    maxNumberOfOperationThreads: <no default value specified>
    maxOperationQueueCapacity: <no default value specified>
    minNumberOfOperationThreads: <no default value specified>
    operationThreadsKeepAliveTime: PT10M
    psIsvpInstallTimeout: PT6M
    stagingIsvpInstallTimeout: PT6M
    synchronizeBackupFolder: ${platform.install.dir}/cws/sync.bck
    synchronizeFolder: ${platform.install.dir}/cws/sync
    teamDevelopmentSvnVersion: <no default value specified>
```

The following table describes various property names, their data types, the default values they accept, and their descriptions.

Property name	Description
buildOutputBackupFolder	Location of the build output backup folder. When deleting a workspace, the contents of the build output folder of that workspace at the time of deletion are moved to this location. Setting: build.output.backup Default value: None
buildOutputFolder	Location for saving the build output. The TomEE user must have read/write permissions for this location. Setting: build.output Default value: \${platform.install.dir}/cws/build
careIsvpInstallTimeout	Timeout for publishing artifacts to CARE. Setting: care.isvp.install.timeout Default value: PT30S

Property name	Description
checkoutFolder	OpenText stores third-party libraries in one location in the source checkout. The goal of this one location is to ensure that every platform component uses the same third party library. This makes it simple to upgrade third party libraries to a newer version. There is no need to upgrade third party libraries stored in the individual platform components. The SDK package <code>OpenText Third Party Library Definition.cap</code> uses the environment variable <code>BUILD_HOME</code> (as used in the standard build script of the platform) or the CWS property <code>checkout.folder</code> to locate the third party folder used in the modeler project. If both are defined, the CWS property <code>checkout.folder</code> is used. Setting: <code>checkout.folder</code> Default value: None
excludedFilenameExtensionsForSynchronization	Files in the synchronize folder with the specified extensions are not synchronized. Setting: <code>synchronize.filesuffix.exclusions</code> Default value: <code>.svn:.bak:.cvs:.swp</code>
excludedFilenamePrefixesForSynchronization	Files in the synchronize folder starting with this prefix are not synchronized. Multiple prefixes are separated with a colon. The complete list must be within quotes (For example: <code>'~!::%'</code> or prefixes starting with <code>'~'</code>, <code>'!</code> or <code>'%'</code>.) Setting: <code>synchronize.fileprefix.exclusions</code> Default value: <code>'~'</code>
maxNumberOfOperationThreads	Maximum number of operation threads. Setting: <code>maximal.allowed.number.of.operation.threads</code> Default value: None Internal calculated default = #CPU's * 10 Dependent properties: <code>maxOperationQueueCapacity</code>

Property name	Description
maxOperationQueueCapacity	Maximum size of the operation thread pool. Setting: operation.queue.capacitys Default value: None Internal calculated default = maxNumberOfOperationThreads * 3
minNumberOfOperationThreads	Minimum number of operation threads. Setting: number.of.operation.threads.to.keepalive Default value: None Internal calculated default = #CPU's * 4
operationThreadsKeepAliveTime	Time to keep an operation thread alive. Setting: idle.operation.expiration.timeout Default value: PT10M
pslIsvpInstallTimeout	Timeout for publishing artifacts to the Published Source Space. Setting: psl.isvp.install.timeout Default value: PT6M
stagingIsvpInstallTimeout	Timeout for publishing artifacts to the staging area when deploying staging application packages. Setting: staging.isvp.install.timeout Default value: PT6M
synchronizeBackupFolder	Location of the synchronization backup folder. When deleting a workspace, the content of the sync folder of that workspace at the time of deletion is moved to this location. Setting: synchronize.folder.backup Default value: \${platform.install.dir}/cws/sync.bck

Property name	Description
synchronizeFolder	Location for writing the synchronize content. The TomEE user must have read/write permissions for this location. When this folder is changed, the CWS tool with subcommand <code>movesynchronizelocation</code> must be used to move the content of existing workspaces to the new folder.  Note: The tool may update the <code>cws.properties</code> file, but the contents of this file are ignored when the <code>cws</code> properties are defined in a <code>yaml</code> file. Setting: <code>synchronize.folder</code> Default value: <code>\${platform.install.dir}/cws/sync</code>
teamDevelopmentSvnVersion	AppWorks Platform 23.4 supports SVN versions 1.6, 1.7, 1.8, 1.9 and 1.10. By default, the latest supported version is used. When your business requires a specific SVN version, this property can be set to a fixed value (For example: '1.6': Note that the quotes are needed to prevent the version number from being interpreted as a floating point number)  Note: SVN 1.6 uses working copy format 1.6 SVN 1.7 uses working copy format 1.7 SVN 1.8, 1.9 and 1.10 use working copy format 1.8. Setting: <code>teamdevelopment.svn.workingcopy.version</code> Default value: None Internal default = latest supported SVN version

19.2.14 FTP Connector

The applicable properties that can be set for the FTP connector are provided in the sample configuration settings:

```
<FTPHandler>:
  kind: com.eibus.applicationconnector.ftp.FTPHandler
  connections:
    <ftpConnection>:
      kind: FTP
      server: <no default value specified>
      port: 21
      userName: ''
      password: ''
      basePath: <no default value specified>
      transferMode: Passive
      transferModeActive:
        externalIp: ''
        port:
          from: '0'
          to: '0'
    <ftpesConnection>:
      kind: FTPES
      server: <no default value specified>
      port: 21
      userName: ''
      password: ''
      basePath: <no default value specified>
      transferMode: Passive
      clearCommandChannel: 'false'
      protectDataChannel: 'true'
      transferModeActive:
        externalIp: ''
        port:
          from: '0'
          to: '0'
    <ftpisConnection>:
      kind: FTPIS
      server: <no default value specified>
      port: 990
      userName: ''
      password: ''
      basePath: <no default value specified>
      transferMode: Passive
      transferModeActive:
        externalIp: ''
        port:
          from: '0'
          to: '0'
    <sftpConnection>:
      kind: SFTP
      server: <no default value specified>
      port: 22
      userName: ''
      password: ''
      basePath: <no default value specified>
      privateKey:
        file: <no default value specified>
        password: <no default value specified>
      transferModeActive:
        externalIp: ''
        port:
          from: '0'
          to: '0'
  proxy:
    type: <no default value specified>
    server: ''
    port: <no default value specified>
```

```

userName: <no default value specified>
password: <no default value specified>

```

The following table describes various property names, their data types, the default values they accept, and their descriptions.

Property name	Description
<FTPHandler>	Section to configure the FTP connector configuration.
connections	Section to configure the FTP connections.
<ftpConnection>	FTP protocol specific properties
kind	Specify 'FTP' in order to use the 'FTP' protocol. Data type: ConnectionKind Default value: FTP
server	Host name or IP address of the FTP server to connect to. Data type: String Default value: None
port	Port at which the FTP server listens to client connection (usually 21 for the FTP protocol). Data type: Integer Default value: 21
userName	User name for FTP server authentication. Data type: String Default value: None
password	Password for FTP server authentication. Data type: Password Default value: None
basePath	Directory used as the base location by the FTP Service for uploading and downloading files. Data type: String Default value: None

Property name	Description
transferMode	'Active' or 'Passive' - Choose the mode of transfer. Active mode FTP means that your client connects to the FTP server on the command port and then the FTP attempts to make a connection to your client for the data port, which might be a problem in a corporate network. Changing to passive mode means that the client connects to the server for both command and data port transmission. You can provide the IP Address and the port ranges which the FTP server can use for data transfer, and the fields would be shown when active mode is selected. Data type: String Default value: Passive
transferModeActive	Following settings are only used when the transfer mode is Active.
externallp	External IP address that the FTP service uses in the PORT command in case the network uses NAT for the FTP server to connect back to the FTP service. Data type: String Default value: None
port	Port range on which the FTP server can connect back.  Note: This range must be open in the firewall for the FTP server to connect back.
from	Port from must be above 1023. Data type: Integer Default value: 0
to	Port to must be above 1023. Data type: Integer Default value: 0
<ftpesConnection>	FTPS explicit protocol specific properties

Property name	Description
kind	Specify 'FTPES' in order to use the 'FTPS explicit' protocol. Data type: ConnectionKind Default value: FTPES
server	Host name or IP address of the FTP server to connect to. Data type: String Default value: None
port	Port at which the FTP server listens to client connection (usually 21 for the FTPS explicit protocol). Data type: Integer Default value: 21
userName	User name for FTP server authentication. Data type: String Default value: None
password	Password for FTP server authentication. Data type: Password Default value: None
basePath	Directory used as the base location by the FTP Service for uploading and downloading files. Data type: String Default value: None
protectDataChannel	Whether to encrypt the data channel. Can be false in case the data is already encrypted in some way. Data type: Boolean Default value: true
clearCommandChannel	Whether to use a Clear Command Channel. When set to true, all the FTP commands are sent in clear text; when set to false, the commands are encrypted. Data type: String Default value: false

Property name	Description
transferMode	'Active' or 'Passive' - Choose the mode of transfer. Active mode FTP means that your client connects to the FTP server on the command port and then the FTP attempts to make a connection to your client for the data port, which might be a problem in a corporate network. Changing to passive mode means that the client connects to the server for both command and data port transmission. You can provide the IP Address and the port ranges which the FTP server can use for data transfer, and the fields would be shown when active mode is selected. Data type: String Default value: Passive
transferModeActive	Following settings are only used when the transfer mode is Active.
externallp	External IP address that the FTP service will use in the PORT command in case the network uses NAT for the FTP server to connect back to the FTP service. Data type: String Default value: None
port	Port range on which the FTP server can connect back. Note that this range must be open in the firewall for the FTP server to connect back.
from	Port from must be above 1023. Data type: String Default value: None
to	Port to must be above 1023. Data type: Integer Default value: 0
<ftpisConnection>	User defined ftpisConnection name

Property name	Description
kind	Specify 'FTPIS' to use the 'FTPS implicit' protocol. Data type: ConnectionKind Default value: FTPIS
server	Host name or IP address of the FTP server to connect to. Data type: String Default value: None
port	Port at which the FTP server listens to the client connection (usually 990 for the FTPS implicit protocol). Data type: Integer Default value: 990
userName	User name for FTP server authentication. Data type: String Default value: None
password	Password for FTP server authentication. Data type: Password Default value: None
basePath	Directory used as the base location by the FTP Service for uploading and downloading files. Data type: String Default value: None

Property name	Description
transferMode	'Active' or 'Passive' - Choose the mode of transfer. Active mode FTP means that your client connects to the FTP server on the command port and then the FTP attempts to make a connection to your client for the data port, which might be a problem in a corporate network. Changing to passive mode means that the client connects to the server for both command and data port transmission. You can provide the IP Address and the port ranges which the FTP server can use for data transfer, and the fields would be shown when active mode is selected. Data type: String Default value: Passive
transferModeActive	Following settings are only used when the transfer mode is Active.
externallp	External IP address that the FTP service will use in the PORT command in case the network uses NAT for the FTP server to connect back to the FTP service. Data type: String Default value: None
port	Port range on which the FTP server can connect back. Note that this range must be open in the firewall for the FTP server to connect back.
from	Port from must be above 1023. Data type: Integer Default value: 0
to	Port to must be above 1023. Data type: Integer Default value: 0
<sftpConnection>	User defined sftpConnection name

Property name	Description
kind	Specify 'SFTP' in order to use the 'SFTP' (SSH File Transfer Protocol) protocol. Data type: ConnectionKind Default value: SFTP
server	Host name or IP address of the FTP server to connect to. Data type: String Default value: None
port	Port at which the FTP server listens to client connection (usually 22 for the SFTP protocol). Data type: Integer Default value: 22
userName	User name for FTP server authentication. Data type: String Default value: None
password	Password for FTP server authentication. Data type: Password Default value: None
basePath	Directory used as the base location by the FTP Service for uploading and downloading files. Data type: String Default value: None
privateKey	Configuration details for SSH private key.
file	File path to the file holding the private key. Used when set, otherwise username or password are used. Data type: String Default value: None
password	Password to open the private key file. Data type: Password Default value: None

Property name	Description
proxy	Proxy server configuration.
type	Type of the proxy server. Following are supported: HTTP, SOCKS 4 and SOCKS 5. Data type: String Default value: None
server	Host name of the proxy server to connect to. Data type: String Default value: None
port	Port where the proxy server listens for connections. Data type: Integer Default value: None
userName	User name for proxy server authentication. Data type: String Default value: None
password	Password for proxy server authentication. Data type: Password Default value: None

19.2.15 Email connector

The applicable properties that can be set for configuring the email connector are provided in the sample configuration settings below:



Notes

- To use the email connector with Oracle database, change the 'xmlEncoding:' value to true in data source.
- 'Disable Poller Configuration' check box configuration is not applicable for Container Environment or MPC configuration.

19.2.15.1 Classic

```

email:
  incoming:
    protocol: POP3
    server: <no default value specified>
    port: '995'
    numberOfPollerThreads: '0'
    properties:
      <propertyName>: <no default value specified>
  outgoing:
    server: <no default value specified>
    port: '465'
    authenticationRequired: 'false'
    contentType: PLAIN
    mime:
      bypassSmime: 'false'
      checkCrl: 'false'
      enableSmime: 'false'
      encryptMails: 'false'
      signMails: 'false'
    properties:
      <propertyName>: <no default value specified>
  profiles:
    <passwordProfile>:
      kind: Password
      emailAddress: <no default value specified>
      displayName: <no default value specified>
      userName: <no default value specified>
      password: <no default value specified>
  emailBoxes:
    instances: ''
    <emailBoxName>:
      triggersDefinitionXmlStoreFilePath: /OpenText/EmailConnector/default_triggers.xml
      profile: <no default value specified>
      automaticRestart: 'true'
      restartInProgress: 'false'
    storage:
      dataSource: <no default value specified>
      pollerInterval: PT10S
      maxEmailsPerPollingCycle: '15'
      maxExecutionThreads: '5'
      maxNumberOfRows: '100'
      compressData: 'false'
  ...

<MailConnector>:
  kind: com.eibus.applicationconnector.email.MailConnector

```

The following table describes various property names, their data types, the default values they accept, and their descriptions.

Property name	Description
email	Section to configure the Email connector.
incoming	Configuration for the incoming mail.

Property name	Description
protocol	Type protocol to use for incoming mail. Following are supported: POP3 and IMAP. Data type: IncomingProtocol Default value: POP3
server	Host name of the Email server to connect to. Data type: String Default value: None
port	Port at which the Email server listens to the client connection (usually 995 for the POP3 encrypted and 993 for IMAP encrypted). Data type: Integer Default value: 995
numberOfPollerThreads	Maximum number of threads the Email connector can use for the email-box poller logic. Default is in thread per mailbox. This setting is useful to limit the amount of simultaneously active threads and associated memory with these threads. Data type: Integer Default value: 0
properties	Allows to set specific connection properties for the incoming mail connection.

Property name	Description
<propertyName>	Name and value of one of the supported properties mentioned below. For example: 'mail.pop3.starttls.enable': true. The complete list of supported properties is: POP3: • mail.pop3.starttls.enable • mail.pop3.starttls.required • mail.pop3.ssl.enable • mail.pop3.proxy.host • mail.pop3.proxy.port • mail.pop3.proxy.user • mail.pop3.proxy.password • mail.pop3.socks.host • mail.pop3.socks.port IMAP: • mail.imap.starttls.enable • mail.imap.starttls.required • mail.imap.ssl.enable • mail.imap.proxy.host • mail.imap.proxy.port • mail.imap.proxy.user • mail.imap.proxy.password • mail.imap.socks.host • mail.imap.socks.port The Microsoft Exchange mail server requires the use of mail.pop3/imap.starttls.enable=true. Data type: String Default value: None
outgoing	Configuration for the outgoing mail, only SMTP is supported.
server	Host name of the Email server to connect to. Data type: String Default value: None

Property name	Description
port	Port at which the Email server listens to client connection (usually 587 for the SMTP protocol). Data type: Integer Default value: 465
authenticationRequired	Whether the Email server requires authentication in order to send mails. When set to 'true', then also provide the user credentials in the 'profiles' section. Data type: Boolean Default value: false
contentType	Type of content. Following are supported: PLAIN and HTML. Data type: String Default value: None
mime	Allows to configure specific mime settings
checkCrl	Whether the certificates should be checked against their CRL. Data type: Boolean Default value: false
bypassSmime	Whether mails can bypass the S/MIME when no certificate is found. Data type: Boolean Default value: false
enableSmime	Whether S/MIME is enabled. Data type: Boolean Default value: false
encryptMails	Whether mails that are being sent should be encrypted. Data type: Boolean Default value: false

Property name	Description
signMails	Whether mails that are being sent should be signed. Data type: Boolean Default value: false
properties	Allows to set specific connection properties for the outgoing mail connection.
<propertyName>	Name of one of the supported properties mentioned below with its value. For example: 'mail.smtp.starttls.enable': true. The complete list of supported properties is: SMTP: • mail.smtp.ssl.enable • mail.smtp.starttls.enable • mail.smtp.starttls.required • mail.smtp.proxy.host • mail.smtp.proxy.port • mail.smtp.proxy.user • mail.smtp.proxy.password • mail.smtp.socks.host • mail.smtp.socks.port The Microsoft Exchange mail server requires the use of mail.smtp.starttls.enable=true. Data type: String Default value: None
profiles	
<passwordProfile>	User defined passwordProfile name
kind	Data type: String Default value: Password
emailAddress	Email address of the user account for which to send emails. Data type: String Default value: None
displayName	Name to use for the sender. Data type: String Default value: None

Property name	Description
userName	User name for Email server authentication. Data type: String Default value: None
password	Password for Email server authentication. Data type: Password Default value: None
emailBoxes	Multiple mailbox configurations.
instances	Specifies the names of the emailboxes to be used by the poller logic. Data type: String[] Default value: None
<emailBoxName>	Specifies the name for an mailbox configuration.
triggersDefinitionXmlStoreFilePath	Path in XML store where the mailbox definition resides (/OpenText/EmailConnector/default_triggers.xml). Data type: String Default value: None
profile	Name of the profile to be used by the poller logic, as defined in the 'profiles' section. Data type: PasswordProfile Default value: None
automaticRestart	Holds whether when starting this thread, the database must be queried to automatically restart the containers that are in status INITIAL (and optionally IN_PROGRESS). Data type: Boolean Default value: true

Property name	Description
restartInProgress	Indicates if the containers in status IN_PROGRESS should be restarted. If a container was already in progress then it will be reprocessed. This means that the SOAP message is called twice depending on the exact moment the processor crashes. Data type: Boolean Default value: false
storage	Section to configure the email storage
dataSource	Name of the data source to use when connecting to the database. Data type: DataSourceConfiguration Default value: None
compressData	Whether the Email Connector should gzip the data before it is written to the database. This can limit the storage you need tremendously. Data type: Boolean Default value: false
pollerInterval	Interval to poll the database for new email messages to be processed. The value must be specified in the ISO-8601 format. Default is 10 seconds. Data type: Duration Default value: PT10S
maxEmailsPerPollingCycle	The maximum number of messages that should be processing in a single polling cycle. Data type: Integer Default value: 15
maxExecutionThreads	Maximum number of worker threads for the execution engine. Data type: Integer Default value: 5

Property name	Description
maxNumberOfRows	Maximum number of rows that will be read from the database per single polling cycle. Data type: Integer Default value: 100
<MailConnector>	Section to configure the Email connector.

19.2.15.2 Classic (OAuth)

```
email:
  mailServerType: CLASSICOAUTH
  oauthDetails:
    clientId: <no default value specified>
    clientSecret: <no default value specified>
    refreshTokenExpiryInDays: <no default value specified and non-mandatory field>
  incoming:
    numberOfPollerThreads: '1'
    port: 993
    properties:
      <propertyName>: <no default value specified>
    protocol: IMAP
    server: <no default value specified>
  outgoing:
    authenticationRequired: true
    port: 587
    server: <no default value specified>
    properties:
      <propertyName>: <no default value specified>
  storage:
    dataSource: <no default value specified>
    pollerInterval: 'PT10S'
  profiles:
    <Password profile>:
      displayName: <no default value specified>
      emailAddress: <no default value specified>
      kind: PASSWORD
      password: <no default value specified>
      userName: <no default value specified>
      refreshTokenExpiry: <no default value specified>
      isRefreshToken: <no default value specified>
```

The following table describes various property names, their data types, the default values they accept, and their descriptions.

Property name	Description
email	Section to configure the Email connector.
mailServerType	CLASSICOAUTH for OAuth mail type.
ClientID	Client Id of the application. Data type: String Default value: None

Property name	Description
ClientSecret	Client secret of the application. Data type: String Default value: None
refreshTokenExpiryInDays	Refresh token expiry (in days) value, Field is non-mandatory. Data type: Integer Default value: None
incoming	Configuration for the incoming mail.
numberOfPollerThreads	Maximum number of threads the Email connector can use for the email-box poller logic. Default is in thread per mailbox. This setting is useful to limit the amount of simultaneously active threads and associated memory with these threads. Data type: Integer Default value: 0
Port	Port at which the Email server listens to the client connection (993 for IMAP encrypted). Data type: Integer Default Value: 110

Property name	Description
<propertyName>	Name and value of one of the supported properties mentioned below. For example: 'mail.starttls.enable': true. The complete list of supported properties is: IMAP: • mail.imap.starttls.enable • mail.imap.starttls.required • mail.imap.ssl.enable • mail.imap.proxy.host • mail.imap.proxy.port • mail.imap.proxy.user • mail.imap.proxy.password • mail.imap.socks.host • mail.imap.socks.port The Microsoft Exchange mail server requires the use of mail.imap.starttls.enable=true. Data type: String Default value: None
protocol	Type protocol to use for incoming mail. Following are supported: IMAP. Data type: IncomingProtocol Default value: IMAP
server	Host name of the Email server to connect to. Data type: String Default value: None
outgoing	Configuration for the outgoing mail, only SMTP is supported.
authenticationRequired	Whether the Email server requires authentication in order to send mails. When set to 'true', then also provide the user credentials in the 'profiles' section. Data type: Boolean Default value: false
port	Port at which the Email server listens to client connection (usually 587 for the SMTP protocol). Data type: Integer

Property name	Description
server	Host name of the Email server to connect to. Data type: String Default value: None
properties	Allows to set specific connection properties for the outgoing mail connection.
<propertyName>	Name of one of the supported properties mentioned below with its value. For example: 'mail.smtp.starttls.enable': true. The complete list of supported properties is: SMTP: • mail.smtp.ssl.enable • mail.smtp.starttls.enable • mail.smtp.starttls.required • mail.smtp.proxy.host • mail.smtp.proxy.port • mail.smtp.proxy.user • mail.smtp.proxy.password • mail.smtp.socks.host • mail.smtp.socks.port The Microsoft Exchange mail server requires the use of mail.smtp.starttls.enable=true. Data type: String Default value: None
storage	Section to configure the email storage.
dataSource	Name of the data source to use when connecting to the database. Data type: DataSourceConfiguration Default value: None
pollerInterval	Interval to poll the database for new email messages to be processed. The value must be specified in the ISO-8601 format. Default is 10 seconds. Data type: Duration Default value: PT10S
profiles	
<passwordProfile>	User defined passwordProfile name.

Property name	Description
displayName	Name to use for the sender.
emailAddress	Email address of the user account for which to send emails. Data type: String Default value: None
kind	Data type: String Default value: Password
password	Refresh token for Email server authentication. Data type: String Default value: None
userName	User name for Email server authentication. Data type: String Default value: None
refreshTokenExpiry	Expiry Date and Time in milli seconds. Data type: long Default value: None
isRefreshToken	Boolean value to determine the password field value as refresh token. Data type: Boolean Default value: None



Notes

- refreshToken is a mandatory field when you setup the Mailbox poller, refreshTokenExpiry is an optional value.
- For all types of OAuth authentication, classicoauth property has been deprecated and has been replaced with oauthDetails property.

19.2.15.3 MS Graph with Application type Authentication

```
email:
  emailBoxes:
    <emailBoxName>:
      profile: <no default value specified>
      triggersDefinitionXmlStoreFilePath: default_triggers
      automaticRestart: 'true'
      restartInProgress: 'false'
      instances: ''
  mailServerType: MSGRAPH
  msgraph:
    appId: <no default value specified>
    clientSecret: <no default value specified>
    tenantId: <no default value specified>
  profiles:
    <passwordProfile>:
      displayName: <no default value specified>
      emailAddress: <no default value specified>
      kind: PASSWORD
      userName: <no default value specified>
  proxy:
    enable: 'false'
    host: <no default value specified>
    password: <no default value specified>
    port: <no default value specified>
    user: <no default value specified>
  storage:
    dataSource: <no default value specified>
    pollerInterval: PT20S
    compressData: 'false'
    maxEmailsPerPollingCycle: '15'
    maxExecutionThreads: '5'
    maxNumberOfRows: '100'
-----
<MailConnector>:
  kind: com.eibus.applicationconnector.email.MailConnector
```

Property name	Description
mailServerType	Differentiates the Mail Server configuration type. For MS graph the value is MSGRAPH.
appId	Application ID of the Azure application.
clientSecret	Client Secret for accessing the Azure application.
tenantId	TenantID of the Azure application.
profiles	
<passwordProfile>	User defined passwordProfile name
kind	Data type: String Default value: Password
emailAddress	Email address of the user account for which to send emails. Data type: String Default value: None

Property name	Description
displayName	Name to use for the sender. Data type: String Default value: None
userName	User name for Email server authentication. Data type: String Default value: None
emailBoxes	Multiple mailbox configurations.
instances	Specifies the names of the emailboxes to be used by the poller logic. Data type: String[] Default value: None
<emailBoxName>	Specifies the name for an mailbox configuration.
triggersDefinitionXmlStoreFilePath	Path in XML store where the mailbox definition resides (/OpenText/EmailConnector/default_triggers.xml). Data type: String Default value: None
profile	Name of the profile to be used by the poller logic, as defined in the 'profiles' section. Data type: PasswordProfile Default value: None
automaticRestart	Holds whether when starting this thread, the database must be queried to automatically restart the containers that are in status INITIAL (and optionally IN_PROGRESS). Data type: Boolean Default value: true

Property name	Description
restartInProgress	Indicates if the containers in status IN_PROGRESS should be restarted. If a container was already in progress then it will be reprocessed. This means that the SOAP message is called twice depending on the exact moment the processor crashes. Data type: Boolean Default value: false
storage	Section to configure the email storage
dataSource	Name of the data source to use when connecting to the database. Data type: DataSourceConfiguration Default value: None
compressData	Whether the Email Connector should gzip the data before it is written to the database. This can limit the storage you need tremendously. Data type: Boolean Default value: false
pollerInterval	Interval to poll the database for new email messages to be processed. The value must be specified in the ISO-8601 format. Default is 10 seconds. Data type: Duration Default value: PT10S
maxEmailsPerPollingCycle	The maximum number of messages that should be processing in a single polling cycle. Data type: Integer Default value: 15
maxExecutionThreads	Maximum number of worker threads for the execution engine. Data type: Integer Default value: 5

Property name	Description
maxNumberOfRows	Maximum number of rows that will be read from the database per single polling cycle. Data type: Integer Default value: 100
<MailConnector>	Section to configure the Email connector.

19.2.15.4 MS Graph with Delegated type Authentication

```
email:
  mailServerType: MSGRAPH_DELEGATED
  oauthDetails:
    clientId: <no default value specified>
    clientSecret: <no default value specified>
    refreshTokenExpiryInDays: <no default value specified and non-mandatoryfield>
  storage:
    dataSource: <no default value specified>
    pollerInterval: 'PT10S'
  profiles:
    <Password profile>:
      displayName: <no default value specified>
      emailAddress: <no default value specified>
      kind: PASSWORD
      password: <no default value specified>
      userName: <no default value specified>
      refreshTokenExpiry: <no default value specified>
      isRefreshToken: <no default value specified>
```

Property name	Description
mailServerType	Differentiates the Mail Server configuration type. For MS graph with delegated authentication type. Value: MSGRAPH_DELEGATED
clientID	Client ID of the application. Data type: String Default value: None
clientSecret	Client Secret for accessing the Azure application. Data type: String Default value: None
refreshTokenExpiryInDays	Refresh token expiry (in days) value. Field is not mandatory. Data type: Integer Default value: None
profiles	

Property name	Description
<passwordProfile>	User defined passwordProfile name
kind	Data type: String Default value: Password
refreshTokenExpiry	Expiry Date and Time in milli seconds. Data type: long Default value: None
isRefreshToken	Boolean value to determine the password field value as refresh token. Data type: Boolean Default value: None
password	Refresh token for Email server authentication. Data type: String Default value: None
emailAddress	Email address of the user account for which to send emails. Data type: String Default value: None
displayName	Name to use for the sender. Data type: String Default value: None
userName	User name for Email server authentication. Data type: String Default value: None
emailBoxes	Multiple mailbox configurations.
instances	Specifies the names of the emailboxes to be used by the poller logic. Data type: String[] Default value: None
<emailBoxName>	Specifies the name for an mailbox configuration.

Property name	Description
triggersDefinitionXmlStoreFilePath	Path in XML store where the mailbox definition resides (/OpenText/EmailConnector/default_triggers.xml). Data type: String Default value: None
profile	Name of the profile to be used by the poller logic, as defined in the 'profiles' section. Data type: PasswordProfile Default value: None
automaticRestart	Holds whether when starting this thread, the database must be queried to automatically restart the containers that are in status INITIAL (and optionally IN_PROGRESS). Data type: Boolean Default value: true
restartInProgress	Indicates if the containers in status IN_PROGRESS should be restarted. If a container was already in progress then it will be reprocessed. This means that the SOAP message is called twice depending on the exact moment the processor crashes. Data type: Boolean Default value: false
storage	Section to configure the email storage
dataSource	Name of the data source to use when connecting to the database. Data type: DataSourceConfiguration Default value: None
compressData	Whether the Email Connector should gzip the data before it is written to the database. This can limit the storage you need tremendously. Data type: Boolean Default value: false

Property name	Description
pollerInterval	Interval to poll the database for new email messages to be processed. The value must be specified in the ISO-8601 format. Default is 10 seconds. Data type: Duration Default value: PT10S
maxEmailsPerPollingCycle	The maximum number of messages that should be processing in a single polling cycle. Data type: Integer Default value: 15
maxExecutionThreads	Maximum number of worker threads for the execution engine. Data type: Integer Default value: 5
maxNumberOfRows	Maximum number of rows that will be read from the database per single polling cycle. Data type: Integer Default value: 100
<MailConnector>	Section to configure the Email connector

The processed emails are deleted automatically, which might cause some data loss in case of unexpected failures. If you want to keep the processed emails, you can configure a target folder that moves the processed emails into the corresponding folders. This feature enables verification of the details in case of any failures. The target folder should exist and can be configured in the yml configuration file as follows:

```
email:
  emailBoxes:
    <emailBoxName>:
      profile: <no default value specified>
      triggersDefinitionXmlStoreFilePath: default_triggers
      targetFolder: <no default value specified>
  instances: ''
-----
```

If the specified target folder does not exist in the email box, the email service container will not start.



Note: As the POP3 protocol does not support accessing other folders, the target folder setting is not applicable in its configuration.

19.2.16 UDDI connector

The applicable properties that can be set for configuring the UDDI connector are provided in the sample configuration settings:

```
<UDDIConnector>:
  kind: com.opentext.applicationconnector.uddi.UDDIConnector
  connections:
    <uddiConnection>:
      url: <no default value specified>
      authenticationType: BASIC
      userName: ''
      password: ''
      checkCertificate: 'true'
      maxConnectionsPerHost: '10'
      maxTotalConnections: '100'
      proxy:
        server: ''
        port: <no default value specified>
        authenticationType: BASIC
        userName: <no default value specified>
        password: ''
      responseTimeout: PT30S
      webServiceInterfaces:
        <webServiceInterface>:
          roleToConnectionMapping:
            <role>:
              connection: <no default value specified>
```

To work with the UDDI connector in a container environment, create a UDDI service container through yml by providing the details shown in the sample:

```
config_ordinal: 210
platform:
  configuration:
    pollInterval: PT30S
  organizations:
    OT:
      services:
        instances: UDDIService
        UDDIService:
          kind: com.eibus.applicationconnector.uddi.UDDIConnector
          packages:
            "My Company UDDI_Proj":
              methodSets: "Add, Divide, Multiply, Subtract"
            "My Company Test":
              methodSets: "DelayTest"
          connections:
            calculator:
              authenticationType: ANONYMOUS
              userName: ''
              password: ''
              url: 'http://www.dneonline.com/calculator.asmx'
            DelayTest:
              authenticationType: BASIC
              userName: 'username'
              password: 'password'
              url: '*****'
          maxConnectionsPerHost: '10'
          maxTotalConnections: '100'
          proxy:
            authenticationType: ANONYMOUS
            password: ''
            port: ''
            server: ''
            userName: ''
          responseTimeout: PT30S
          webServiceInterfaces:
```

```

Calc.CalculatorSoap:
  roleToConnectionMapping:
    TestRole:
      connection: calculator
testweb.Package2WebServiceInterface:
  roleToConnectionMapping:
    TestRole:
      connection: DelayTest

```

The following table describes various property names, their data types, the default values they accept, and their descriptions.

Property name	Description
<UDDIConnector>	Section to configure the UDDI connector configuration.
webServiceInterfaces	Section to configure the customization of WebServiceInterfaces handled by the UDDI connector.
<webServiceInterface>	Section to configure the endpoint and user credentials for the WebServiceInterface. Expected format: <package name>#<wsi name> - For published projects and the interfaces in a non-sytem organization, provide only the <wsi name>. - No need to give <package name> as prefix. Just <wsi name> is enough.
roleToConnectionMapping	
<role>	Name of the role for which to configure the wWebServiceInterface.Role format: <package name>#<role name>; for published project just <role name>
connection	Name of the connection, as defined below, to use for the role mapping. Data type: UDDIConnectorConnection Default value: None
connections	Section to configure the connections that can be used in the role mapping above.
<uddiConnection>	Section per connection configuration.
url	URL to use when connecting to the external web-service. Data type: String Default value: None

Property name	Description
authenticationType	Authentication type to use when authenticating at the external service. Following are supported: ANONYMOUS, BASIC, DIGEST and NTLM. Data type: UDDIAuthenticationType Default value: BASIC
userName	User name for the external service authentication. Data type: String Default value: None
password	Password to use for the external service authentication. Data type: Password Default value: None
checkCertificate	Whether to validate the HTTPS certificates of the external service. Data type: Boolean Default value: true
maxConnectionsPerHost	Maximum number of connections the UDDI connector will allow per single external service host. Data type: Integer Default value: 10
maxTotalConnections	Maximum number of connections the UDDI connector will allow for all external service hosts. Data type: Integer Default value: 100
proxy	Configures the proxy server to use when connecting to external services.
server	Host name of the proxy server to use. Data type: String Default value: None

Property name	Description
port	Port where the proxy server listens for connections. Data type: Integer Default value: None
authenticationType	Authentication type to use when authenticating at the proxy server. The following servers are supported: ANONYMOUS, BASIC, DIGEST and NTLM. Data type: UDDIAuthenticationType Default value: BASIC
userName	User name for proxy server authentication. Data type: String Default value: None
password	Password for proxy server authentication. Data type: Password Default value: None
responseTimeout	Time the UDDI connector will wait for a response from the external service before timing out and returning an error. Data type: Duration Default value: PT30S

19.2.17 HTTP connector

The applicable properties that can be set for configuring the HTTP connector are provided in the sample configuration settings:

```
<HttpConnector>:
  kind: com.opentext.applicationconnector.httpconnector.HttpConnector
  connections:
    <httpConnection>:
      url: <no default value specified>
      authenticationType: BASIC
      userName: ''
      password: ''
      authenticateAlways: 'false'
      checkCertificate: 'true'
      maxConnectionsPerHost: '0'
      maxTotalConnections: '0'
      parameters:
        <parameterName>: <no default value specified>
      responseTimeout: PT30S
      useProxy: 'false'
  proxy:
```

```
server: <no default value specified>
port: <no default value specified>
authenticationType: BASIC
userName: <no default value specified>
password: <no default value specified>
```

The HTTP connector also supports the OAuth authentication. The applicable properties that can be set for configuring the OAuth in the HTTP connector are provided in the sample configuration settings:

```
<HttpConnector>:
  kind: com.opentext.applicationconnector.httpconnector.HttpConnector
  connections:
    <httpConnection>:
      url: ' http://server.example.com/oauth2/userinfo '
      authenticationType: OAuth
      authenticateAlways: false
      checkCertificate: false
      maxConnectionsPerHost: 0
      maxTotalConnections: 0
      responseTimeout: PT30S
      useProxy: false
      clientId: 'testclient'
      clientSecret: 't81U5wQ0J3ciI3mufLhoUQ0Q9hmmZ4L9'
      authenticationUrl: ' http://server.example.com/oauth2/token '
      scope: 'search'
  proxy:
    authenticationType: BASIC
    password: <no default value specified>
    port: <no default value specified>
    server: <no default value specified>
    userName: <no default value specified>
```

To work with the HTTP connector in a container environment, create a HTTP service container through yml by providing the details shown in the sample:

```
config_ordinal: 210
platform:
  configuration:
    pollInterval: PT30S
  organizations:
    Org1:
      services:
        instances: HttpConnector
        HttpConnector:
          kind: com.opentext.applicationconnector.httpconnector.HttpConnector
          connections:
            GITHUB:
              url: 'https://api.publicapis.org'
            GETLABEL:
              url: 'https://www.googleapis.com'
              authenticateAlways: true
              checkCertificate: false
              userName: 'username@gmail.com'
              password: 'password'
            OAUTH_CONTAINER:
              url: 'http://otds/otdsws/oauth2/userinfo'
              authenticationType: OAuth
              clientId: 'TestOAuth'
              clientSecret: 'your clientSecret'
              authenticationUrl: 'http://otds/otdsws/oauth2/token'
              scope: 'search'
            Basic_Auth:
              url: 'https://httpbin.org'
              authenticationType: BASIC
              userName: 'username'
              password: 'password'
          proxy:
```

```

authenticationType: ANONYMOUS
password: ''
port: ''
server: ''
userName: ''

```

The following table describes various property names, their data types, the default values they accept, and their descriptions.

Property name	Description
<HttpConnector>	Section to configure the HTTP connector configuration.
connections	Section to configure connections for external services.
<httpConnection>	Section per connection for which to configure they endpoint and user credentials. The connection id is used in the connection-id element in the web-service method implementation.
url	The URL to use when connecting to the external web-service. Data type: String Default value: None
authenticationType	Authentication type to use when authenticating at the external service. The following are supported: BASIC, NTLM , ANONYMOUS, and OAUTH. Data type: HttpAuthenticationType Default value: BASIC
userName	User name for the external service authentication. Data type: String Default value: None
password	Password for the external service authentication. Data type: Password Default value: None

Property name	Description
authenticateAlways	Whether to use 'Preemptive Authentication'. If set to 'true' then the HTTP connector will send authentication information with every request even before the server asks for it. Only applicable when using authentication type BASIC. Data type: Boolean Default value: false
checkCertificate	Whether to validate the HTTPS certificates of the external service. Data type: Boolean Default value: true
maxConnectionsPerHost	Maximum number of connections the HTTP connector will allow per single external service host. Data type: Integer Default value: 0
maxTotalConnections	Maximum number of connections the HTTP connector will allow for all external service hosts. Data type: Integer Default value: 0
parameters	Section to configure uri parameters.
<parameterName>	Name of the uri-parameter to configure. Data type: String Default value: None
responseTimeout	Time the HTTP connector will wait for a response from the external service before timing out and returning an error. Data type: Duration Default value: PT30S
useProxy	Whether to use the configured proxy for this connection. Data type: Boolean Default value: false

Property name	Description
proxy	Configures the proxy server to use when connecting to external services.
server	Host name of the proxy server to use. Data type: String Default value: None
port	Port where the proxy server listens for connections. Data type: Integer Default value: None
authenticationType	Authentication type to use when authenticating at the proxy server. Following are supported: BASIC, NTLM, and ANONYMOUS. Data type: HttpAuthenticationType Default value: BASICs
userName	User name for proxy server authentication. Data type: String Default value: None
password	Password for proxy server authentication. Data type: Password Default value: None
clientId	OAuth Client ID. Only required if the authentication type is OAuth. Data type: String
clientSecret	The OAuth Client ID secret key value. Only required if the authentication type is OAuth. If you want to make the Client Secret a secret, you can configure it through the Vault. Data type: String
authenticationUrl	The OAuth Authentication URL to get the access token. Only required if the authentication type is OAuth. Data type: String

Property name	Description
scope	The application scope issued for the corresponding access token. Data type: String Default value: None

19.2.18 Magellan/iHub

The applicable properties that can be set for Magellan in a specific organization in the MPC file are provided in the following sample configuration settings:



Note: The following configuration repeats for every organization.

```
Platform:
#Magellan is fixed tag used for iHub Service container configuration.
magellan:
  iportalUrl: http://<Magellan server hostname>:<iportal port>/
  restUrl: http://<Magellan server hostname>:<Magellan REST client port>/
  volume: Default Volume
  authType: <no default value specified>
  clientId: <no default value specified>
  clientSecret: <no default value specified>
```

The following table describes various property names, their data types, the default values they accept, and their descriptions.

Property name	Description
iportalUrl	Iportal URL of the iHub/Magellan server. Data type: String Default value: The default protocol for the iportalUrl is HTTPS and listens on port 8701. However, if you want to use the HTTP protocol, provide port as 8700.
restUrl	Rest URL of the iHub/Magellan server. Data type: String Default value: The default protocol for the restUrl is HTTPS and listens on port 8001. However, if you want to use the HTTP protocol, provide port as 8000.
volume	iHub/Magellan server volume name where your organization related reports are deployed. Data type: String Default value: Default volume

Property name	Description
authType	Authentication type of the integration. OTDS \oAuth or RSSE. Data type: String
clientId	iHub Client ID from OTDS Server. Only required if authType is oAuth. Data type: String
clientSecret	iHub Client secret from OTDS Server. Only required if authType is oAuth. Data type: String

19.2.19 Audit

The configuration exists only at the Shared level. Any configurations provided at the Organizational level are not considered.

```
# Used for shared configuration
Platform:
  #For Audit service container configuration. it is a fixed tag
  auditing:

    #Name of the datasource from the configuration file
    connection: Connection name

  #Database configuration. it is a fixed tag
  database:

    #Data source configuration. it is a fixed tag
    dataSources:

      #<Should be equal to the provided data source Value in Audit configuration>
      <Can be one or multiple>. Replace tag with your data source name.
      DataSourceName:

        #Name of the database connection from the configuration file
        connection: Connection name

      # Database connections. it is a fixed tag
      connections:

        #<Should be equal to the provided data base connection Value in Data source>
        <Can be one or multiple>. Replace tag with your database Connection name.
        Connectionname:

          #The connection string to establish the connection with DB.
          connectionString: SQL - jdbc:sqlserver://<Host
name>:<Portno>;sendStringParametersAsUnicode=false or Postgres - jdbc:postgresql://
<Hostname>:<Port no>/<databasename> or Oracle - jdbc:oracle:thin:@<Host name>:<port
no>:<SID name>

          #Name of the database to establish the connection
          defaultDatabase: Name of the database

          #Name of the driver based on the database type we use
          driverName: JDBC for SQL,postgres & OJDBC for Oracle

          #JDBC Driver name
          jdbcDriver: com.microsoft.sqlserver.jdbc.SQLServerDriver - for SQL,
org.postgresql.Driver - For Postgres, oracle.jdbc.driver.OracleDriver - For Oracle
```

```

#JDBC driver XA Class name
jdbcDriverXAClass: com.microsoft.sqlserver.jdbc.SQLServerXADataSource -
For SQL, org.postgresql.xa.PGXADatasource - For Postgres,
Oracle.jdbc.xa.client.OracleXADataSource - For Oracle

#Password for connecting to the database
password: password

#User name for connecting to the database
userName: user name

```

Property name	Description
connection	Name of the datasource from the configuration file. Data type: String Default value: Not applicable

19.2.20 Security Administration

The applicable properties that can be set for administering security in a shared organization are provided in the following sample configuration settings:

```

# Used for shared configuration
Platform:

#For Sec Admin Service container configuration. As Sec Admin is deployed by default
with following Name this Tag will be fixed.
Security Administration:

#Name of the XDS Connection from the configuration file.Sec Admin Uses XDS for
connection. XDS Uses Database for connection.
xds: XDS Connection name

#Name of the XDS Connection. Fixed Tag
xds:

#<Should be equal to the provided XDS Value in SecAdmin configuration>. Replace
tag with your XDS Connection name.
xds connection name:

#XDS in-memory cache size
cacheSize: 9876

#max size of the compressed content of a specific document
compressionContentSize: 8543

#Name of the Datasource from the configuration file
dataSource: Data Source Connection name

#while storing the XML into database do we want the content to be compressed
or not
enableCompression: true\false

#Database configuration. Fixed tag
database:

#Data source Configuration. Fixed tag
dataSources:

#<Should be equal to the provided data source Value in Audit configuration>
<Can be one or multiple>. Replace tag with your data source name.
DataSourceName:

```

```

        #Name of the Database connection from the configuration file
        connection: Connection name

        # Database connections. Fixed tag
        connections:

            #<Should be equal to the provided data base connection Value in Data source>
            <Can be one or multiple>. Replace tag with your database Connection name.
            Connectionname:

                #The connection string to establish the connection with DB.
                connectionString: SQL - jdbc:sqlserver://<Host
                name>:<Portno>;sendStringParametersAsUnicode=false or Postgres - jdbc:postgresql://
                <Hostname>:<Port no>/<databasename> or Oracle - jdbc:oracle:thin:@<Host name>:<port
                no>:<SID name>

                #Name of the database to establish the connection
                defaultDatabase: Name of the database

                #Name of the driver based on the database type we use
                driverName: JDBC for SQL,postgres & OJDBC for Oracle

                #JDBC Driver name
                jdbcDriver: com.microsoft.sqlserver.jdbc.SQLServerDriver – for SQL,
                org.postgresql.Driver – For Postgres, oracle.jdbc.driver.OracleDriver - For Oracle

                #JDBC driver XA Class name
                jdbcDriverXAclass: com.microsoft.sqlserver.jdbc.SQLServerXADataSource –
                For SQL, org.postgresql.xa.PGXDataSource - For Postgres,
                Oracle.jdbc.xa.client.OracleXADataSource - For Oracle

                #Password for connecting to the database
                password: password

                #User name for connecting to the database
                userName: user name

```

The following table describes various property names, their data types, the default values they accept, and their descriptions.

Property name	Description
xds	Name of the XDS Connection from the configuration file. The Sec Admin service uses XDS for connection. XDS Uses Database for connection. Data type: String Default value: Not applicable

19.2.21 XForms Connector

The applicable properties that can be set for the XForms connector are provided in the following sample configuration settings:

```
platform:
  XForms:
    cacheSize: '200'
    expiryOption: minutes
    expiryType: afterTime
    expiryValue: 10
```

The following table describes various property names, their data types, the default values they accept, and their descriptions.

Property	Description
expiryValue	Expiry value with number value as mentioned in the Type. Data type: Number (if date, value should be in milliseconds) Default value: Not applicable
expiryType	Expiry type with any of the mentioned entries such as Immediately, Everytime, After_time, On_date. Data type: (one of these) immediately, everytime, afterTime, onDate Default value: Not applicable
expiryOption	Expiry options with any of the mentioned entries such as day, minute, and hours. Data type: (one of these) days, minutes, hours Default value: Not applicable
cacheSize	Size of the cache. Data type: String Default value: Not applicable

19.2.22 Google Maps

The applicable properties that can be set for configuring google maps are provided in the sample configuration settings:

```
platform:
  googleMapsKeys:
    "https://other.company.com:80":
      key: llsdfCiiqw0IS-Mijsi23jsd123123j0sdfj9j-U2H
    "https://other.company1.com:80":
      key: aqwqfCii0IS-Mijsi23jsdf123213dfj9j-2IH
```

The following table describes various property names, their data types, the default values they accept, and their descriptions.

Property name	Description
Google map Domain	Domain and port number in the property. For example: http://other.company.com:8080 Data type: String Default value: Not applicable
Key	Create and provide the Google Map Project Key. For example: llsdfCii0IS-Mijsi23jsdf089uj0sdfj9j-UIH. Data type: String Default value: Not applicable

19.2.23 AppWorks Gateway(Mobile App)

The applicable properties that can be set for configuring AppWorks Gateway(Mobile App) are provided in the sample configuration settings:

```
platform:
  #appworksgateway is fixed tag used for AppWorks Gateway configuration.
  appworksgateway:
    #Url of AppWorks Gateway server
    url: http://trism10.lab.opentext.com:8080/
    #Admin User Name of AppWorks Gateway server
    username: otag
    #Admin Password of AppWorks Gateway server
    password: otag
```

The following table describes various property names, their data types, the default values they accept, and their descriptions.

Property name	Description
url	URL of AppWorks Gateway server. For example: <code>http://myappworksgateway:8080</code> Data type: String Default value: Not applicable
username	Administrator user name of the AppWorks Gateway server. For example: <code>otag</code> Data type: String Default value: Not applicable
password	Administrator password of the AppWorks Gateway server. For example: <code>otag</code> Data type: String Default value: Not applicable

19.2.24 XDS

The applicable properties that can be set for the XDS repository are provided in the following sample configuration settings:

```
platform:
  xds:
    <xds name>:
      cacheSize: '10000'
      compressionContentSize: 500
      dataSource: system
      enableCompression: 'true'
```

The following table describes various property names, their data types, the default values they accept, and their descriptions.

Property name	Description
cacheSize	Document cache size for every connector in repository. Data type: Long Default value: 5000

Property name	Description
dataSource	Name of the platform database configuration/datasource. Data type: Not applicable Default value: Not applicable
enableCompression	Switch to turn compression ON or OFF. Data type: String Default value: false
compressionContentSize	Size in KB, above which compression is applicable, in XDS for every connector. This is applicable when enableCompression is set as ON. Data type: Integer Default value: 100

19.2.25 Application package

The applicable properties that can be set for configuring application packages are provided in the following sample configuration settings:

```
platform:
  services:
    <CAP service name>:
      kind: CAP
      xds: system
```

The following table describes various property names, their data types, the default values they accept, and their descriptions.

Property name	Description
xds	Name of XDS defined in the configuration. Data type: String Default value: None

19.2.26 XML Store

The applicable properties that can be set for configuring XML Store are provided in the following sample configuration settings:

```
platform:
  services:
    <XMLStore service name>:
      kind: XMLStore
      xds: system
```

The following table describes various property names, their data types, the default values they accept, and their descriptions.

Property name	Description
xds	Name of XDS defined in the configuration. Data type: String Default value: None

19.2.27 Tag

The applicable properties that can be set for configuring Tag are provided in the following sample configuration settings:

```
platform:
  services:
    <Tag service name>:
      kind: Tag
      xds: system
```

The following table describes various property names, their data types, the default values they accept, and their descriptions.

Property name	Description
xds	Name of XDS defined in the configuration. Data type: Integer Default value: 100

19.2.28 Task

The applicable properties that can be set for configuring Task are provided in the following sample configuration settings:

```
platform:
  services:
    <Task service name>:
      kind: Task
      xds: system
```

The following table describes various property names, their data types, the default values they accept, and their descriptions.

Property name	Description
xds	Name of XDS defined in the configuration. Data type: String Default value: None

19.2.29 Preference

The applicable properties that can be set for configuring Preferences are provided in the following sample configuration settings:

```
platform:
  services:
    <Preference service name>:
      kind: Preference
      xds: system
```

The following table describes various property names, their data types, the default values they accept, and their descriptions.

Property name	Description
xds	Name of XDS defined in the configuration. Data type: String Default value: None

19.2.30 WS AppServer

The applicable properties that can be set for configuring XML Store are provided in the following sample configuration settings:

```
platform:
  services:
    <WsAppServer service name>:
      kind: WsAppServer
      applicationInitializerClassName: ''
      dataSource: <application data source name>
      initializerClassName: ''
      enableAutoCleanup: 'false'
      enableRules: 'false'
      initializeDatabase: 'true'
      multiTenant: 'false'
      ruleConfiguration: system
```

The following table describes various property names, their data types, the default values they accept, and their descriptions.

Property name	Description
initializerClassName	Class name Data type: String Default value: None
applicationInitializerClassName	Class name Data type: String Default value: None
enableAutoCleanup	Data type: Boolean Default value: false
ruleConfig	Name of the rule configuration. Data type: String Default value: None
initializeDatabase	Data type: Boolean Default value: true
multiTenant	Data type: Boolean Default value: false
dataSource	Name of the application database configuration/datasource. Data type: String Default value: None

19.2.31 Email Building Block

The applicable properties that can be set for configuring Email Building Block are provided in the following sample configuration settings:

```
platform:
  entityRuntime:
    solutions:
      <Project name>:
        entities:
          <Entity Name>:
            email:
              attachmentRepository: Business workspace
```

The following table describes various property names, their data types, the default values they accept, and their descriptions.

Property name	Description
attachmentRepository	Repository for the email attachments. Accepted values: • Content Server • Business workspace • None Data type: String Default value: Business workspace  Note: If any other value is provided, an error message is displayed when you click the attachments icon.

19.2.32 File Building Block

The applicable properties that can be set for configuring File Building Block are provided in the following sample configuration settings:

```
platform:
  entityRuntime:
    solutions:
      <Project name>:
        entities:
          <Entity Name>:
            file:
              fileUploadRestrictionType: 2
              maxFileSize: 1024000
              restrictedFileTypes: .exe,.bin,.bat
```

The following table describes various property names, their data types, the default values they accept, and their descriptions.

Property name	Description
fileUploadRestrictionType	This is to allow or control the type of documents that can be uploaded by the end user. 0 - Accept all formats (No restriction) 1 - Accept specified formats 2 - Do not accept specified formats Data type: Integer Default value: value given in entity
maxFileSize	Allowed maximum file size when user uploads documents. Data type: String Default value: value given in entity
restrictedFileTypes	This is related to the fileUploadRestrictionType property. If the fileUploadRestrictionType value is 1 (Accept specified formats) or 2 (Do not accept specified formats), then extensions of such files which are to be restricted for upload can be provided as a comma separated list. Data type: String Default value: value given in entity
documentTypeConfiguration	This is specific to Documentum and the value is a json string , where administrators can provide the configuration for metadata type(or document type) and its attributes available for the end user. Data type: String Default value: None

19.2.33 Content Building Block

The applicable properties that can be set for configuring Content Building Block are provided in the following sample configuration settings:

```
platform:
  entityRuntime:
    solutions:
      <Project name>:
        entities:
          <Entity Name>:
            content:
              maxNumberOfFilesUploaded: 50
            file:
              fileUploadRestrictionType: 1
              maxFileSize: 10240000
              restrictedFileTypes: .pdf,.docx
              documentTypeConfiguration : "{ \"MetadataConfiguration\":
{ \"Configuration\": [{ \"Type\": \"dm_employee\", \"Filter\": [{ \"PropertyList\": [ \"name\",
\"role\", \"address\" ], \"ReadOnlyPropertyList\": [ \"employeeid\" ] } ] } ] } }
```

The following table describes various property names, their data types, the default values they accept, and their descriptions.

Property name	Description
maxNumberOfFilesUploaded	Maximum number of files that can be uploaded for a given entity instance. Data type: String Default value: Value given in entity
file:	
fileUploadRestrictionType	This is to allow or control the type of documents that can be uploaded by the end user 0 - Accept all formats (No restriction) 1 - Accept specified formats 2 - Do not accept specified formats Data type: String Default value: value given in entity
maxFileSize	Allowed maximum file size when user uploads documents. Data type: String Default value: Value given in entity

Property name	Description
restrictedFileTypes	This is related to fileUploadRestrictionType property. If the fileUploadRestrictionType value is 1 (Accept specified formats) or 2(Do not accept specified formats), then extensions of such files which are to restricted for upload can be provided as a comma separated list. Data type: String Default value: Value given in entity
documentTypeConfiguration	This is specific to Documentum and the value is a json string , where administrators can provide the configuration for metadata type(or document type) and its attributes available for the end user. Data type: String Default value: None

19.2.34 Business Workspace

The applicable properties that can be set for configuring Business Workspace are provided in the following sample configuration settings:

```
platform:
  entityRuntime:
    solutions:
      <Project name>:
        entities:
          <Entity Name>:
            businessWorkspace:
              crossAppType: Shared
              externalSystemId: MyLeadApp
              workspaceTypeId : 5
              searchExpression: Item.Properties.EmployeeName='Name'
```

The following table describes various property names, their data types, the default values they accept, and their descriptions.

Property name	Description
crossAppType	This is required when an entity application has to work with another business application which creates the workspace or if the entity instance has to link with an existing workspace. The values can be Shared, Related, and Link. Data type: String Default value: None

Property name	Description
externalSystemID	ID of the system/application configured in CS. Applicable only when crossAppType is Shared or Related. Data type: String Default value: None
businessObjectType	Name of the business object configured in the CS corresponding to the entity. Applicable only when crossAppType is Shared or related. Data type: String Default value: None
workspaceTypeID	ID of the workspace type created in CS . Applicable only when crossAppType is Shared or related. Data type: String Default value: None
searchExpression	Search string, which can be an expression consisting of static and dynamic(using entity properties) strings. This search expression is evaluated and will be used as a search string to search for matching workspaces in CS. Data type: String Default value: None
linkBWSSearchExpression	Search string, which can be an expression consisting of static and dynamic(using entity properties) strings. This search expression will be evaluated and will be used as a search string to search for matching workspaces in CS. Data type: String Default value: None
linkBWSWorkspaceTypeID	ID of the workspace type created in CS. Applicable only when crossAppType is Link. Data type: String Default value: None

Property name	Description
createBWSWithAutoSync	Applicable only when crossAppType is Link. This determines whether a new workspace needs to be created in CS, when there is no matching workspace exists. If the value is true, and search returns no matching workspace, a new workspace is automatically. If the value is false, you will see a popup where you can manually create workspace. Data type: Boolean Default value: None

19.2.35 instanceRoleToTeamRoleMap

```
entityRuntime:
  solutions:
    <Project name>:
      entities:
        <Entity Name>:
          businessWorkspace:
            crossAppType: Shared
            externalSystemId: MyLeadApp
            workspaceTypeId : 5
            searchExpression: Item.Properties.EmployeeName='Name'
            instanceRoleToTeamRoleMap: "[{\\"instanceRole\\":\\"InstanceRole_SalesManager\\",\\"teamRole\\":\\"TeamRole_SalesManager\\"},{\\"instanceRole\\":\\"InstanceRole_SalesRep\\",\\"teamRole\\":\\"TeamRole_SalesRep\\"}]"
```

Property name	Description
instanceRoleToTeamRoleMap	The value is a json string , where administrators can provide the mapping between instance roles and team roles of business workspace. Data type: String Default value: None

19.2.36 Authenticator

The applicable properties that can be set for the Authenticator are provided in the sample configuration settings:

```
authenticators:
  defaultLogin: <no default value specified>
  defaultSigner: <no default value specified>
  <oauthAuthenticator>:
    kind: OAuth
    clientId: <no default value specified>
    publicLoginUrl: ${platform.otds.url}/otds/ws/login
  <otdsAuthenticator>:
    kind: OTDS
    publicLoginUrl: ${platform.otds.url}/otds/ws/login
  <platformAuthenticator>:
```

```
kind: Platform
loginPath: /wcp/sso/login.htm
```

The following table describes various property names, their data types, the default values they accept, and their descriptions.

Property name	Description
authenticators	Section to configure authentication authenticators.
defaultLogin	ID of the authenticator to use when no authID is specified on the URL. Data type: String Default value: None
defaultSigner	ID of the authenticator to use when no OAuth signer is specified on the URL. Data type: String Default value: None
<oauthAuthenticator>	OAuth authenticator specific configuration.
kind	Enter 'OAuth' to provide an OAuth authenticator. Data type: String Default value: OAuth
clientId	OAuth client ID as specified in OTDS. Data type: String Default value: None
publicLoginUrl	OTDS server URL to be used by end-users. This step is only required when the OTDS URL differs from the URL by which AppWorks Platform communicates internally with OTDS, for example in an SSL offloader or (reverse) proxy scenario. For example: https://public.otds.company.com/otdsws/login Data type: String Default value: \${platform.otds.url}/otdsws/login
<otdsAuthenticator>	OTDS authenticator specific configuration.

Property name	Description
kind	Specify 'OTDS' to specify an OAuth authenticator. Data type: String Default value: None
publicLoginUrl	OTDS server URL to be used by end-users. This step is only required when the OTDS URL differs from the URL by which AppWorks Platform communicates internally with OTDS, for example in an SSL offloader or (reverse) proxy scenario. For example: https://public.otds.company.com/otdsws/login Data type: String Default value: \${platform.otds.url}/otdsws/login
<platformAuthenticator>	Platform authenticator specific configuration.
kind	Specify 'Platform' to specify an OAuth authenticator. Data type: String Default value: Platform
loginPath	Allows to configure a custom login page for Platform authentication. Data type: String Default value: /wcp/sso/login.htm

19.2.37 Gateway

The applicable properties that can be set for configuring the gateway are provided in the sample configuration settings:

```
gateway:
  requestValidation: 'false'
  responseValidation: 'false'
```

The following table describes various property names, their data types, the default values they accept, and their descriptions.

Property name	Description
gateway	Section to configure the Web Gateway.

Property name	Description
requestValidation	Validate the request and the web service WSDL definition. Data type: Boolean Default value: false
responseValidation	Validate the response and the web service WSDL definition. Data type: Boolean Default value: false

19.2.38 Licensing

The applicable properties that can be set for configuring Licensing are provided in the sample configuration settings:

```
licensing:
  customerName: <no default value specified>
  licenseKey: <no default value specified>
  siteName: <no default value specified>
  sitePassword: <no default value specified>
```

The following table describes various property names, their data types, the default values they accept, and their descriptions.

Property name	Description
licensing	Section to configure licensing.
customerName	Customer name provided by OpenText for the AppWorks Platform license. Data type: String Default value: None
licenseKey	Product license key provided by OpenText for this AppWorks Platform deployment. When not specified, the deployment tries to retrieve a license key based on the customer name, site name, site password, and deployment details. Data type: String Default value: None
siteName	Site name provided by OpenText for the AppWorks Platform license. Data type: String Default value: None

Property name	Description
sitePassword	Site password provided by OpenText for the AppWorks Platform license. Data type: Password Default value: None

19.2.39 OTDS

The applicable properties that can be set for configuring the OTDS are provided in the sample configuration settings:

```
otds:
  resourceId: <no default value specified>
  url: <no default value specified>

organizations:
  <organization>:
    otds:
      rolesPartition: <no default value specified>
```

The following table describes various property names, their data types, the default values they accept, and their descriptions.

Property name	Description
otds	Section to configure the OTDS integration.
resourceId	ID of the resource in OTDS used by this AppWorks Platform environment. When not yet activated, AppWorks tries to activate this resource. Data type: String Default value: None
url	URL of the OTDS server to use for this AppWorks Platform environment. Data type: String Default value: None

Property name	Description
rolesPartition	Name of the OTDS partition to push application roles from AppWorks Platform to OTDS. If the partition does not exist in OTDS, a non-synchronized user partition is created.  Note: This is an organization level property only. Data type: String Default value: None

19.2.40 Security

The applicable properties that can be set for configuring security are provided in the sample configuration settings:

```
platform:
  security:
    keyStore:
      inherit: 'true'
      keyPassword: ${platform.security.keyStore.storePassword:-}
      location: ''
      storePassword: ''
    trustStore:
      inherit: ${platform.security.keyStore.inherit:-true}
      location: ${platform.security.keyStore.location:-}
      storePassword: ${platform.security.keyStore.storePassword:-}

  security:
    systemAdministratorPassword: <no default value specified>

  security:
    concurrentSessions:
      adminRoles: systemAdmin, setupUser, Cordys@Work#Administrator
      adminLimit: '3'
      enabled: false
      userLimit: '2'

  security:
    authentication:
      httpOnlyCookie: false

  security:
    platformAuthentication
      enabled: 'true'
      allowedSystemToSystemUsers: <no default value specified>
```

The following table describes various property names, their data types, the default values they accept, and their descriptions.

Property name	Description
security	Section to configure the OTDS integration.

Property name	Description
keyStore	Section to configure the key store. The key store is used to store key and can contain certificates.
inherit	Whether to include certificates from the java cacert . jks file. Data type: Boolean Default value: true
keyPassword	Passwords to access the key store. Data type: Password Default value: None
location	File location of the key store. Data type: String Default value: None
storePassword	Passwords to access the trust store. Data type: Password Default value: None
trustStore	Section to configure the trust store. The trust store can contain certificates.
inherit	Whether to include certificates from the java cacert . jks file. Data type: Boolean Default value: None
location	File location of the trust store. Data type: String Default value: None
storePassword	Passwords to access the trust store. Data type: Password Default value: None
systemAdministratorPassword	Password of the 'sysadmin' administration. Data type: Password Default value: None

Property name	Description
concurrentSessions	Section to configure the number of sessions users are allowed to access concurrently. Enabling <code>concurrentSessions</code> impacts the validity of the security token. When a user logs out of AppWorks, the SAMLart security token invalidates it so that the token cannot be used anymore. Based on your requirement, you can increase the <code>adminLimit</code> and <code>userLimit</code> values to allow concurrent access.
adminRoles	Role that determines the user as an administrator. Data type: String Default value: <code>systemAdmin, setupUser, Cordys@Work#Administrator</code>
adminLimit	Number of sessions the user, with any one of the admin roles specified above, is allowed to access concurrently. Data type: Integer Default value: 5
enabled	Whether to count the concurrent sessions. Data type: Boolean Default value: <code>true</code>
userLimit	Number of sessions a user, without any of the admin roles specified above, is allowed to access concurrently. Data type: Integer Default value: 5
authentication	Section to configure the authentication.
httpOnlyCookie	HttpOnly flag setting of cookie. Data type: Boolean Default value: <code>false</code>
platformAuthentication	Section to configure AppWorks Platform-specific authentication.

Property name	Description
enabled	Whether AppWorks Platform authentication is enabled. When not enabled, users cannot log in through the AppWorks Platform login user interface and cannot authenticate through the authentication web service. Data type: Boolean Default value: true
allowedSystemToSystemUsers	Comma-separated list of user IDs. Users in this list are allowed to authenticate through the authentication web service even when AppWorks Platform authentication is not enabled. This is useful for system to system integrations such as the user used in the OTDS Push connector configuration.

19.2.41 Application package deployment

The applicable properties that can be set for configuring application package deployment are provided in the sample configuration settings:

```
package:
  security:
    loadUnsignedCaps: 'false'

organizations:
  <organization>:
    package:
      deployList: ''
```

The following table describes various property names, their data types, the default values they accept, and their descriptions.

Property name	Description
package	Section for CAP package configuration.
security	Section to configuration security for CAP packages.
loadUnsignedCaps	Whether to allow the deployment of unsigned CAP packages. Data type: Boolean Default value: false

Property name	Description
deployList	List of application packages to deploy in the specific organization. This is an organization level property only. Data type: String Default value: true

19.2.42 Auto logout

The auto logout feature enables you to automatically log out the user after a certain period of inactivity. If the period is defined as 15 minutes and if a user does not move the mouse or press any key on the keyboard within the AppWorks Platform application, the user is logged out automatically. The applicable properties that can be set for configuring the auto logout feature are provided in the sample configuration settings:

```
platform:
  ui:
    autoLogout:
      logoutTime:PT15M

platform:
  ui:
    autoLogout:
      enabled:true
```

The following table describes various property names, their data types, the default values they accept, and their descriptions.

Property name	Description
appWorksClient	Section to configure the application's UI preferences.
autoLogout	Section to configure the application's auto logout time.
LogoutTime	Set the auto logout time as required for your application. Data type: Duration Default value: PT15M The accepted values are H and M indicating Hours and Minutes respectively. For example, PT2H10M, indicates auto logout time of 130 minutes.
enabled	The auto logout feature is turned off by default. Set enabled to true to enable the auto logout feature.

19.2.43 Configuring error behavior in item layouts

By default, item layouts are configured to display errors on load. This setting can be changed to display errors on save. To do this, below is the sample configuration settings:

```
platform:
  ui:
    application:
      error:
        onload:false
```

19.2.44 Auditing

The applicable properties that can be set for configuring auditing are provided in the sample configuration settings:

```
auditing:
  dataSource: system
  <auditComponent>: 'false'
```

The following table describes various property names, their data types, the default values they accept, and their descriptions.

Property name	Description
auditing	Section to configure auditing.
dataSource	Name of the data source used for auditing. Data type: String Default value: System

Property name	Description
<auditComponent>	Entry per audit component for which auditing must be enabled. The following audit components can be used (use the exact name, including spaces): • Authentication • BPM • BPM Configuration • Business CalendarCAP (by default enabled) • CAP Security Settings (by default enabled) • Certificate Store • Content Configuration • Database Configuration • EIS Repository Configuration • Email Configuration • Entity • Entity Data Audit • Entity Instance • History Log • Home Page Layout • LDAP • Organization Unit • OTDS Resource (by default enabled) • Rule • Schedule • Service Container • Solution • Trusted Application Publishers • Web Gateway • Web Service • XML Store Data type: Boolean Default value: None

19.2.45 LDAP

The applicable properties that can be set for configuring LDAP are provided in the sample configuration settings:

```
ldap:
  adminPassword: ${ldap.password:-admin}
  adminUserDn: cn=Directory Manager,o=${ldap.domain:-opentext.net}
  configPassword: ${ldap.config.password:-config}
  configUserDn: cn=admin,cn=config
  maxCacheSize: '10000'
  maxResultSize: '2000'
  rootDn: cn=cordys,cn=${instance.name:-defaultInst},o=${ldap.domain:-opentext.net}
  url: ldaps://${ldap.host:-ldap}:${ldap.service.port:-6366}
```

The following table describes various property names, their data types, the default values they accept, and their descriptions.

Property name	Description
ldap	Section to configure LDAP.
adminUserDn	Administrator user DN to connect to the LDAP server. Data type: String Default value: cn=Directory Manager,o=\${ldap.domain:-opentext.net}
adminPassword	The password for the LDAP administrator user, by default taken from the environment variable for <code>ldap.admin.password</code> . Data type: Password Default value: \${ldap.admin.password:-admin}
configUserDn	OpenLDAP admin user used to create a new database in the LDAP server. Data type: String Default value: cn=admin,cn=config
configPassword	Password for the LDAP config user, by default taken from the environment variable for <code>ldap.admin.password</code> . Data type: Password Default value: \${ldap.config.password:-config}

Property name	Description
maxCacheSize	Maximum size of the total LDAP cache. Data type: Integer Default value: 10000
maxResultSize	Maximum entries of a single LDAP query result. Data type: Integer Default value: 2000
rootDn	LDAP root DN for this AppWorks Platform instance. Data type: String Default value: cn=cordys,cn=\${instance.name:-defaultInst},o=\${ldap.domain:-opentext.net}
url	URL of the OpenLDAP server. Data type: String Default value: ldaps://\${ldap.host:-ldap}:\${ldap.service.port:-6366}

19.2.46 Generic service deployment

The applicable properties that can be set for configuring Generic service deployment are provided in the sample configuration settings:

```
platform:
  services:
    instances: ''
    <service>:
      kind: ${name}
      classpath: ''
      implementation: <no default value specified>
      packages:
        instances: ''
        <package>:
          methodSets: ''
      requestTimeout: PT30S
      validation: 'false'
```

The following table describes various property names, their data types, the default values they accept, and their descriptions.

Property name	Description
services	Section to configure services.

Property name	Description
instances	List of services that must be created for this deployment. When a service is not mentioned here, it will not be created even though there is a service definition for it.
<service>	Section to configure a non-specific service. This can be used to configure, for example a custom connector. All properties available for 'service' also can be used in the configuration of all specific services. Data type: String Default value: None
kind	Type of the application connector. If left out, the service name is used to determine the application connector. Data type: String Default value: None
classpath	Classpath to use for this service.  Note: The file path of the jars must be specified relative to the <AppWorks Platform_installdir>. Data type: String[] Default value: None
implementation	Package and class of the application connector main class. Data type: String Default value: None
packages	Section to configure packages handled by this service.
instances	Comma-separated list of all packages handled by this service. Only when specified in here, a package is handled by the service. Data type: String[] Default value: None
<package>	Section of a specific package.

Property name	Description
methodSets	List of all method sets for the package to be handled by this service. Data type: String[] Default value: None
requestTimeout	Time before a timeout error is returned for requests sent to this specific service. Data type: Duration Default value: PT30S
validation	Whether to validate the request and response sent to and from this service. Data type: Boolean Default value: false

19.2.47 MicroProfile Configuration metadata

The applicable properties that can be set for configuring MicroProfile Configuration metadata are provided in the sample configuration settings:

```
platform:
  configuration:
    organizationPattern: platform.organizations.${organizationName}.${key}
    pollInterval: PT1M
    variableOrdinal: '1000'
    yamlFiles: ''
```

The following table describes various property names, their data types, the default values they accept, and their descriptions.

Property name	Description
configuration	Section for MicroProfile Configuration meta configuration.
organizationPattern	Data type: String Default value: platform.organizations.\${organizationName}.\${key}
pollInterval	Interval to check if this configuration file has changed. Data type: Duration Default value: PT1M

Property name	Description
variableOrdinal	Ordinal of this configuration file. A higher value taking priority over a lower one. Data type: Integer Default value: 1000
yamlFiles	List of YAML files to load. This can be a file path relative to the AppWorks Platform config folder. Data type: String[] Default value:

19.2.48 Document viewer

AppWorks supports Brava and Intelligent Viewing (IV) for viewing documents stored in content management systems. Administrators can configure the required viewer using the `kind` property. The possible values for `kind` are `Brava` and `IntelligentViewing`. However, both viewers cannot be specified at the same time.

The applicable properties that can be set for configuring the viewers are provided in the sample configuration settings:

```
platform:
  content:
    viewers:
      <brava>:
        kind: Brava
        capabilitySet: <no default value specified>
        publishOnUpload: 'false'
        serverUrl: <no default value specified>
      <intelligentViewing>:
        kind: IntelligentViewing
        clientId: <no default value specified>
        clientSecret: <no default value specified>
        highlightServiceUrl: <no default value specified>
        markupServiceUrl: <no default value specified>
        publicationServiceUrl: <no default value specified>
        viewerServiceUrl: <no default value specified>
```

The following table describes various property names, their data types, the default values they accept, and their descriptions while using Brava Viewer as the document viewer.

Property name	Description
kind	Type of viewer to use. Data type: String Default value: Brava

Property name	Description
capabilitySet	Brava capability set, if customer already has a different Brava license other than the one shipped with AppWorks. Default is PS_BV, which enables AppWorks applications/users to use Brava as a viewer to view limited file types and compare such files. Data type: String Default value: PS_BV
publishOnUpload	Whether documents uploaded must be published to Brava cache. Publishing to Brava cache improves performance when the document is viewed using Brava. Data type: Boolean
serverUrl	Brava Server URL. If this is empty, Brava will not be configured. Data type: String

The following table describes various property names, their data types, the default values they accept, and their descriptions while using Intelligent Viewing as the document viewer.

Property name	Description
kind	Type of viewer to use. Data type: String Default value: IntelligentViewing
clientId	Client ID for oAuth authentication with Intelligent Viewing. Data type: String
clientSecret	Client secret for oAuth authentication with Intelligent Viewing. Data type: String
publicationServiceUrl	Publication service URL to publish documents. If this is empty, Intelligent Viewing will not be configured. Data type: String
viewerServiceUrl	Intelligent Viewing Server URL. If this is empty, Intelligent Viewing will not be configured. Data type: String

Property name	Description
highlightServiceUrl	Intelligent Viewing Highlight Server URL. When configured, enables the search button in Intelligent Viewer while viewing a document, which can be used for searching content. This is an optional property. Data type: String
markupServiceUrl	Intelligent Viewing Markup Service URL. When configured, enables the markup tools in Intelligent Viewer while viewing a document, which can be used to add annotations or redactions to the document. This is an optional property. Data type: String

19.2.49 Platform Repository

The applicable properties that can be set for the Platform Repository are provided in the sample configuration settings:

```
platform:
  organizations:
    <Organization Name>:
      services:
        instances: DocumentStoreForPlatform
        DocumentStoreForPlatform:
          kind: DocumentStore
      content:
        default: PlatformConfig
        PlatformConfig:
          kind: PlatformRepository
          connectionName: system
          rootFolder: MyOrg/AppWorks
          viewer: <no default value specified>
```

The possible values for PlatformConfig: > kind: are as follows:

- PlatformRepository - Document store will be configured with XDS or platform repository
- ContentServer - Document store will be configured with OpenText Content Server
- Documentum - Document store will be configured with OpenText Documentum
- ArchiveCenter - Document store will be configured with OpenText Archive Center
- CMIS - Document store will be configured with CMIS
- CoreShare - Document store will be configured with OpenText Core
- AppEnhancer- Document store will be configured with AppEnhancer

- Others - Document store will be configured with custom integrations

The following table describes various property names, their data types, the default values they accept, and their descriptions.

Property name	Description
kind	Type of content management system, which must be connected using document store. Data type: String Default value: PlatformRepository
connectionName	Configured data source name configured with XDS. Data type: String
rootFolder	Root folder path under which document store manages documents. Data type: String
viewer	Viewer to use as a document viewer. Data type: String Accepted values: Brava or IntelligentViewing

19.2.50 Content Server

The applicable properties that can be set for configuring Content Server are provided in the sample configuration settings:

```
platform:
  organizations:
    <Organization Name>:
      services:
        instances: DocumentStoreForCS
        DocumentStoreForCS:
          kind: DocumentStore
      content:
        default: ContentServerConfig
        ContentServerConfig:
          kind: ContentServer

      authenticationHandler: <Provide authentication handler>
      authenticationKind: OAuthHandler
      contentServerWebServicesUrl: http://cs.myorg.com:28080/cws/services/
DocumentManagement
  otdsResourceId: 94D99DED-F7D0-CB49-5EAC-615764959867
  rootFolder: MyOrg/AppWorks
  userForServiceStart: otadmin@otds.admin
  viewer: <no default value specified>
```

The following table describes various property names, their data types, the default values they accept, and their descriptions.

Property name	Description
kind	Type of content management system which needs to be connected using document store. Data type: String Default value: ContentServer
authenticationKind	Type any one of the three possible values based on the authentication type configured: • OtdsResource • OAuthClient • OAuthHandler Data type: String Default value:OtdsResource  Note: This is required only when the authenticationHandler authentication is configured.
authenticationHandler	Authentication handler name as configured in OTDS. Data type: String  Note: In scenarios where AppWorks Platform and Content Server are configured two different OTDS servers, for instance, in a hybrid deployment scenario, where AppWorks is deployed on the cloud and Content Server is deployed on-premise, you must use the Authentication Handler authentication type only. In all other deployment scenarios, you must use either the resource or oauth client-based authentication.
rootFolder	Root folder path under which document store manages documents. Data type: String
clientId	Client ID for oAuth authentication with Content Server, which is supported from Content Server 16.2.11 onward. Data type: String

Property name	Description
clientSecret	Client secret for oAuth authentication with Content Server, which is supported from Content Server 16.2.11 onward. Data type: String
otdsResourceId	OTDS resource ID configured in OTDS for Content Server authentication. This is required for OTDS resource-based authentication (with Content Server). Data type: String
contentServerWebServicesUrl	URL for Content Web services (CWS). Data type: String
userForServiceStart	User context in which the document store service performs operations when it starts. This user context is used only during starting of document store. Data type: String
officeOnlineDiscoveryUrl	Provide the Office Online discovery URL, if the discovery URL is different from the default discovery URL, <code>https://onenote.officeapps.live.com/hosting/discovery</code>. Set this to <code>none</code> if Content Server is not configured with Office Online or there is no Internet connectivity.  Note: Instead, you can also set the following property in the <code>wcp.properties</code> file: <code>com.opentext.documentstore.officeOnlineDiscoveryUrl = none</code>
viewer	Viewer to use as a document viewer. Data type: String Accepted values: Brava or IntelligentViewing

19.2.51 Content Server with Extended ECM

The applicable properties that can be set for configuring Content Server with Extended ECM are provided in the sample configuration settings:



Note: The Authentication handler authentication type is not supported for Extended ECM. It supports either the resource or oauth client-based authentication only.

```
platform:
  organizations:
    <Organization Name>:
      services:
        instances: DocumentStoreForxECM
        DocumentStoreForxECM:
          kind: DocumentStore
      content:
        default: ContentServerConfigForxECM
        ContentServerConfigForxECM:
          kind: ContentServer
          rootFolder: MyOrg/AppWorks
          clientId:
          clientSecret:
          otdsResourceId: 94D99DED-F7D0-CB49-5EAC-615764959867
          contentServerWebServicesUrl: http://cs.myorg.com:28080/cws/services/
DocumentManagement
  userForServiceStart: otadmin@otds.admin
  contentServerConfigurationName: AWPConnection
  contentServerSupportDirectoryUrl: http://cs.myorg.com:28080/img
  contentServerUrl: http://cs.myorg.com:28080/OTCS/cs.exe
  xecmSyncBusinessWorkspaceManually: true
  xecmWebServiceUrl: http://cs.myorg.com:28080/otsapxecm/services/ECMLink
viewer: <no default value specified>
```

The following table describes various property names, their data types, the default values they accept, and their descriptions.

Property name	Description
kind	Type of content management system, which must be connected using document store. Data type: String Default value: ContentServer
rootFolder	Root folder path under which document store manages documents. Data type: String
clientId	Client ID for oAuth authentication with Content Server, which is supported from Content Server 16.2.11 onward. Data type: String

Property name	Description
clientSecret	Client secret for oAuth authentication with Content Server, which is supported from Content Server 16.2.11 onward. Data type: String
otdsResourceId	OTDS resource ID configured in OTDS for Content Server authentication. This is required for OTDS resource-based authentication with Content server. Data type: String
contentServerWebServicesUrl	URL for Content Web services (CWS). Data type: String
userForServiceStart	User context in which the document store service performs operations when it starts. This user context is used only during starting of document store. Data type: String
contentServerConfigurationName	Name of the configuration done in Content Server. Data type: String
contentServerSupportDirectoryUrl	Content Sserver support directory URL. Data type: String
contentServerUrl	Content Server URL. Data type: String
xecmSyncBusinessWorkspaceManually	Specifies whether synchronization with Content Server must occur automatically. If the value is false, workspace creation and metadata updates occur automatically when the entity is created and updated. If the value is true, the user must perform the Synchronize workspace action to create the workspace and update metadata in Content Server. Data type: Boolean Default value: false
viewer	Viewer to use as a document viewer. Data type: String Accepted values: Brava or IntelligentViewing

19.2.52 Documentum

OpenText supports three types of authentication for Documentum: Basic, OAuthClient and AuthenticationHandler.

- The applicable properties that can be set for configuring Documentum when using the OAuthClient authentication are provided in the sample configuration settings:

```
platform:
  organizations:
    <OrganizationName>:
      services:
        instances: Document Store DCTM
        Document Store DCTM:
          kind: DocumentStore
      content:
        default: Document Store DCTM Configuration
        Document Store DCTM Configuration:
          kind: Documentum
          authenticationKind: OAuthClient
          clientId: <Documentum OAuth client ID>
          clientSecret: <Documentum OAuth client secret>
          mediaUrlPolicy: Default
          restEndpointUrl: http://dctm.myorg.com/dctm-rest/repositories/<DocBase>
          rootFolder: Documentum Root
          userForServiceStart: Admin
          viewer: <no default value specified>
```

- The applicable properties that can be set for configuring Documentum when using the Basic authentication are provided in the sample configuration settings:

```
platform:
  organizations:
    <OrganizationName>:
      services:
        instances: Document Store DCTM
        Document Store DCTM:
          kind: DocumentStore
      content:
        default: Document Store DCTM Configuration
        Document Store DCTM Configuration:
          kind: Documentum
          authenticationKind: Basic
          username: Administrator
          password: *****
          customHeaders:
            instances: Bus_NNI, Bus_ExchangeID
            Bus_NNI:
              type: Dynamic
              value: CurrentUser
            Bus_ExchangeID:
              type: Static
              value: Bus_ExchangeID__value
          mediaUrlPolicy: Default
          restEndpointUrl: http://10.96.74.85:8080/dctm-rest/repositories/corp
          rootFolder: JDoe
          userForServiceStart: Administrator
```

The following table describes various property names, their data types, the default values they accept, and their descriptions.

Property name	Description
kind	Type of content management system, which must be connected using document store. Data type: String Default value: Documentum
authenticationKind	Type of authentication. Data type: String Acceptable values: OAuthClient, OAuthHandler, or Basic
username	 Note: Configure this property only when authenticationKind is Basic. Mandatory. Name of the user as a plain text entry. Data type: String
password	 Note: Configure this property only when authenticationKind is Basic. Mandatory. Password for the username as a plain text entry. Data type: String

Property name	Description
customHeaders	 Note: Configure this property only when authenticationKind is Basic. Optional. customHeaders has a list of customHeaders provided as comma-separated list in instances. Each customHeader has three properties - • key - name of the customHeader as plain text entry. For example: Bus_NNI • type - type of customHeader, Static or Dynamic • Value - value of the customHeader type. – When the type of customHeader is Static, the value is entered by the admin user configuring it. – When the type of customHeader is Dynamic, the value of the customHeader accepts CurrentUser as the predefined value and the logged-in user's username is set as the value for the corresponding customHeader.
clientId	 Note: Configure this property only when authenticationKind is OAuthClient. Client ID for oAuth authentication with Documentum, which is supported from Content Server 16.2.11 onward. Data type: String
clientSecret	 Note: Configure this property only when authenticationKind is OAuthClient. Client secret for oAuth authentication with Documentum, which is supported from Content Server 16.2.11 onward. Data type: String  Note: Configure this property only when authenticationKind is OAuthClient.

Property name	Description
authenticationHandler	Authentication handler name as configured in OTDS. Data type: String  Note: Configure this property only when authenticationKind is OAuthHandler.
rootFolder	Root folder path under which document store manages documents. Data type: String
mediaUrlPolicy	Media UrlPolicy. The possible values are Local and Default. Data type: String
restEndpointUrl	Documentum REST service URL. Data type: String
userForServiceStart	User context in which the document store service performs operations, when it starts. This user context is used only during starting of document store. Data type: String
viewer	Viewer to use as a document viewer. Data type: String Accepted values: Brava or IntelligentViewing

19.2.53 Core Share

The applicable properties that can be set for configuring Core Share are provided in the sample configuration settings:

```
platform:
  organizations:
    <Organization Name>:
      services:
        instances: DocumentStoreForCore
        DocumentStoreForCore:
          kind: DocumentStore
      content:
        default: CoreShareConfig
        CoreShareConfig:
          kind: CoreShare
          rootFolder: MyOrg/AppWorks
          bindingType: Browser
          cmisBindingUrl: https://core.myorg.com/cm11/v1/browser
          cmisRepositoryId: CoreForMyOrg
          coreBaseUrl: https://unicorn.core.opentext.com
          authenticationHandler: <Authentication handler in AppWorks OTDS for Core>
```

```
username: user@myorg.com
password: myPassword123$
viewer: <no default value specified>
```

The following table describes various property names, their data types, the default values they accept, and their descriptions.

Property name	Description
kind	Type of content management system which must be connected using document store. Data type: String Default value: Core Share
rootFolder	Root folder path under which document store manages documents. Data type: String
authenticationHandler	Authentication handler name as configured in OTDS. Data type: String
bindingType	CMIS binding type. The possible values are: • AtomPub • Browser • Webservice Data type: String
cmisBindingUrl	CMIS binding URL based on the bindingType used. Data type: String
cmisRepositoryId	Repository ID configured in Core share. Data type: String
coreBaseUrl	URL of Core share. Data type: String
username	User name for the service account. Data type: String
password	Password for the service account. Data type: String

Property name	Description
viewer	Viewer to use as a document viewer. Data type: String Accepted values: Brava or IntelligentViewing

19.2.54 Archive Center

The applicable properties that can be set for configuring Archive Center are provided in the sample configuration settings:

```
platform:
  organizations:
    <Organization Name>:
      services:
        instances: DocumentStoreForOTAC
        DocumentStoreForOTAC:
          kind: DocumentStore
      content:
        default: ArchiveCentereConfig
        ArchiveCentereConfig:
          kind: ArchiveCenter
          rootFolder: MyOrg/AppWorks
          bindingType: Browser
          cmisBindingUrl: https://core.myorg.com/cm11/v1/browser
          cmisRepositoryId: AWPRepo
          datasourceId: aff91331-5048-43c4-b203-bb24087e93e7
          otdsResourceId: 150555f8-6882-4bea-8ce7-8917c4f75ad1
          viewer: <no default value specified>
```

The following table describes various property names, their data types, the default values they accept, and their descriptions.

Property name	Description
kind	Type of content management system, which must be connected using document store. Data type: String Default value: ArchiveCenter
rootFolder	Root folder path under which document store manages documents. Data type: String
bindingType	CMIS binding type. The possible values are: • AtomPub • Browser • Webservice Data type: String

Property name	Description
cmisBindingUrl	CMIS binding URL based on the bindingType used. Data type: String
cmisRepositoryId	Repository ID configured in Core share. Data type: String
datasourceId	Data source ID configured in Archive Center. Data type: String
otdsResourceId	ID of OTDS resource for Archive Center. Data type: String
viewer	Viewer to use as a document viewer. Data type: String Accepted values: Brava or IntelligentViewing

19.2.55 CMIS

The applicable properties that can be set for configuring CMIS are provided in the sample configuration settings:

```
platform:
  organizations:
    <Organization Name>:
      services:
        instances: DocumentStoreForCMIS
        DocumentStoreForCMIS:
          kind: DocumentStore
      content:
        default: CMISConfig
        CMISConfig:
          kind: CMIS
          rootFolder: MyOrg/AppWorks
          bindingType: Browser
          cmisBindingUrl: https://core.myorg.com/cmis11/v1/browser
          cmisRepositoryId: AWPrepo
          username: user@myorg.com
          password: myPassword123$
          viewer: <no default value specified>
```

The following table describes various property names, their data types, the default values they accept, and their descriptions.

Property name	Description
kind	Type of content management system, which must be connected using document store. Data type: String Default value: CMIS

Property name	Description
rootFolder	Root folder path under which document store manages documents Data type: String
bindingType	CMIS binding type. Possible values are: • AtomPub • Browser • Webservice Data type: String
cmisBindingUrl	CMIS binding URL based on the bindingType used. Data type: String
cmisRepositoryId	Repository ID configured in Core share. Data type: String
username	User name Data type: String
password	Password for user Data type: String
viewer	Viewer to use as a document viewer. Data type: String Accepted values: Brava or IntelligentViewing

19.2.56 AppEnhancer

The applicable properties that can be set for configuring AppEnhancer are provided in the sample configuration settings:

```
platform:
  organizations:
    <Organization Name>:
      services:
        instances: DocumentStoreForAE
        DocumentStoreForAE:
          kind: DocumentStore
      content:
        default: AEConfig
        AEConfig:
          kind: AppEnhancer
          clientId:
          clientSecret:
          dataSourceName: AppEnhancerDEMO
          restEndpointUrl: http://<AE_Server>/AppEnhancerReST/api/AEDataSources
          viewer: <no default value specified>
```


The following table describes various property names, their data types, the default values they accept, and their descriptions.

Property name	Description
kind	Type of content management system, which must be connected using document store. Data type: String Default value: AppEnhancer
clientID	Type the client ID of AppWorks Platform that is registered in the OTDS of AppEnhancer. Data type: String
clientSecret	Type the client secret of AppWorks Platform that is registered in the OTDS of AppEnhancer. Data type: String
dataSourceName	Type the name of the datasource in AppEnhancer. Data type: String
restEndpointUrl	Type the AppEnhancer REST Endpoint. For example, <code>http://<AppEnhancer hostname>:<portnumber>/AppEnhancerReST</code> Data type: String
viewer	Viewer to use as a document viewer. Data type: String Accepted values: Brava or IntelligentViewing

19.2.57 Others

The applicable properties that can be set for configuring other repositories are provided in the sample configuration settings:

```
platform:
  organizations:
    <Organization Name>:
      services:
        instances: DocumentStoreForCustom
        DocumentStoreForCustom:
          kind: DocumentStore
      content:
        default: CustomConfig
        CustomConfig:
          kind: Others
```

```

    rootFolder: MyOrg/AppWorks
    implementationClass:
com.documentstore.plugin.custom.CustomDocumentStoreHandler
    contentRepositoryConfiguration: /opt/appworks/config/extConfiguration.xml
    viewer: <no default value specified>

```

The following table describes various property names, their data types, the default values they accept, and their descriptions.

Property name	Description
kind	Type of content management system, which needs to be connected using document store. Data type: String Default value: Others
rootFolder	Root folder path under which document store manages documents. Data type: String
implementationClass	Fully qualified name of connector implementation class. Data type: String
contentRepositoryConfiguration	XML configuration for the custom repository as expected by connector implementation. Data type: String
viewer	Viewer to use as a document viewer. Data type: String Accepted values: Brava or IntelligentViewing

19.2.58 CORS

The applicable properties that can be set for enabling CORS are provided in the sample configuration settings:

```

platform:
  security:
    corsFilter:
      enabled: false # default false
      allowedOrigins: * # default all origins otherwise configure list of origins comma
separated
      allowedMethods: GET,POST,HEAD,OPTIONS,PUT,DELETE
      allowedHeaders: SAMLart,Content-Type,X-Requested-With,accept,Origin,Access-Control-
Request-Method,Access-Control-Request-Headers
      exposedHeaders: Access-Control-Allow-Origin,Access-Control-Allow-Credentials
      supportsCredentials: false
      preflightMaxAge: 10

```

The following table describes various property names, their data types, the default values they accept, and their descriptions.

Property name	Description
enabled	Enable this filter when other Application clients access the AppWorks Services. Data type: Boolean Default: false
allowedOrigins	List of comma separated cross origins that can access the AppWorks Services. Data type: String Default value: '*' allows all the origins.
allowedMethods	List of HTTP methods supported by cross origin request. Data type: String
allowedHeaders	List of headers supported by cross origin request.
exposedHeaders	List of headers sent by cross origin response.
supportsCredentials	To set the Access-Control-Allow-Credentials value to true in a cross origin response.
preflightMaxAge	To set the Access-Control-Max-Age value in a cross origin response.

19.2.59 Content Security Policy (CSP)

Content Security Policy is provided by using the mpc settings given below:

```
platform:
  security:
    contentSecurityPolicy: frame-ancestors 'self'
```

The default value is `frame-ancestors 'self'`, it is a standard HTTP header. Refer <https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP> for more information on the possible values of the policies.

To allow specific domains, provide the Content security policy as given below:

```
platform:
  security:
    contentSecurityPolicy: frame-ancestors 'self' https://example.com https://store.example.com
```



Note: In a xGov Integrated Environment that uses the AppWorks Platform Container Instance, add the CS and the OTDS domains to the `contentSecurityPolicy` as shown in the example below.

```
platform:
  security:
```

```
contentSecurityPolicy: frame-ancestors 'self' <CS_HostName:CS_port>
<OTDS_HostName:OTDS_port> platform: security: contentSecurityPolicy: frame-ancestors
'self' <CS_HostName:CS_port> <OTDS_HostName:OTDS_port>
```

19.2.60 Organization Creation

```
platform:
  organizations:
    instances: <Organization name>
    <Organization name>:
      adminUsers: <organization administartor users>
```

Property name	Description
instances	Names of the organizations to be created. Only the organizations mentioned in this property are created. Data type: String Default value: none
<Organization name>	Name of the organization.
adminUsers	Names of the users to be assigned as the Organization Administrators.  Note: Provide at least one valid user name. Data type: String Default value: none

19.2.61 Core Signature

The applicable properties that can be set for enabling core signature are provided in the sample configuration settings:

```
platform:
  organizations:
    <Organization Name>:
      signature:
        coreSignature:
          kind: CoreSignature
          baseUrl:
          siteId:
          authenticationUrl:
          clientId:
          clientSecret:
          ocp: <ocp-name>
      opentextCloudPlatform:
        <ocp-name>:
          authenticationUrl:
          clientId:
          clientSecret:
```

Property name	Description
kind	Core signature. Data type: String Default value: none
baseUrl	Base Url of the core signature. Data type: String Default value: none
SiteId	The site id of the core signature. Data type: String Default value: none
authenticationUrl	The authentication url of the core signature (the va Data type: String Default value: none
clientId	The client id of the core signature (the value provid Data type: String Default value: none
clientSecret	The client secret of the core signature (the value pr Data type: String Default value: none
ocp	The ocp configuration you want to use. Data type: String Default value: none

You can download the signed document automatically by configuring Core signature to call back AppWorks Platform once the document is signed, this is the default behavior or AppWorks Platform continuously checks or polls Core signature to check the status of the documents.

This can be configured in `extraEnv`, by setting the properties given below:

- To enable polling:
Set `com.opentext.esignature.polling.enabled=` as `true` and provide the polling interval by setting `com.opentext.esignature.polling.interval=<interval in minutes>`.
For example:

```
extraEnv:
  -name:CATALINA_OPTS
  value: -Dcom.opentext.esignature.polling.enabled=true -
Dcom.opentext.esignature.polling.interval=10
```

- To enable callback:

Set `com.opentext.esignature.eventcallback.enabled=` as `true` and `com.opentext.esignature.polling.enabled=` as `false`

For example:

```
extraEnv:
  -name:CATALINA_OPTS
  value: -Dcom.opentext.esignature.polling.enabled=false -
Dcom.opentext.esignature.eventcallback.enabled=true
```

19.2.62 Core Content

The applicable properties that can be set for enabling core content are provided in the sample configuration settings:

```
platform:
  organizations:
    <OrganizationName>:
      opentextCloudPlatform:
        <ocp-name>:
          authenticationUrl:
          clientId:
          clientSecret:
          siteId:
        coreContent:
          <core-content-name>:
            ocp: <ocp-name>
            businessObjectUrl:
            metadataUrl:
            storageUrl:
            widgetUrl:
            baseUrl:
            workspaceUrl:
            syncWorkspaceManually:
```

Property name	Description
businessObjectUrl	The Business Object Url of Core Content. Data type: String Default value: none
metadataUrl	The Metadata Url of Core Content. Data type: String Default value: none
storageUrl	The Storage Url of Core Content. Data type: String Default value: none

Property name	Description
widgetUrl	The Widget Url of Core Content. Data type: String Default value: none
baseUrl	The Base Url of Core Content. Data type: String Default value: none
workspaceUrl	The Workspace Url of Core Content. Data type: String Default value: none
syncWorkspaceManually	Boolean option to specify regarding syncing of Workspace M Data type: String Default value: none
ocp	The Ocp configuration provided under opentextCloudPlatfo The properties authenticationUrl, clientId, clientSecret, and

To configure Document Store for Core Content

The sample configuration settings for creating a Document Store Service container with Store Type as Core Content are provided below:

```
platform:
  organizations:
    <OrganizationName>:
      services:
        instances: Document Store CoreContent
        Document Store CoreContent:
          kind: DocumentStore
      content:
        default: DocumentStoreCoreContent
        DocumentStoreCoreContent:
          kind: CoreContent
          coreContent: <core-content-name>
```

Property name	Description
kind	Type of content management system which must be connect Data type: String Default value: CoreContent

Property name	Description
coreContent	This refers to the configuration name which is provided in opentextC this reference. Data type: String Default value: none

To configure Admin MPC for Core Content

The applicable properties that can be set for enabling core content are provided in the sample configuration settings:

```
platform:
  organizations:
    <organization name>:
      entityRuntime:
        solutions:
          <project name>:
            entities:
              <entity name>:
                businessWorkspace:
                  workspaceOperation: <no default value specified>
                  workspaceType: <no default value specified>
                  workspaceTemplate: <no default value specified>
                  workspaceName: <no default value specified>
                  coreContentPropertyMapping:
                    - entityPropertyName: <no default value specified>
                      workspaceAttributeName: <no default value specified>
                      workspaceAttributeDisplayName: <no default value specified>
```

Property name	Description
workspaceOperation	Possible values: • CreateNew: to create a new workspace. • None
workspaceType	Provide the internal name or workspace-item-id of the Workspace Type  Notes • This parameter is mandatory. • No two entity should be mapped to the same workspace type when the entities are in different AppWorks Platform instance
workspaceTemplate	Provide the internal name of the workspace template created in Core
workspaceName	You can provide a pattern to compose the name of the workspace instance Patterns can include entity properties (as [Properties.Propertyname]) instance id as the name. For example, to use the value of entity property named Property1 and Property2, the pattern would be '[Properties.Property1] -[Properties.Property2]'.  Note: If a name is not provided then the entity instance Id becomes the name.

Property name	Description
coreContentPropertyMapping	You can map the entity properties to the Business workspace.  Note: While mapping the entity instance with the Core workspace type and AppWorks Platform entity should
entityPropertyName	Provide name of the entity property which is selected for Co
workspaceAttributeName	Provide the internal name of the workspaceAttribute which
workspaceAttributeDisplayName	Provide the display name of the workspaceAttribute which

Publishing the Core content Business Object Configuration in PE Admin

In container deployment:

- To use Core content, enable Container deployment in PE Admin.
- The PE Admin page displays all the Core content Business Object Configurations and mappings in read only format.
- The Publish action button can be enabled only through PE Admin page. The administrator must select **Publish** to use the Core Content features

In non-container deployment when yml configuration is enabled:

- The PE Admin page displays all the Core content Business Object Configurations and mappings in read only format.
- The Publish action button can be enabled only through PE Admin page. The administrator must select **Publish** to use the Core Content features.



Notes

- To get internal names for property mapping, the administrator must invoke the URL:

```
<appworks-protocol><appworks-host>:<appworks-port>/home /
<organization-name>/app/documentsservices/rest/admin/core-content /
workspace/types/<workspace-internal-name or workspace-item-id>
```

For example:

```
http://10.96.74.104:8080/home/mpcOrg/app/documentsservices/rest /
admin/core-content/workspace/types/sriwrktype_6oi92
```

- To get internal names for workspaceTemplate, the administrator must invoke the URL and copy as follows :

```
<appworks-protocol><appworks-host>:<appworks-port>/home /
<organization-name>/app/documentsservices/rest/admin/core-content /
workspace/types/<workspace-internal-name or workspace-item-id> /
templates
```

For example:

```
http://10.96.74.104:8080/home/mpcOrg/app/documentservices/rest/  
admin/core-content/workspace/types/sriwrktype_6oi92/templates
```

Chapter 20

Appendix B – Disaster recovery

20.1 Prerequisites

Following are the prerequisites for disaster recovery in the containerized mode.

- The disaster recovery site does not need to be set up for the containerized mode before a disaster occurs.
- You must have access to the databases and OpenLDAP used by the containers or have access to a replicated copy of the original database instances and LDAP. To replicate the database and LDAP, see the corresponding vendor documentation for a detailed procedure.
- Back up the following items to set up the production site:
 - Latest AppWorks Platform image on which the production site is running
 - Configurations provided in the environment.
 - Instance name
 - Credentials for the administrator
 - Kubernetes or Docker-compose scripts

20.2 Recovering the production site to the disaster recovery site

User configuration and application packages are already present in the Docker image.

To recover the production site to the disaster recovery site:

1. Edit the backup Kubernetes or Docker-compose scripts.
 - Update LDAP and database related environment variables, if required.
 - Add the new license information if you are using the new replicated LDAP server.
2. To start the cluster, see [“Deploying Application images on Docker” on page 61.](#)

Chapter 21

Appendix C – Firewall issues when using Docker on Linux

When running Docker containers on a Linux host, issues with the firewall might occur, which block communication between the containers. This can occur because some Linux versions use firewalls, which do not propagate all nftables to iptables.

Enabling firewalls to use masquerading resolves this issue. For more information, see Docker iptables documentation.

21.1 Executing firewall commands

Following are the firewall commands to execute.

```
# Masquerading allows for docker ingress and egress
firewall-cmd --zone=public --add-masquerade --permanent

# Reload firewall to apply permanent rules
firewall-cmd --reload

# restart firewall
service firewalld restart

# restart docker
service docker restart
```

21.2 Verifying that masquerading is successful

To verify that masquerading is successful:

- Open `public.xml` in `/etc/firewalld/zones/`. The file must contain the `<masquerade/>` tag.

```
<?xml version="1.0" encoding="utf-8"?>
<zone>
  <short>Public</short>
  <description>For use in public areas. You do not trust the other computers on
networks to not harm your computer. Only selected incoming connections are
accepted.</description>
  <service name="dhcpv6-client"/>
  <service name="ssh"/>
  ...
  <masquerade/>
</zone>
```


Chapter 22

Appendix D – Deploying applications with External Secrets Operator

External Secrets Operator is a Kubernetes operator that integrates external secret management systems. The operator reads information from external APIs and automatically injects values into a Kubernetes Secret.

The following are the examples of how you can use External Secrets Operator:

- “Configuring AppWorks Platform to use Vault” on page 279
- “Configuring AppWorks Platform to use AWS Secrets Manager” on page 280
- “Configuring AppWorks Platform to use Azure Key Vault” on page 281
- “Configuring AppWorks Platform to use GCP Secret Manager” on page 283

For information about configuring External Secrets Operator for different external management systems, refer to the External Secrets Operator documentation (<https://external-secrets.io/>).

22.1 Configuring AppWorks Platform to use Vault

You can deploy applications using Hashicorp Vault. To configure Hashicorp Vault, refer to the documentation on the Hashicorp Vault website.

To configure AppWorks Platform to use Vault:

1. Download `appworks-platform-23.4-containerization.tar.gz` and extract it to a folder. The name of this folder will be used as a prefix for the appworks, ldap, and database container names.
2. Navigate to the `helm-charts/appworks` folder.
3. In the `values.yml` file, update the template values in the external secrets and secret store sections for appworks as shown in the example below:

```
useExternalSecrets: true
createSecretStore: true
useSecretStore: true
externalSecretSpec:
  secretStoreRef:
    name: secretstore #secret store name
    kind: SecretStore
  target:
    name: appworks
  dataFrom:
    - extract:
        key: <secret-path> Ex: secret/data/dev/appworks
secretStoreSpec:
  provider:
    vault:
```

```
server: <vault server url>
auth:
  kubernetes:
    mountPath: kubernetes
    role: appworks-role #vault access role
```

4. Update the external secrets section for ldap as shown in example below:

```
ldap:
  ...
  useExternalSecrets: true
  externalSecretSpec:
    secretStoreRef:
      name: secretstore # secret store name
      kind: SecretStore
    target:
      name: ldap-secrets
    dataFrom:
      - extract:
          key: <secrets-path> Ex:secret/data/dev/ldap
  ...
```



Note: As the deployment is performed with the Vault enabled, the following properties can be ignored in the helm-chart: `appworks.password`, `appworks.keystorePassword`, `ldap.adminPassword`, `ldap.configPassword`, `database.password`, and `database.adminPassword`

5. Deploy AppWorks Platform using the following command:
`helm install appworks ./appworks`

After all the pods are running and ready, you can start using AppWorks Platform.

22.2 Configuring AppWorks Platform to use AWS Secrets Manager

You can deploy applications using AWS Secrets Manager. To configure AWS secrets manager to access secrets, refer to the documentation on the AWS Secrets Manager website.

To configure AppWorks Platform to use AWS Secrets Manager:

1. Download `appworks-platform-23.4-containerization.tar.gz` and extract it to a folder. The name of this folder will be used as a prefix for the `appworks`, `ldap`, and `database` container names.
2. Navigate to the `helm-charts/appworks` folder.
3. In the `values.yml` file, update `useExternalSecrets` to `true`.

```
global:
  ...
  useExternalSecrets:      &use_external_secrets      true
  ...
```

4. Update the `aws.yml` file with the AWS Secret Manager information as shown in the example below:


```
externalSecretSpec:
  secretStoreRef:
    name: &secret_store_name secret-store #name of the secret store
    kind: &secret_store_kind SecretStore
  dataFrom:
    - extract:
        key: <secret-path> #Example: secrets/appworks/platform
secretStoreSpec:
  provider:
    aws:
      service: SecretsManager
      region: ap-south-1 #region
      auth:
        jwt:
          serviceAccountRef:
            name: external-secret #service account name
            namespace: default #namespace
  ldap:
    externalSecretSpec:
      secretStoreRef:
        name: *secret_store_name
        kind: *secret_store_kind
      dataFrom:
        - extract:
            key: <ldap-secret-path> #Example: secrets/appworks/platform
```



Note: For ClusterSecretStore, enter the namespace under serviceAccountRef tag. See [“Configuration files for Helm charts” on page 32](#) for more information on the contents in various values.yaml and aws.yaml files

5. Deploy AppWorks Platform using the following command: `helm install appworks ./appworks`

```
helm install appworks ./appworks
```

After all the pods are running and ready, you can start using AppWorks Platform.

22.3 Configuring AppWorks Platform to use Azure Key Vault

You can deploy applications using Azure Key Vault. To configure Azure Key Vault, refer to the documentation on the Azure Key Vault website.

To configure AppWorks Platform to use Azure Key Vault:

1. Download `appworks-platform-23.4-containerization.tar.gz` and extract it to a folder. The name of this folder will be used as a prefix for the appworks, ldap, and database container names.
2. Navigate to the `helm-charts/appworks` folder.
3. In the `values.yml` file, update `useExternalSecrets` to `true`.

```
global:
  ...
useExternalSecrets:    &use_external_secrets    true
  ...
```

4. Update the external secrets section for ldap as shown in example below:

```

externalSecretSpec:
  secretStoreRef:
    name: &secret_store_name secret-store #name of the secret store
    kind: &secret_store_kind SecretStore
  data:
    - secretKey: DATABASE_ADMIN_PASSWORD
      remoteRef:
        key: DATABASE-ADMIN-PASSWORD
    - secretKey: DATABASE_PASSWORD
      remoteRef:
        key: DATABASE-PASSWORD
    - secretKey: KEYSTORE_PASSWORD
      remoteRef:
        key: KEYSTORE-PASSWORD
    - secretKey: LDAP_ADMIN_PASSWORD
      remoteRef:
        key: LDAP-ADMIN-PASSWORD
    - secretKey: LDAP_CONFIG_PASSWORD
      remoteRef:
        key: LDAP-CONFIG-PASSWORD
    - secretKey: LICENSE_CUSTOMER_NAME
      remoteRef:
        key: LICENSE-CUSTOMER-NAME
    - secretKey: LICENSE_KEY
      remoteRef:
        key: LICENSE-KEY
    - secretKey: LICENSE_SITE_NAME
      remoteRef:
        key: LICENSE-SITE-NAME
    - secretKey: PLATFORM_PASSWORD
      remoteRef:
        key: PLATFORM-PASSWORD
secretStoreSpec:
  provider:
    azurekv: # azure tenant ID, see: https://docs.microsoft.com/en-us/azure/active-
directory/fundamentals/active-directory-how-to-find-tenant
    tenantId: "" # URL of your vault instance, see: https://docs.microsoft.com/en-
us/azure/key-vault/general/about-keys-secrets-certificates
    vaultUrl: ""
    authSecretRef: # points to the secret that contains the azure service
principal credentials
    clientId:
      name: <azure-secret> #name of the secret created with clientId and
clientSecret.
      key: ClientID
    clientSecret:
      name: <azure-secret> #name of the secret created with clientId and
clientSecret.
      key: ClientSecret
  ldap:
    externalSecretSpec:
      secretStoreRef:
        name: *secret_store_name
        kind: *secret_store_kind
      data:
        - secretKey: LDAP_ADMIN_PASSWORD
          remoteRef:
            key: LDAP-ADMIN-PASSWORD
        - secretKey: LDAP_CONFIG_PASSWORD
          remoteRef:
            key: LDAP-CONFIG-PASSWORD

```



Note: For ClusterSecretStore, enter the namespace under `clientId` and `clientSecret` tags. See “[Configuration files for Helm charts](#)” on page 32 for more information on the contents in various `values.yaml` and `azure.yaml` files.

5. Deploy AppWorks Platform using the following command:
`helm install appworks ./appworks`

After all the pods are running and ready, you can start using AppWorks Platform.

22.3.1 Configuring AppWorks Platform to use GCP Secret Manager

You can deploy applications using GCP Secrets Manager. To configure GCP secrets manager to access secrets, refer to the documentation on the GCP Secrets Manager website.

To configure AppWorks Platform to use Vault:

1. Download `appworks-platform-23.4-containerization.tar.gz` and extract it to a folder. The name of this folder will be used as a prefix for the appworks, ldap, and database container names.
2. Navigate to the `helm-charts/appworks` folder.
3. In the `values.yml` file, update the `useExternalSecrets` to `true`.

```
global:
...
useExternalSecrets:    &use_external_secrets    true
...
```

4. Update the `gcp.yml` file with GCP Secret Manager information as shown in the example below:

```
externalSecretSpec:
secretStoreRef:
  name: &secret_store_name secret-store #name of the secret store
  kind: &secret_store_kind SecretStore
data:
- secretKey: DATABASE_ADMIN_PASSWORD
  remoteRef:
    key: DATABASE_ADMIN_PASSWORD
- secretKey: DATABASE_PASSWORD
  remoteRef:
    key: DATABASE_PASSWORD
- secretKey: KEYSTORE_PASSWORD
  remoteRef:
    key: KEYSTORE_PASSWORD
- secretKey: LDAP_ADMIN_PASSWORD
  remoteRef:
    key: LDAP_ADMIN_PASSWORD
- secretKey: LDAP_CONFIG_PASSWORD
  remoteRef:
    key: LDAP_CONFIG_PASSWORD
- secretKey: LICENSE_CUSTOMER_NAME
  remoteRef:
    key: LICENSE_CUSTOMER_NAME
- secretKey: LICENSE_KEY
  remoteRef:
    key: LICENSE_KEY
- secretKey: LICENSE_SITE_NAME
  remoteRef:
    key: LICENSE_SITE_NAME
- secretKey: PLATFORM_PASSWORD
  remoteRef:
    key: PLATFORM_PASSWORD
```

```
secretStoreSpec:
  provider:
    gcpsm:
      auth:
        secretRef:
          secretAccessKeySecretRef:
            name: <secret-name> # secret name containing SA key
            key: <key-name> # key name containing SA key
      projectID: <project ID> #GCP project id
  ldap:
    externalSecretSpec:
      secretStoreRef:
        name: *secret_store_name
        kind: *secret_store_kind
      data:
        - secretKey: LDAP_ADMIN_PASSWORD
          remoteRef:
            key: LDAP_ADMIN_PASSWORD
        - secretKey: LDAP_CONFIG_PASSWORD
          remoteRef:
            key: LDAP_CONFIG_PASSWORD
```



Note: For ClusterSecretStore, enter the namespace under secretAccessKeySecretRef tag. See [“Configuration files for Helm charts” on page 32](#) for more information on the contents in various values.yaml and gcp.yaml files.

5. Deploy AppWorks Platform using the following command:
helm install appworks ./appworks

After all the pods are running and ready, you can start using AppWorks Platform.

Chapter 23

Appendix E – Setting up High Availability with LDAP pods

This topic lists steps to set up High Availability (HA) with LDAP pods and upscale or downscale LDAP pods. For a fresh installation, HA is automatically set up if the `replicaCount` of LDAP is greater than one.



Note: The maximum supported value of LDAP replica is 2.

23.1 Fresh installation with LDAP in High Availability mode

For a fresh installation of AppWorks Platform with LDAP in High Availability mode, update the helm chart with the details below:

- Set AppWorks Platform `replicaCount` to 0
- Set `ldap.replicaCount` to 2
- Add `ldap.sessionAffinity` and set the value to true
- Set `-Dawp.start.delay=30000` under `extraEnv` for parameter `CATALINA_OPTS`

Deploy AppWorks and after both the LDAP pods are up and running, you must upscale AppWorks Platform to the required `replicaCount` by performing the following steps after all your LDAP pods are live and ready.

1. Ensure that the sync replication is complete by waiting until you get the *Setting up HA DONE* message in `setup-ha.log` file. You can run the following commands to check the replication status:

- `kubectl exec -it ldap-0 -- cat /opt/cars/service/setup-ha.log`
- `kubectl exec -it ldap-1 -- cat /opt/cars/service/setup-ha.log`

2. Upscale AppWorks Platform to the required `replicaCount`.

```
kubectl scale statefulsets appworks --replicas=<n>
```

Example:

```
kubectl scale statefulsets appworks --replicas=2
```

23.2 Setting LDAP High Availability on an existing AppWorks Platform Instance

Prerequisites

Ensure the following:

- AppWorks Platform 22.4 or later version is installed on Kubernetes with one or more LDAP pods.
- All the AppWorks Platform and LDAP pods are ready and live.

Modify AppWorks helm chart and set the following values:

- Set AppWorks Platform `replicaCount` to 0
- Set `ldap.replicaCount` to 2
- Add `ldap.sessionAffinity` and set the value to `true`
- Set `-Dawp.start.delay=30000` under `extraEnv` for parameter `CATALINA_OPTS`

Upgrade using `helm upgrade` to apply these changes:

```
helm upgrade <NAME> ./appworks
```

Example: `helm upgrade appworks ./appworks`

23.2.1 Setting up HA after upscaling the pods

You can set up high availability with LDAP pods after upscaling the pods.

1. Create a folder in shared volume for each of the pods:

```
kubectl exec -it <ldap-pod-name> -- mkdir -p /opt/cars/shared/<ldap-pod-name>
```

Example:

```
(for ldap-0): kubectl exec -it ldap-0 -- mkdir -p /opt/cars/shared/ldap-0
```

```
(for ldap-1): kubectl exec -it ldap-1 -- mkdir -p /opt/cars/shared/ldap-1
```

2. Copy the certificates from both the ldap pods to the shared volume.

Example:

```
(for ldap-0): kubectl exec -it ldap-0 -- cp /opt/cars/var/certs/ca.crt /opt/cars/shared/ldap-0/ kubectl exec -it ldap-0 -- cp /opt/cars/var/certs/ldap.crt /opt/cars/shared/ldap-0/ kubectl exec -it ldap-0 -- cp /opt/cars/var/certs/ldap.key /opt/cars/shared/ldap-0/
```

```
(for ldap-1): kubectl exec -it ldap-1 -- cp /opt/cars/var/certs/ca.crt /opt/cars/shared/ldap-1/ kubectl exec -it ldap-1 -- cp /opt/cars/var/certs/ldap.crt /opt/cars/shared/ldap-1/ kubectl exec -it ldap-1 -- cp /opt/cars/var/certs/ldap.key /opt/cars/shared/ldap-1/
```

3. Create a backup from the first LDAP (ldap-0) pod and copy it to the local filesystem as follows:

- a. Create the backup using the following command: `kubectl exec ldap-0 -- /opt/cars/sbin/slapd -T cat -n 1 -F /opt/cars/var/slapd.d -l backup.ldif`
 - b. Copy backup from pod to local file system using the following command: `kubectl cp ldap-0:/opt/cars/backup.ldif backup.ldif`
4. Import the data from backup to the second LDAP (ldap-1) pod as follows:
 - a. Using any text editing tool, delete the first root entry from the backup.ldif file.
 - b. Copy the backup from local file system to the second LDAP pod using the following command: `kubectl cp backup.ldif ldap-1:/opt/cars/backup.ldif`
 - c. Import data from the backup.ldif file using the following command: `kubectl exec ldap-1 -- /opt/cars/sbin/slapd -T add -n 1 -F /opt/cars/var/slapd.d -q -l /opt/cars/backup.ldif`
5. Run the script in each LDAP pod: `/opt/cars/service/setup-ha.sh` and wait till the execution is completed.

```
kubectl exec -it ldap-0 -- /opt/cars/service/setup-ha.sh -o -r 2
```

```
kubectl exec -it ldap-1 -- /opt/cars/service/setup-ha.sh -o -r 2
```

23.2.2 Upscale AppWorks to required number of pods

To upscale the AppWorks pods, use the following command:

```
kubectl scale statefulsets appworks --replicas=<number of appworks pods>
```

Example:

```
kubectl scale statefulsets appworks --replicas=2
```


Chapter 24

Appendix F— Regenerating the expired LDAP certificates in the container

You can regenerate the expired LDAP certificates as follows:

1. Login inside the LDAP pod

```
kubect1 exec -it ldap-0 bash
```

2. Set the environments variables as follows:

```
export LD_LIBRARY_PATH=/opt/cars/lib:/opt/cars/lib/sasl:$LD_LIBRARY_PATH
export PATH=/opt/cars/sbin:$PATH
export SASL_PATH=/opt/cars/lib
export OPENSSL_CONF=/opt/cars/sbin/openssl.cnf
```

3. Generate the certificates using the following command:

```
openssl genrsa -des3 -passout pass:0pentext1! -out /opt/cars/var/certs/ca.key 4096
openssl req -new -x509 -key /opt/cars/var/certs/ca.key -passin pass:0pentext1! -
out /opt/cars/var/certs/ca.crt -subj "/C=IN/ST=Telangana/L=Hyderabad/
O=OpenLDAPContainer/CN=ldap-0"
openssl genrsa -out /opt/cars/var/certs/ldap.key 4096
openssl req -new -key /opt/cars/var/certs/ldap.key -out /opt/cars/var/certs/
ldap.csr -subj "/C=IN/ST=Telangana/L=Hyderabad/O=OpenLDAPContainer/CN=ldap-0"
openssl x509 -req -in /opt/cars/var/certs/ldap.csr -CA /opt/cars/var/certs/ca.crt -
CAkey /opt/cars/var/certs/ca.key -passin pass:0pentext1! -out /opt/cars/var/certs/
ldap.crt -CAcreateserial -days 1825 -sha256
```

The LDAP certificates are regenerated.

